

FlashAttention 硬件加速器 IP — 讲义

第 0 章：项目总览

讲义生成器

2026-04-19

目录

1 第 0 章：项目总览	2
1.1 0.1 项目背景与目标	2
1.2 0.2 顶层架构图	3
1.3 0.3 目录结构与讲义章节安排	4
1.4 0.4 验证与性能结果摘要	5
2 第 1 章：Baseline RTL — 计算通路	6
2.1 1.1 rtl/include/fa_params.svh	6
2.2 1.2 rtl/flash_attention_top.sv	9
2.3 1.3 rtl/tile_controller.sv	14
2.4 1.4 rtl/compute_core.sv	20
2.5 1.5 rtl/dot_product_array.sv	26
2.6 1.6 rtl/causal_mask_unit.sv	29
2.7 1.7 rtl/online_softmax_unit.sv	30
2.8 1.8 rtl/exp_approx_unit.sv	34
2.9 1.9 rtl/exp_lut_rom.sv	37
2.10 1.10 rtl/reciprocal_unit.sv	38
2.11 1.11 rtl/output_accumulator.sv	41
3 第 2 章：Baseline RTL — 存储与总线	47
3.1 2.1 rtl/buffer_system.sv	47
3.2 2.2 rtl/dma_engine.sv	51
3.3 2.3 rtl/axi4_lite_slave.sv	56
3.4 2.4 rtl/axi4_master_if.sv	61
4 第 3 章：Baseline UVM 验证环境	66
4.1 3.1 UVM 环境包	66
4.2 3.2 AXI4-Lite Agent (6 个文件)	76

4.3	3.3 AXI4 Memory Agent + 协议检查器	82
4.4	3.4 sequences + tests + tb_top + system_tb + axi4_slave_mem	86
5	第 4 章：Bonus RTL	95
5.1	4.0 9 项 Bonus 功能总览	96
5.2	4.1 rtl/include/fa_params_bonus.svh	97
5.3	4.2 rtl/flash_attention_bonus_top.sv	98
5.4	4.3 rtl/tile_controller_bonus.sv	100
5.5	4.4 rtl/compute_core_bonus.sv	102
5.6	4.5 rtl/axi4_lite_slave_bonus.sv	104
5.7	4.6 rtl/mask_unit.sv	104
5.8	4.7 rtl/dropout_unit.sv	106
5.9	4.8 rtl/task_queue.sv	108
5.10	4.9 rtl/bf16_exp_unit.sv	110
5.11	4.10 rtl/bf16_reciprocal_unit.sv	113
5.12	4.11 rtl/int8_quantizer.sv	114
5.13	4.12 rtl/axi4_stream_if.sv	116
6	第 5 章：Bonus UVM 验证 + scripts/seqgen_stub	118
6.1	5.1 bonus/tb/uvm_env/fa_bonus_env_pkg.sv	118
6.2	5.2 bonus/tb/uvm_env/fa_bonus_env.sv	119
6.3	5.3 bonus/tb/uvm_env/fa_bonus_scoreboard.sv	120
6.4	5.4 bonus/tb/uvm_env/fa_bonus_coverage.sv	122
6.5	5.5 bonus/tb/sequences/fa_bonus_sequences.sv	123
6.6	5.6 bonus/tb/tests/fa_bonus_tests.sv	125
6.7	5.7 bonus/tb/tb_top/fa_bonus_tb_top.sv	127
6.8	5.8 bonus/tb/bonus_system_tb.sv	129
6.9	5.9 scripts/seqgen_stub.v	130
6.10	全书结束语	131

1 第 0 章：项目总览

1.1 0.1 项目背景与目标

本项目是一个可综合的 **FlashAttention-style** 注意力算子硬件加速器 IP，用 SystemVerilog 实现，完成 Scaled Dot-Product Attention (SDPA) 计算：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V$$

它的核心思想来自 **FlashAttention** (Dao 等, 2022): 把 Q, K, V 切成小块 (tile) 在片上 SRAM 内逐块计算, **永不显式构造完整的 $s \times s$ 注意力矩阵**, 从而把片外内存带宽从 $O(s^2)$ 降到 $O(sd)$ 。所谓的 *online softmax* 是把 softmax 的两次扫描 (先求 max, 再算 exp 与归一化) 改写成 **一次单遍 + 增量更新** 的形式, 使得 tile 与 tile 之间只需要在每行维护两个滚动量 m_i (行 max) 和 ℓ_i (行 exp 和), 就能正确累加。

设计目标参数	值
序列长度 s	256
Head 维度 d	64
数据格式	Q8.8 (16-bit 有符号定点)
累加器位宽	40-bit
Tiling: $B_r \times B_c$	4×16
AXI 数据宽度	128-bit (8 个 Q8.8 / beat)
片上存储	~8.75 KB
目标频率	500 MHz (TSMC 12 nm)

整个项目分为:

- **baseline/**: 必选项 — 14 个 RTL 模块 + 完整 UVM 验证环境 + SDC 约束。
- **bonus/**: 9 项加分功能 — BF16/FP16、多头、变长、Padding mask、Q6.10/Q4.12、Dropout、INT8 量化、AXI4-Stream、DMA Task Queue。
- **scripts/**: 仿真、综合、后仿真脚本。
- **vcs_fopen_fix.c / vc_hdrs.h**: VCS 在新内核 / Docker 上的兼容性修复。

1.2 0.2 顶层架构图

下图展示了 baseline 顶层 flash_attention_top 的 5 大子模块以及它们与外部 DDR、CPU 的连接关系。

数据通路 (Data path) — 蓝色框是控制 / AXI 接口, 绿色框是片上存储, 橙色框是计算单元:

1. CPU 通过 **AXI4-Lite Slave** 写 15 个寄存器 ($Q/K/V/O$ 基址、stride、scale、causal_en、CTRL.START ...)。
2. **tile_controller** 是一个 12 状态 FSM, 按 $\lceil s/B_r \rceil = 64$ 个外层循环 $\times \lceil s/B_c \rceil = 16$ 个内层循环顺序遍历所有 (Q-tile, KV-tile) 对, 给 dma_engine 发读写请求、给 compute_core 发 start。
3. **dma_engine** 把外部 DDR 中的 $B_r \times d$ 的 Q tile 与 $B_c \times d$ 的 K/V tile 通过 AXI4 Master 突发搬到片上 SRAM, **K/V buffer 双缓冲 (ping-pong)** —— 当前 tile 计算时下一 tile 已经在加载。
4. **compute_core** 是 6 状态主 FSM (IDLE $\rightarrow$ DOT_PRODUCT $\rightarrow$ MASK $\rightarrow$ SOFTMAX $\rightarrow$ ACCUMULATE $\rightarrow$ DONE), 每个 KV tile 走完一遍流水。其中 dot_product_array 用 8 路并行 MAC 跑 $B_r \times B_c$ 个点积; online_softmax_unit 调用 1024-entry exp LUT 做 softmax; output_accumulator 用 ℓ_{old} / ℓ_{new} 完成在线归一化。

FlashAttention Accelerator Top-Level Architecture

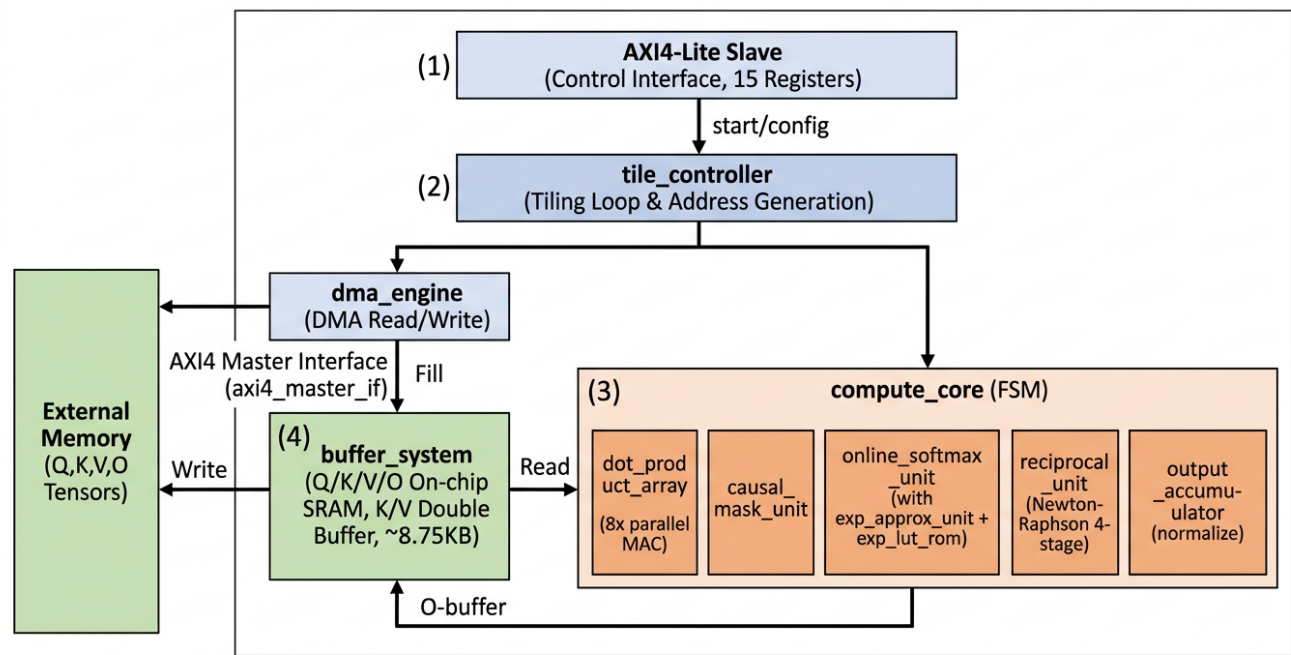


图 1: FlashAttention Accelerator Top-Level Architecture

5. KV 内层循环结束后, `output_accumulator` 用 `reciprocal_unit` (Newton-Raphson 4 级流水) 做 $O = O/\ell$, 结果写回 `buffer_system` 的 `O-buffer`, 再由 `dma_engine` 通过 AXI4 Master 写回 DDR。

1.3 0.3 目录结构与讲义章节安排

submission/	
├─ README.md	项目总说明
├─ docker_run_all.sh	一键 Docker 仿真
├─ docker_run_all_bonus.sh	Bonus 一键 Docker 仿真
├─ vcs_fopen_fix.c	VCS LD_PRELOAD 修复
├─ vc_hdrs.h	VCS DPI 头
├─	
├─ baseline/	===== 必选 =====
├─ rtl/	14 个 RTL + fa_params.svh + exp_lut.hex
├─ tb/	UVM 验证环境 (24 测试)
└─ constraints/flash_attention.sdc	
└─ docs/	design_report.md, verification_report.md
├─	
├─ bonus/	===== 9 项加分 =====
└─ rtl/	11 个增强 RTL + fa_params_bonus.svh

```
|   |─ tb/          Bonus UVM 环境 (24 测试)
|   |─ constraints/flash_attention.sdc
|
└─ scripts/          ===== 7 个脚本 =====
    |─ run_baseline.sh, run_uvm_all.sh
    |─ run_bonus.sh,   run_bonus_uvm.sh
    |─ run_synthesis.sh, run_postsim.sh
    └─ run_gate_rerun.sh
```

本讲义将.v / .sv / .svh 文件分为 5 章讲解：

章节	内容	文件数
第 1 章：Baseline RTL — 计算通路	flash_attention_top, tile_controller, compute_core, dot_product_array, online_softmax_unit, exp_approx_unit, exp_lut_rom, reciprocal_unit, output_accumulator, causal_mask_unit, fa_params.svh	11
第 2 章：Baseline RTL — 存储与总线	buffer_system, dma_engine, axi4_lite_slave, axi4_master_if	4
第 3 章：Baseline UVM 验证环境	20 个.sv (agents + env + sequences + tests + tb_top)	20
第 4 章：Bonus RTL	flash_attention_bonus_top + 10 个增强模块 + fa_params_bonus.svh	12
第 5 章：Bonus UVM 验证	9 个.sv + 1 个.v + scripts/seqgen_stub	10

每个文件都按 “整体架构图 → 代码块切片 → 每块源代码 + 解释 + (必要时) 图示” 的顺序呈现。

1.4 0.4 验证与性能结果摘要

版本	测试总数	通过	失败	执行周期	mean_abs_err	max_abs_err
Baseline	24	24	0	276,100	0.013350	0.238281
Bonus	24	24	0	276,165	0.013350	0.238281
合计	48	48	0	—	—	—

7 大验证领域: (1) 6 个验证点 VP01-VP06, (2) 7 个寄存器测试 REG01-REG07, (3) 3 个 DMA 测试 DMA01-DMA03, (4) 3 个 AXI 协议测试 AXI01-AXI03, (5) 2 个性能测试 PERF01-PERF02, (6) 3 个覆盖率测试 COV01-COV02 + COMP, (7) 39 条 SVA 协议断言 (21 条 AXI4 + 18 条 AXI4-Lite)。

2 第 1 章: Baseline RTL — 计算通路

本章覆盖 11 个文件 (10 个.sv + 1 个.svh):

§	文件	行数	角色
1.1	rtl/include/fa_params.svh	79	全局参数定义
1.2	rtl/flash_attention_top.sv	239	顶层 (5 大子模块集成)
1.3	rtl/tile_controller.sv	215	12 状态 tiling FSM
1.4	rtl/compute_core.sv	256	6 状态计算 FSM (调度 dot/mask/softmax/accum)
1.5	rtl/dot_product_array.sv	119	4 × 16 = 64 个 MAC, 8 路并行
1.6	rtl/causal_mask_unit.sv	28	因果 mask 组合逻辑
1.7	rtl/online_softmax_unit.sv	188	online softmax 6 状态 FSM + exp 流水
1.8	rtl/exp_approx_unit.sv	117	1024-entry LUT exp 近似, 3 级流水
1.9	rtl/exp_lut_rom.sv	56	1024 项 ROM (sim/syn 双模式)
1.10	rtl/reciprocal_unit.sv	135	Newton-Raphson 4 级流水 1/x
1.11	rtl/output_accumulator.sv	242	P · V 累加 + (last tile) 归一化

2.1 1.1 rtl/include/fa_params.svh

2.1.1 1.1.1 概述

全局参数头文件, 所有 RTL 都用 ``include "fa_params.svh"` 引入。它定义了 4 类常量: (1) 算法维度 (SEQ_LEN、HEAD_DIM、TILE_BR、TILE_BC); (2) 数据格式 (DATA_WIDTH=16/Q8.8、ACC_WIDTH=40、EXP_WIDTH=24); (3) AXI 总线宽度 (数据 128-bit、地址 64-bit); (4) 寄存器映射偏移与默认值。整个文件没有逻辑, 过于简单, **不需要架构图**。

2.1.2 1.1.2 完整源代码

```

`ifndef FA_PARAMS_SVH
`define FA_PARAMS_SVH

// =====
// FlashAttention Accelerator — Global Parameters
// =====

// --- Baseline fixed dimensions ---
parameter SEQ_LEN      = 256;
parameter HEAD_DIM     = 64;

// --- Tiling parameters ---
parameter TILE_BR      = 4;      // Q tile rows
parameter TILE_BC      = 16;    // K/V tile rows
parameter NUM_Q_TILES  = SEQ_LEN / TILE_BR;    // 64
parameter NUM_KV_TILES = SEQ_LEN / TILE_BC;    // 16

// --- Data widths ---
parameter DATA_WIDTH  = 16;    // Q8.8 fixed-point
parameter FRAC_BITS    = 8;    // fractional bits in Q8.8
parameter ACC_WIDTH    = 40;    // accumulator width for dot-products
parameter EXP_WIDTH    = 24;    // exp output width (wider for precision)
parameter SCORE_WIDTH  = 40;    // score/softmax intermediate width

// --- AXI parameters ---
parameter AXI_ADDR_WIDTH = 64;
parameter AXI_DATA_WIDTH = 128; // 8 Q8.8 values per beat
parameter AXI_ID_WIDTH   = 4;
parameter AXI_STRB_WIDTH = AXI_DATA_WIDTH / 8;

parameter AXIL_ADDR_WIDTH = 8;
parameter AXIL_DATA_WIDTH = 32;

// --- AXI burst ---
parameter AXI_BURST_LEN  = 16; // beats per burst

// --- Register map offsets ---
parameter REG_CTRL       = 8'h00;
parameter REG_STATUS     = 8'h04;

```

```

parameter REG_CFG          = 8'h08;
parameter REG_Q_BASE_L     = 8'h14;
parameter REG_Q_BASE_H     = 8'h18;
parameter REG_K_BASE_L     = 8'h1C;
parameter REG_K_BASE_H     = 8'h20;
parameter REG_V_BASE_L     = 8'h24;
parameter REG_V_BASE_H     = 8'h28;
parameter REG_O_BASE_L     = 8'h2C;
parameter REG_O_BASE_H     = 8'h30;
parameter REG_STRIDE_BYTES = 8'h34;
parameter REG_NEG_LARGE    = 8'h38;
parameter REG_SCALE        = 8'h3C;
parameter REG_CYCLES       = 8'h40;

// --- CTRL register bits ---
parameter CTRL_START       = 0;
parameter CTRL_SOFT_RESET = 1;
parameter CTRL_IRQ_EN      = 2;

// --- STATUS register bits ---
parameter STATUS_BUSY      = 0;
parameter STATUS_DONE      = 1;
parameter STATUS_ERROR     = 2;

// --- CFG register bits ---
parameter CFG_CAUSAL_EN    = 0;

// --- Default values ---
parameter DEFAULT_STRIDE   = HEAD_DIM * 2;           // d * sizeof(Q8.8)
parameter DEFAULT_NEG_LARGE = 16'h8000;             // -128.0 in Q8.8
parameter DEFAULT_SCALE    = 16'h0020;             // 1/8 ≈ 1/√64 in Q8.8

// --- Parallelism ---
parameter PAR_MACS         = 8;           // parallel MACs per dot-product per cycle

// --- Exp LUT ---
parameter EXP_LUT_DEPTH    = 256;
parameter EXP_LUT_WIDTH    = 16;

`endif

```

2.1.3 1.1.3 解释

- SEQ_LEN=256, HEAD_DIM=64: 一次推理的 attention 矩阵是 256×256 ; 如果用 16-bit 全部物化, 需要 128 KB 片上 SRAM, 超出 8.75 KB 的预算。这正是 **FlashAttention 必须 tiling 的根本原因**。
- TILE_BR=4, TILE_BC=16: 每次 compute_core 处理 4×64 的 Q 子块和 16×64 的 KV 子块, 得到 4×16 的子分数矩阵。一共 $64 \times 16 = 1024$ 个 (Q-tile, KV-tile) 对。
- DATA_WIDTH=16, FRAC_BITS=8: Q8.8 定点 — 8 位整数 + 8 位小数。乘法结果是 Q16.16, 所以 ACC_WIDTH=40 给定点累加留 8 位 headroom (256 个累加项最多增加 $\log_2 256 = 8$ 位)。
- AXI_DATA_WIDTH=128: 每个 beat 装 $128/16 = 8$ 个 Q8.8 值, 正好对应 PAR_MACS=8, 让 DMA 的拍率与 MAC 阵列的吞吐对齐。
- DEFAULT_SCALE=16'h0020: 即 $32/256 = 0.125 = 1/\sqrt{64}$, 这就是 $\frac{1}{\sqrt{d}}$ 。
- DEFAULT_NEG_LARGE=16'h8000: Q8.8 的 -128 , 用作 causal-mask 的“负无穷大”, 进 exp 后得 0。
- 寄存器映射: 所有偏移量都按 4 字节对齐, 方便 AXI4-Lite 32-bit 访问。REG_CYCLES 是只读性能计数器。

2.2 1.2 rtl/flash_attention_top.sv

2.2.1 1.2.1 概述

顶层模块, 把 5 个子模块 (axi4_lite_slave、tile_controller、dma_engine、buffer_system、compute_core) 连接起来, 并把 AXI4-Lite Slave 端口和 AXI4 Master 端口暴露到 IP 边界。它本身只做“互连 + 32-bit 周期计数器 + 状态信号汇聚”, 几乎没有原创逻辑。

2.2.2 1.2.2 文件架构图

参见第 0 章的 顶层架构图。

2.2.3 1.2.3 代码块切片

2.2.3.1 块 ① 端口定义 (行 8-64) 源代码:

```
module flash_attention_top (
    input logic          clk,
    input logic          rst_n,

    // --- AXI4-Lite Slave Interface (Control) ---
```

```

input logic [AXIL_ADDR_WIDTH-1:0] s_axil_awaddr,
input logic                        s_axil_awvalid,
output logic                      s_axil_awready,
input logic [AXIL_DATA_WIDTH-1:0] s_axil_wdata,
input logic [AXIL_DATA_WIDTH/8-1:0] s_axil_wstrb,
input logic                      s_axil_wvalid,
output logic                      s_axil_wready,
output logic [1:0]                s_axil_bresp,
output logic                      s_axil_bvalid,
input logic                      s_axil_bready,
input logic [AXIL_ADDR_WIDTH-1:0] s_axil_araddr,
input logic                      s_axil_arvalid,
output logic                      s_axil_arready,
output logic [AXIL_DATA_WIDTH-1:0] s_axil_rdata,
output logic [1:0]                s_axil_rresp,
output logic                      s_axil_rvalid,
input logic                      s_axil_rready,

```

```
// --- AXI4 Master Interface (Data) ---
```

```

output logic [AXI_ID_WIDTH-1:0] m_axi_awid,
output logic [AXI_ADDR_WIDTH-1:0] m_axi_awaddr,
output logic [7:0] m_axi_awlen,
/* ...(AW, W, B, AR, R 共 31 条 AXI4 信号略)... */
output logic m_axi_rready,

```

```
// Interrupt
```

```
output logic irq
```

```
);
```

说明: 标准 ARM AMBA AXI4 + AXI4-Lite 接口。AXI4-Lite Slave (s_axil_*) 用于 CPU 配置寄存器与读取状态; AXI4 Master (m_axi_*) 用于 DMA 把 Q/K/V 张量从外部 DDR 搬到片上、把 O 张量写回。irq 是任务结束中断, 由 axi4_lite_slave 内部根据 STATUS.DONE + CTRL.IRQ_EN 生成。代码块本身只是端口列表, 过于简单, 无需图示。

2.2.3.2 块 ② AXI4-Lite Slave 实例化 (行 66-93) 源代码:

```

// ===== Register File Signals =====
logic reg_start, reg_soft_reset, reg_irq_en, reg_causal_en;
logic [63:0] reg_q_base, reg_k_base, reg_v_base, reg_o_base;
logic [31:0] reg_stride_bytes;
logic signed [15:0] reg_neg_large, reg_scale;
logic status_busy, status_done, status_error;

```

```

logic [31:0] cycle_count;
logic      done_clear;

axi4_lite_slave #(.ADDR_WIDTH(AXIL_ADDR_WIDTH), .DATA_WIDTH(AXIL_DATA_WIDTH))
u_axil (
    .clk(clk), .rst_n(rst_n),
    .s_axil_awaddr(s_axil_awaddr), .s_axil_awvalid(s_axil_awvalid), /*...*/
    .reg_start(reg_start), .reg_soft_reset(reg_soft_reset), .reg_irq_en(reg_irq_en),
    .reg_causal_en(reg_causal_en),
    .reg_q_base(reg_q_base), .reg_k_base(reg_k_base),
    .reg_v_base(reg_v_base), .reg_o_base(reg_o_base),
    .reg_stride_bytes(reg_stride_bytes),
    .reg_neg_large(reg_neg_large), .reg_scale(reg_scale),
    .status_busy(status_busy), .status_done(status_done), .status_error(status_error),
    .cycle_count(cycle_count), .done_clear(done_clear), .irq(irq)
);

```

说明: axi4_lite_slave 提供 15 个寄存器 (CTRL/STATUS/CFG + 4 个 64-bit 张量基址 + 1 个 stride + neg_large + scale + cycle_count)。CPU 写 CTRL.START=1 后, reg_start 单周期脉冲传给 tile_controller。reg_causal_en 告诉 compute_core 是否要做 causal mask。这段是普通的端口绑定, **无需图示**。

2.2.3.3 块 ③ tile_controller 实例化 (行 95-128) 源代码:

```

// ===== DMA / Tile Controller / Buffer Signals =====
logic      tc_dma_rd_req, tc_dma_rd_done;
logic [AXI_ADDR_WIDTH-1:0] tc_dma_rd_addr;
logic [15:0] tc_dma_rd_len;
logic [1:0] tc_dma_rd_target;
logic      tc_dma_wr_req, tc_dma_wr_done;
logic [AXI_ADDR_WIDTH-1:0] tc_dma_wr_addr;
logic [15:0] tc_dma_wr_len;
logic      tc_compute_start, tc_compute_first_kv, tc_compute_last_kv, tc_compute_done;
logic [$clog2(SEQ_LEN)-1:0] tc_q_tile_idx, tc_kv_tile_idx;
logic      tc_kv_buf_sel;
logic      tc_all_done, tc_busy;
logic      tc_o_wb_start, tc_o_wb_done;

tile_controller #(/*...*/) u_tile_ctrl (
    .clk(clk), .rst_n(rst_n),
    .start(reg_start), .all_done(tc_all_done), .busy(tc_busy),
    .q_base_addr(reg_q_base), .k_base_addr(reg_k_base),

```

```

        .v_base_addr(reg_v_base), .o_base_addr(reg_o_base),
        .stride_bytes(reg_stride_bytes),
        .dma_rd_req(tc_dma_rd_req), .dma_rd_addr(tc_dma_rd_addr),
        .dma_rd_len_bytes(tc_dma_rd_len), .dma_rd_target(tc_dma_rd_target),
        .dma_rd_done(tc_dma_rd_done),
        /* ...DMA write + compute control + tile indices + ping-pong sel... */
    );

```

说明: tile_controller 接收 reg_start 启动信号、4 个张量基址、stride, 输出 DMA 请求和 compute 控制。tc_kv_buf_sel 是片上 K/V buffer 的 ping-pong 选择位 (每个 KV tile 翻转一次), 它同时被发给 dma_engine (决定写哪一片) 和 buffer_system (决定读另一片)。

2.2.3.4 块 ④ dma_engine 实例化 (行 130-169) 源代码:

```

// Buffer wires
logic          buf_q_wr_en, buf_k_wr_en, buf_v_wr_en;
logic [$clog2(TILE_BR*HEAD_DIM)-1:0] buf_q_wr_addr;
logic [$clog2(TILE_BC*HEAD_DIM)-1:0] buf_k_wr_addr, buf_v_wr_addr;
logic [AXI_DATA_WIDTH-1:0] buf_q_wr_data, buf_k_wr_data, buf_v_wr_data;
logic          buf_o_rd_en;
logic [$clog2(TILE_BR)-1:0] buf_o_rd_row;
logic [$clog2(HEAD_DIM/(AXI_DATA_WIDTH/DATA_WIDTH))-1:0] buf_o_rd_col_grp;
logic [AXI_DATA_WIDTH-1:0] buf_o_rd_data;

dma_engine #(/*...*/) u_dma (
    .clk(clk), .rst_n(rst_n),
    /* AXI4 Master 31 ports */
    .dma_rd_req(tc_dma_rd_req), .dma_rd_addr(tc_dma_rd_addr),
    .dma_rd_len_bytes(tc_dma_rd_len), .dma_rd_target(tc_dma_rd_target),
    .dma_rd_done(tc_dma_rd_done),
    .dma_wr_req(tc_dma_wr_req), .dma_wr_addr(tc_dma_wr_addr),
    .dma_wr_len_bytes(tc_dma_wr_len), .dma_wr_done(tc_dma_wr_done),
    .buf_q_wr_en(buf_q_wr_en), .buf_q_wr_addr(buf_q_wr_addr), .buf_q_wr_data(buf_q_wr_data),
    .buf_k_wr_en(buf_k_wr_en), .buf_k_wr_addr(buf_k_wr_addr), .buf_k_wr_data(buf_k_wr_data),
    .buf_v_wr_en(buf_v_wr_en), .buf_v_wr_addr(buf_v_wr_addr), .buf_v_wr_data(buf_v_wr_data),
    .buf_o_rd_en(buf_o_rd_en), .buf_o_rd_row(buf_o_rd_row),
    .buf_o_rd_col_grp(buf_o_rd_col_grp), .buf_o_rd_data(buf_o_rd_data),
    .kv_buf_sel(tc_kv_buf_sel)
);

```

说明: 互连而已, 仍然简单。注意 kv_buf_sel 同时进 dma_engine 和后面的 buffer_system, 但二者的选择关系是反的 (写哪片 vs 读另一片) —— 这是 ping-pong 双缓冲的关键。

2.2.3.5 块 ⑤ buffer_system + compute_core 实例化 (行 171-215) 源代码:

```

// Compute core buffer read signals
logic          comp_q_rd_en, comp_k_rd_en, comp_v_rd_en;
logic [$clog2(HEAD_DIM/PAR_MACS)-1:0] comp_q_step, comp_k_step, comp_v_step;
logic signed [DATA_WIDTH-1:0] comp_q_data [TILE_BR-1:0][PAR_MACS-1:0];
logic signed [DATA_WIDTH-1:0] comp_k_data [TILE_BC-1:0][PAR_MACS-1:0];
logic signed [DATA_WIDTH-1:0] comp_v_data [TILE_BC-1:0][PAR_MACS-1:0];
logic signed [DATA_WIDTH-1:0] comp_o_tile [TILE_BR-1:0][HEAD_DIM-1:0];
logic          comp_o_valid;

buffer_system #(/*...*/) u_buffers (
    .clk(clk), .rst_n(rst_n),
    .q_wr_en(buf_q_wr_en), .q_wr_addr(buf_q_wr_addr), .q_wr_data(buf_q_wr_data),
    .k_wr_en(buf_k_wr_en), .k_wr_addr(buf_k_wr_addr), .k_wr_data(buf_k_wr_data),
    .k_buf_sel(tc_kv_buf_sel),
    /* ... */
    .k_rd_buf_sel(~tc_kv_buf_sel), .k_rd_data(comp_k_data),
    /* ... */
    .o_wr_en(comp_o_valid), .o_wr_data(comp_o_tile),
    .o_rd_en(buf_o_rd_en), .o_rd_row(buf_o_rd_row),
    .o_rd_col_grp(buf_o_rd_col_grp), .o_rd_data(buf_o_rd_data)
);

compute_core #(/*...*/) u_compute (
    .clk(clk), .rst_n(rst_n),
    .start(tc_compute_start), .first_kv_tile(tc_compute_first_kv),
    .last_kv_tile(tc_compute_last_kv),
    .done(tc_compute_done), .busy(),
    .q_tile_idx(tc_q_tile_idx), .kv_tile_idx(tc_kv_tile_idx),
    .causal_en(reg_causal_en), .scale(reg_scale), .neg_large(reg_neg_large),
    .q_rd_en(comp_q_rd_en), .q_rd_step(comp_q_step), .q_data(comp_q_data),
    /* ... */
    .o_tile(comp_o_tile), .o_valid(comp_o_valid)
);

```

说明: 注意 `k_rd_buf_sel(~tc_kv_buf_sel)`: 当 `tc_kv_buf_sel=0` 时, DMA 写 `K_buf[0]`, `compute_core` 读 `K_buf[1]`。下一轮翻转后, DMA 写 `K_buf[1]`, `compute` 读 `K_buf[0]`。这就是**乒乓双缓冲**, 让“加载下一 tile”与“计算当前 tile”重叠。

2.2.3.6 块 ⑥ 周期计数器与状态信号 (行 217-238) 源代码:

```
// ===== Cycle Counter =====
logic [31:0] cycle_cnt;
assign cycle_count = cycle_cnt;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        cycle_cnt <= '0;
    else if (reg_start)
        cycle_cnt <= '0;
    else if (tc_busy)
        cycle_cnt <= cycle_cnt + 1;
end

// ===== Status =====
assign status_busy = tc_busy;
assign status_done = tc_all_done;
assign status_error = 1'b0; // placeholder

// 0 writeback done
assign tc_o_wb_done = tc_dma_wr_done;

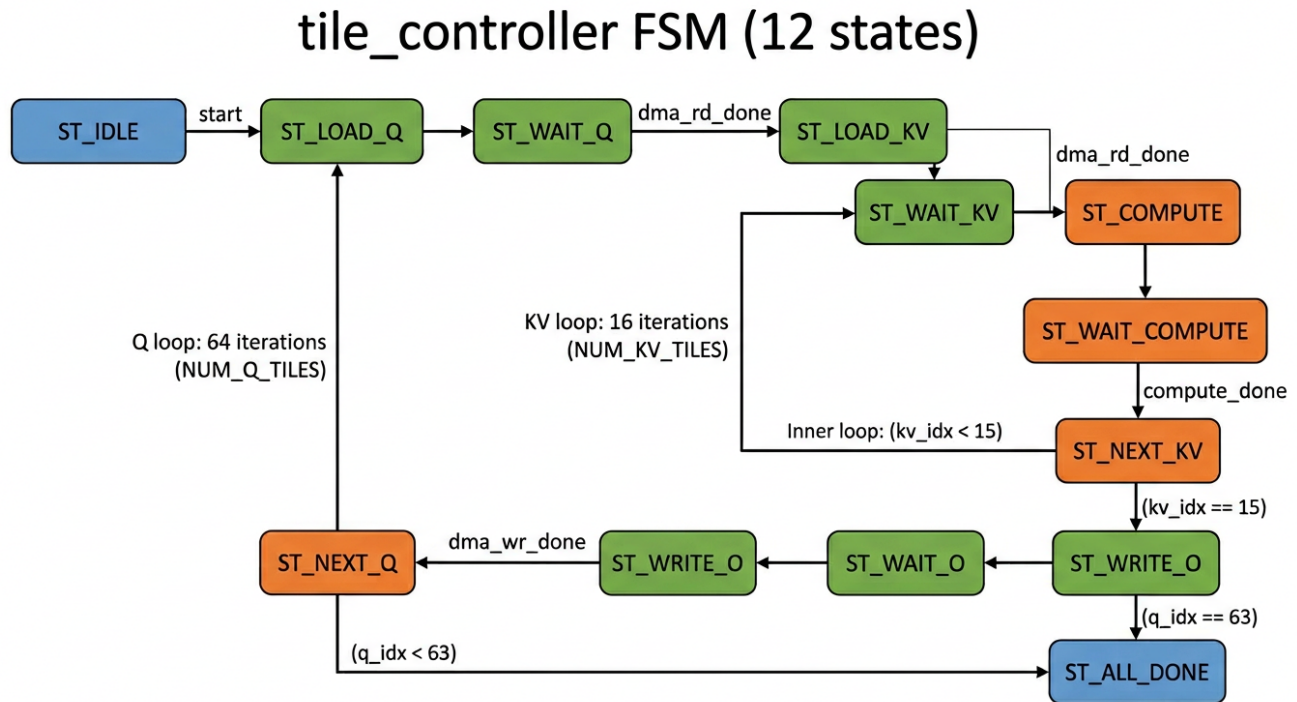
endmodule
```

说明: cycle_cnt 在 reg_start 时清零、在 tc_busy 期间每周期加 1，结束时硬件总耗时被记录到 REG_CYCLES 寄存器，CPU 可以读出做 KPI (baseline 实测 276,100 cycles)。status_error 暂时硬接 0，没有错误检测路径——这是后续可改进点。

2.3 1.3 rtl/tile_controller.sv

2.3.1 1.3.1 概述

整个 IP 的“任务调度大脑”。一个 12 状态 FSM，按 $q_idx \in [0, 64)$ 外层 $\times$ $kv_idx \in [0, 16)$ 内层的顺序遍历所有 (Q-tile, KV-tile) 对，针对每个 tile 顺序发出：(1) Q DMA 请求 $\rightarrow$ (2) K DMA 请求 $\rightarrow$ (3) V DMA 请求 $\rightarrow$ (4) compute_start $\rightarrow$ (5) 等 compute_done $\rightarrow$ 翻转 ping-pong 选择 $\rightarrow$ 下一 KV tile；当 KV 内层走完发起 (6) O write-back DMA $\rightarrow$ 下一 Q tile。



```

input logic [31:0]
// DMA request interface
output logic
output logic [AXI_ADDR_WIDTH-1:0]
output logic [15:0]
output logic [1:0]
input logic
output logic
output logic [AXI_ADDR_WIDTH-1:0]
output logic [15:0]
input logic
// Compute core control
output logic

input logic
// Tile indices (for causal mask)
output logic [$clog2(SEQ_LEN)-1:0]
// Buffer sel for ping-pong
output logic
// 0 write-back trigger
output logic
input logic

stride_bytes,
dma_rd_req,
dma_rd_addr,
dma_rd_len_bytes,
dma_rd_target, // 0=Q, 1=K, 2=V
dma_rd_done,
dma_wr_req,
dma_wr_addr,
dma_wr_len_bytes,
dma_wr_done,

compute_start, compute_first_kv,
compute_last_kv,
compute_done,

q_tile_idx, kv_tile_idx,

kv_buf_sel,

o_writeback_start,
o_writeback_done
);

```

说明: 端口分 5 组: (1) 时钟复位; (2) 启动/状态; (3) 配置寄存器 (基址 + stride) ; (4) DMA 通道 (读和写各一组 req/done) ; (5) compute_core 控制 + ping-pong 选择信号。dma_rd_target 是一个 2-bit 编码: 0=Q、1=K、2=V, dma_engine 据此把 AXI 读回数据路由到正确 buffer。compute_first_kv/compute_last_kv 是 online softmax 状态的边界提示: 第一个 tile 时初始化 m, ℓ ; 最后一个 tile 时触发归一化。**端口列表过于简单, 无需图示。**

2.3.3.2 块 ② FSM 状态定义与初始化 (行 59-106) 源代码:

```

localparam NUM_Q_TILES = SEQ_LEN / TILE_BR;
localparam NUM_KV_TILES = SEQ_LEN / TILE_BC;
localparam Q_TILE_BYTES = TILE_BR * HEAD_DIM * (DATA_WIDTH / 8);
localparam KV_TILE_BYTES = TILE_BC * HEAD_DIM * (DATA_WIDTH / 8);
localparam O_TILE_BYTES = TILE_BR * HEAD_DIM * (DATA_WIDTH / 8);

typedef enum logic [3:0] {
    ST_IDLE, ST_LOAD_Q, ST_WAIT_Q,
    ST_LOAD_KV, ST_WAIT_KV,
    ST_COMPUTE, ST_WAIT_COMPUTE,

```

```

        ST_NEXT_KV,
        ST_WRITE_0, ST_WAIT_0,
        ST_NEXT_Q, ST_ALL_DONE
    } state_t;

state_t state;
logic [$clog2(NUM_Q_TILES):0] q_idx;
logic [$clog2(NUM_KV_TILES):0] kv_idx;
localparam integer TILE_IDX_W = $clog2(SEQ_LEN);

assign q_tile_idx = TILE_IDX_W'(q_idx);
assign kv_tile_idx = TILE_IDX_W'(kv_idx);

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= ST_IDLE;
        all_done <= 0; busy <= 0;
        q_idx <= '0; kv_idx <= '0;
        kv_buf_sel <= 0;
        dma_rd_req <= 0; dma_wr_req <= 0;
        compute_start <= 0; compute_first_kv <= 0; compute_last_kv <= 0;
        o_writeback_start <= 0;
    end else begin
        all_done <= 0;
        compute_start <= 0;
        o_writeback_start <= 0;
        /* ... case (state) ... */
    end
end

```

说明: 关键派生量: 每个 Q tile 大小 = $4 * 64 * 2 = 512$ 字节; 每个 KV tile = $16 * 64 * 2 = 2048$ 字节; 每个 O tile 与 Q tile 同大 = 512 字节。复位时所有 req/compute_start 拉低, 进入 IDLE。注意 compute_start、o_writeback_start、all_done 默认值在 always_ff 内每个周期被重置为 0 (自清除型脉冲), 需要的时候在 case 内显式置 1。

2.3.3.3 块 ② FSM 主体 (case 部分) (行 107-212) 源代码 (完整 case):

```

case (state)
    ST_IDLE: begin
        dma_rd_req <= 0; dma_wr_req <= 0;
        if (start) begin
            state <= ST_LOAD_Q;
            busy <= 1;
            q_idx <= '0; kv_idx <= '0;
        end
    end
end

```

```

        end
    end
    ST_LOAD_Q: begin
        dma_rd_req      <= 1;
        dma_rd_addr     <= q_base_addr + AXI_ADDR_WIDTH'(q_idx) *
                        AXI_ADDR_WIDTH'(stride_bytes) * TILE_BR;
        dma_rd_len_bytes <= Q_TILE_BYTES;
        dma_rd_target   <= 2'd0;
        state           <= ST_WAIT_Q;
    end
    ST_WAIT_Q: begin
        dma_rd_req <= 0;
        if (dma_rd_done) begin
            kv_idx <= '0; kv_buf_sel <= 0;
            state <= ST_LOAD_KV;
        end
    end
    ST_LOAD_KV: begin
        dma_rd_req      <= 1;
        dma_rd_addr     <= k_base_addr + AXI_ADDR_WIDTH'(kv_idx) *
                        AXI_ADDR_WIDTH'(stride_bytes) * TILE_BC;
        dma_rd_len_bytes <= KV_TILE_BYTES;
        dma_rd_target   <= 2'd1;
        state           <= ST_WAIT_KV;
    end
    ST_WAIT_KV: begin
        dma_rd_req <= 0;
        if (dma_rd_done) begin
            // Load V tile
            dma_rd_req      <= 1;
            dma_rd_addr     <= v_base_addr + AXI_ADDR_WIDTH'(kv_idx) *
                        AXI_ADDR_WIDTH'(stride_bytes) * TILE_BC;
            dma_rd_len_bytes <= KV_TILE_BYTES;
            dma_rd_target   <= 2'd2;
            state           <= ST_COMPUTE;
        end
    end
    ST_COMPUTE: begin
        dma_rd_req <= 0;
        if (dma_rd_done) begin
            compute_start <= 1;
        end
    end
end

```

```

        compute_first_kv <= (kv_idx == 0);
        compute_last_kv  <= (kv_idx == NUM_KV_TILES - 1);
        state             <= ST_WAIT_COMPUTE;
    end
end
ST_WAIT_COMPUTE: begin
    if (compute_done) state <= ST_NEXT_KV;
end
ST_NEXT_KV: begin
    if (kv_idx == NUM_KV_TILES - 1) state <= ST_WRITE_0;
    else begin
        kv_idx      <= kv_idx + 1;
        kv_buf_sel <= ~kv_buf_sel;
        state       <= ST_LOAD_KV;
    end
end
end
ST_WRITE_0: begin
    dma_wr_req      <= 1;
    dma_wr_addr     <= o_base_addr + AXI_ADDR_WIDTH'(q_idx) *
                        AXI_ADDR_WIDTH'(stride_bytes) * TILE_BR;
    dma_wr_len_bytes <= 0_TILE_BYTES;
    o_writeback_start <= 1;
    state           <= ST_WAIT_0;
end
end
ST_WAIT_0: begin
    if (dma_wr_done) begin
        dma_wr_req <= 0;
        state <= ST_NEXT_Q;
    end
end
end
ST_NEXT_Q: begin
    if (q_idx == NUM_Q_TILES - 1) state <= ST_ALL_DONE;
    else begin
        q_idx <= q_idx + 1;
        state <= ST_LOAD_Q;
    end
end
end
ST_ALL_DONE: begin
    all_done <= 1;
    busy     <= 0;
    state    <= ST_IDLE;
end

```

```

        end
    endcase

```

说明：这就是上面架构图的代码实现。三个值得展开的细节：

1. **K 和 V 串行加载 (ST_LOAD_KV → ST_WAIT_KV):** K 加载完成后, 在 ST_WAIT_KV 内重叠地发起 V 加载 (注意 dma_rd_req 在同一周期被重新拉高), 然后跳到 ST_COMPUTE 等 V 完成。这样 V 加载和“等 V”是同一状态完成的, 省了一个状态。
2. **kv_buf_sel 的翻转时机 (ST_NEXT_KV):** 每完成一个 KV tile 后翻转, 下次写入下一个 buffer, 但当前 compute 还在用旧 buffer 读, 所以 buffer_system 内部对读端口用 ~kv_buf_sel。
3. **first/last KV 标志的生成 (ST_COMPUTE):** 组合判断 kv_idx==0 / kv_idx==NUM_KV_TILES-1 作为 compute_first_kv/compute_last_kv 输出。compute_core 用前者初始化 $m = -\infty, \ell = 0$, 用后者触发归一化。

性能分析: 从架构图看, 每个 (Q-tile, KV-tile) 对都是“严格串行”的 DMA→compute→DMA→compute 模式, 没有真正利用 **ping-pong**——这是 baseline 的简化实现, bonus 版本的 task_queue 才解决了这个问题。

2.4 1.4 rtl/compute_core.sv

2.4.1 1.4.1 概述

针对单个 (Q-tile, KV-tile) 对, 调度 4 个子模块完成完整的 attention block 计算: (1) dot_product_array 算分数 $S = Q \cdot K^T \cdot \text{scale}$; (2) 组合逻辑 mask 把 S 中 causal 位置改成 neg_large; (3) on-line_softmax_unit 算 $P = \text{softmax}(S)$ 并更新滚动 m, ℓ ; (4) output_accumulator 算 $O += P \cdot V$, 最后一个 KV tile 时归一化 $O = O/\ell$ 。

2.4.2 1.4.2 文件架构图

2.4.3 1.4.3 代码块切片

2.4.3.1 块 ① 端口与 dot_product 实例 (行 6-87) 源代码 (节选):

```

module compute_core #(* ... 9 个 parameter ... *) (
    input  logic clk, rst_n,
    input  logic start, first_kv_tile, last_kv_tile,
    output logic done, busy,
    input  logic [$clog2(SEQ_LEN)-1:0] q_tile_idx, kv_tile_idx,
    input  logic causal_en,
    input  logic signed [DATA_WIDTH-1:0] scale,

```

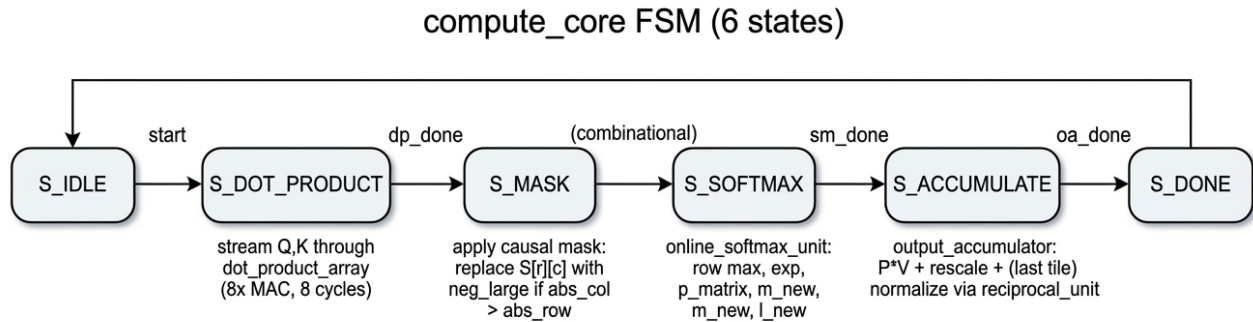


图 3: compute_core FSM

```

input  logic signed [15:0]          neg_large,
output logic q_rd_en, k_rd_en, v_rd_en,
output logic [$clog2(HEAD_DIM/PAR_MACS)-1:0] q_rd_step, k_rd_step, v_rd_step,
input  logic signed [DATA_WIDTH-1:0] q_data [TILE_BR-1:0][PAR_MACS-1:0],
input  logic signed [DATA_WIDTH-1:0] k_data [TILE_BC-1:0][PAR_MACS-1:0],
input  logic signed [DATA_WIDTH-1:0] v_data [TILE_BC-1:0][PAR_MACS-1:0],
output logic signed [DATA_WIDTH-1:0] o_tile [TILE_BR-1:0][HEAD_DIM-1:0],
output logic o_valid
);

localparam NUM_STEPS = HEAD_DIM / PAR_MACS;  // 64/8 = 8

typedef enum logic [2:0] {
    S_IDLE, S_DOT_PRODUCT, S_MASK, S_SOFTMAX, S_ACCUMULATE, S_DONE
} state_t;
state_t state;

// Dot product signals
logic dp_start, dp_done, dp_busy;
logic signed [ACC_WIDTH-1:0] dp_scores [TILE_BR-1:0][TILE_BC-1:0];
logic dp_scores_valid, dp_data_valid;
logic [$clog2(NUM_STEPS):0] dp_step;

```

```
dot_product_array #(/*...*/) u_dp (
    .clk(clk), .rst_n(rst_n),
    .start(dp_start), .done(dp_done), .busy(dp_busy),
    .q_data(q_data), .k_data(k_data), .data_valid(dp_data_valid),
    .scale(scale),
    .scores(dp_scores), .scores_valid(dp_scores_valid)
);
```

说明: q_data/k_data/v_data 是 unpacked array 端口直接连到 buffer_system, 每周期 buffer 给 4×8 (Q) / 16×8 (K 或 V) 个 16-bit 值。q_rd_step/k_rd_step/v_rd_step 控制 buffer 在 HEAD_DIM/PAR_MACS = 8 个 step 之间切换地址 (8 step $\times$ 8 元素 = 64 = HEAD_DIM 一行)。dp_data_valid 是给 dot_product 的“本周期数据有效”信号。

2.4.3.2 块 ② online_softmax 实例 + 持久 m,l 状态 (行 89-153) 源代码:

```
// Masked scores
logic signed [ACC_WIDTH-1:0] masked_scores [TILE_BR-1:0][TILE_BC-1:0];

// Online softmax signals
logic sm_start, sm_done, sm_busy;
logic signed [ACC_WIDTH-1:0] m_old [TILE_BR-1:0];
logic [ACC_WIDTH-1:0] l_old [TILE_BR-1:0];
logic signed [ACC_WIDTH-1:0] m_new [TILE_BR-1:0];
logic [ACC_WIDTH-1:0] l_new [TILE_BR-1:0];
logic [EXP_WIDTH-1:0] p_matrix [TILE_BR-1:0][TILE_BC-1:0];
logic [ACC_WIDTH-1:0] rescale_vals [TILE_BR-1:0];
logic sm_valid;

online_softmax_unit #(/*...*/) u_softmax (
    .clk(clk), .rst_n(rst_n),
    .start(sm_start), .first_tile(first_kv_tile),
    .done(sm_done), .busy(sm_busy),
    .scores(masked_scores), .neg_large(neg_large),
    .m_old(m_old), .l_old(l_old),
    .m_new(m_new), .l_new(l_new),
    .p_matrix(p_matrix), .rescale(rescale_vals),
    .results_valid(sm_valid)
);
/* ... output_accumulator 实例化 ... */

// Persistent softmax state across KV tiles
```

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (int r = 0; r < TILE_BR; r++) begin
            m_old[r] <= {ACC_WIDTH{1'b1}}; // -inf
            l_old[r] <= '0;
        end
    end else if (sm_valid) begin
        for (int r = 0; r < TILE_BR; r++) begin
            m_old[r] <= m_new[r];
            l_old[r] <= l_new[r];
        end
    end else if (state == S_IDLE && start && first_kv_tile) begin
        for (int r = 0; r < TILE_BR; r++) begin
            m_old[r] <= {ACC_WIDTH{1'b1}};
            l_old[r] <= '0;
        end
    end
end
end

```

说明: 这是 **online softmax** 的关键状态机。m_old[r] / l_old[r] 在跨 KV tile 时滚动保留, 每个 tile 完成 softmax 后 (sm_valid=1) 把 m_new 写回 m_old、l_new 写回 l_old。当一个新的 Q tile 开始 (start && first_kv_tile) 时, 把它们重置为 -∞ 和 0 —— 这正是 FlashAttention online softmax 的“每行独立累计”语义。{ACC_WIDTH{1'b1}} 在二补码下是 0xFF...F = -1, 但代码意图是 -∞; 这里实际用的是 sign-extended 全 1, 等价于负数最大绝对值 (约 -2^{39})。

2.4.3.3 块 ③ output_accumulator 实例 (行 113-134) 源代码:

```

output_accumulator #(/*...*/) u_oa (
    .clk(clk), .rst_n(rst_n),
    .start(oa_start), .first_tile(first_kv_tile), .last_tile(last_kv_tile),
    .done(oa_done), .busy(oa_busy),
    .p_matrix(p_matrix),
    .v_data(v_data), .v_valid(oa_v_valid),
    .rescale(rescale_vals), .l_values(l_new),
    .o_out(o_tile), .o_valid(o_valid),
    .v_request(oa_v_request)
);

```

说明: output_accumulator 接收 softmax 输出的 p_matrix 和滚动 rescale_vals (即 ℓ_{old}), 乘 V 累加到 o_tile。v_request 是从 oa 输出回 compute_core 的“我需要 V 数据”反压信号; oa_v_valid 是 compute_core 给 oa 的“V 数据本周期有效”。l_values(l_new) 在最后一个 tile 时被用来计算 $1/\square$ 做归一化。

2.4.3.4 块④ 主控 FSM (行 155-253) 源代码 (关键部分):

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= S_IDLE;
        done <= 0; busy <= 0;
        dp_start <= 0; sm_start <= 0; oa_start <= 0;
        /* ... reset all pulse signals ... */
    end else begin
        /* default deassert pulses */
        done <= 0; dp_start <= 0; sm_start <= 0; oa_start <= 0;
        dp_data_valid <= 0;
        q_rd_en <= 0; k_rd_en <= 0; v_rd_en <= 0; oa_v_valid <= 0;

        case (state)
            S_IDLE: if (start) begin
                state <= S_DOT_PRODUCT;
                busy <= 1; dp_start <= 1; dp_step <= '0;
            end
            S_DOT_PRODUCT: begin
                if (!dp_done && dp_busy) begin
                    // Stream Q and K to dot-product
                    q_rd_en <= 1; k_rd_en <= 1;
                    q_rd_step <= dp_step[$clog2(NUM_STEPS)-1:0];
                    k_rd_step <= dp_step[$clog2(NUM_STEPS)-1:0];
                    dp_data_valid <= 1;
                    dp_step <= dp_step + 1;
                end
                if (dp_done) state <= S_MASK;
            end
            S_MASK: begin
                // Apply causal mask combinatorially
                for (int r = 0; r < TILE_BR; r++) begin
                    for (int c = 0; c < TILE_BC; c++) begin
                        logic mask_bit;
                        logic [IDX_W-1:0] abs_row, abs_col;
                        abs_row = q_tile_idx * TILE_BR + IDX_W'(r);
                        abs_col = kv_tile_idx * TILE_BC + IDX_W'(c);
                        mask_bit = causal_en & (abs_col > abs_row);
                        if (mask_bit)
                            masked_scores[r][c] <=
                                {{(ACC_WIDTH-16){neg_large[15]}}, neg_large};
                    end
                end
            end
        endcase
    end
end

```

```

        else
            masked_scores[r][c] <= dp_scores[r][c];
        end
    end
    state <= S_SOFTMAX;
    sm_start <= 1;
end
S_SOFTMAX: if (sm_done) begin
    state <= S_ACCUMULATE;
    oa_start <= 1; oa_v_step <= '0';
end
S_ACCUMULATE: begin
    if (oa_v_request && !oa_done && oa_v_step < NUM_STEPS) begin
        v_rd_en <= 1;
        v_rd_step <= oa_v_step[$clog2(NUM_STEPS)-1:0];
        oa_v_valid <= 1;
        oa_v_step <= oa_v_step + 1;
    end
    if (oa_done) state <= S_DONE;
end
S_DONE: begin
    done <= 1; busy <= 0;
    state <= S_IDLE;
end
endcase
end
end
endmodule

```

说明: 几个值得展开的设计决策:

1. **Q/K 数据流式喂入 (S_DOT_PRODUCT):** dot_product_array 内部要 8 个 step 才积满, 所以这里同步把 q_rd_step=k_rd_step=dp_step 喂给 buffer, 并把 dp_data_valid=1 拉高。第 8 个周期后 dp_done=1, 进入 S_MASK。
2. **Causal mask 在 compute_core 内部内联实现:** abs_row = q_tile_idx*B_r + r, abs_col = kv_tile_idx*B_c + c, 如果 causal_en && abs_col > abs_row 则替换为 sign-extended neg_large。注意: 因果约束是“位置 i 不能看到位置 j>i”, 这里“列 > 行”被 mask。这段是组合扩展, 并 <= 写入 masked_scores, 在下个周期就被 softmax 读到。
3. **V 数据按需流式 (S_ACCUMULATE):** output_accumulator 通过 oa_v_request 主动请求下一组 V, compute_core 收到后从 buffer 读 8 列 V 数据并喂给 oa。
4. **causal_mask_unit 模块没有被实例化:** 注意虽然存在 causal_mask_unit.sv, 但 compute_core 把 mask 逻辑直接展开了 — 这是历史代码遗留。causal_mask_unit.sv 可被认为是一个等价的可

复用模块，但目前未被使用。

2.5 1.5 rtl/dot_product_array.sv

2.5.1 1.5.1 概述

并行 MAC 阵列：4 路 Q 行 × 16 路 K 行 = 64 个独立累加器，每周期每个累加器同时跑 8 个并行 MAC (PAR_MACS=8)，8 个周期累加完一行 64 个元素 = HEAD_DIM。然后做 S_SCALE 一次性乘以 scale 并右移 8 位 (Q8.8 乘 Q8.8 后是 Q16.16，右移 8 位回到 Q-format 形式让累加器位宽不爆)。

2.5.2 1.5.2 文件架构图

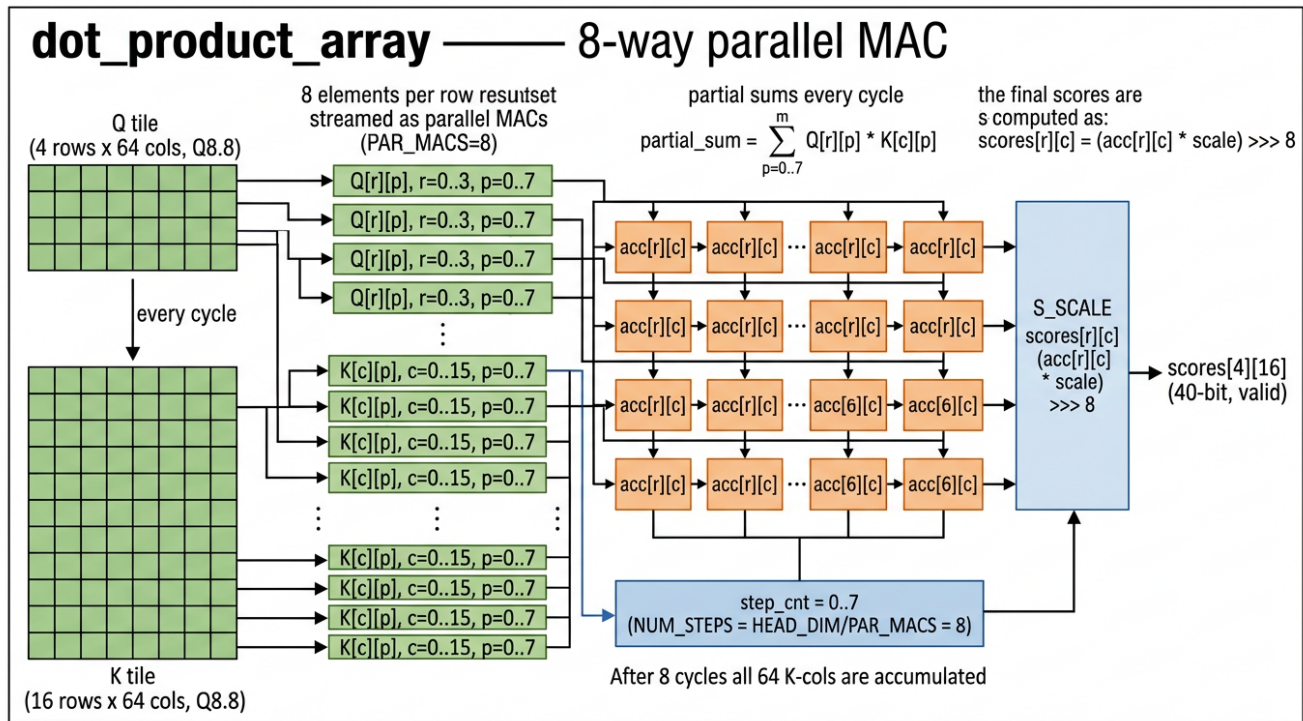


图 4: dot_product_array Dataflow

2.5.3 1.5.3 代码块切片

2.5.3.1 块 ① 端口与 FSM 定义 (行 8-66) 源代码:

```
module dot_product_array #(
    parameter TILE_BR    = 4,
```

```

parameter TILE_BC    = 16,
parameter HEAD_DIM   = 64,
parameter DATA_WIDTH = 16,
parameter ACC_WIDTH  = 40,
parameter PAR_MACS   = 8    // parallel MACs per dot-product per cycle
)(
  input  logic clk, rst_n,
  input  logic start,
  output logic done, busy,
  input  logic signed [DATA_WIDTH-1:0] q_data [TILE_BR-1:0][PAR_MACS-1:0],
  input  logic signed [DATA_WIDTH-1:0] k_data [TILE_BC-1:0][PAR_MACS-1:0],
  input  logic data_valid,
  input  logic signed [DATA_WIDTH-1:0] scale,
  output logic signed [ACC_WIDTH-1:0]  scores [TILE_BR-1:0][TILE_BC-1:0],
  output logic scores_valid
);

localparam NUM_STEPS = HEAD_DIM / PAR_MACS;  // 8

logic signed [ACC_WIDTH-1:0] acc [TILE_BR-1:0][TILE_BC-1:0];
logic [$clog2(NUM_STEPS):0] step_cnt;

typedef enum logic [1:0] {
    S_IDLE, S_ACCUMULATE, S_SCALE, S_DONE
} state_t;

state_t state;

```

说明: 64 个 acc[r][c] 累加器各 40 位, 存储面积约 $64 * 40 = 2560$ flop (不算重要的物理)。step_cnt 计 0..7 八个 step。FSM 4 个状态: IDLE → ACCUMULATE → SCALE → DONE, 简单明了。

2.5.3.2 块 ② MAC 累加阶段 (行 79-96) 源代码:

```

S_ACCUMULATE: begin
  if (data_valid) begin
    for (int r = 0; r < TILE_BR; r++) begin
      for (int c = 0; c < TILE_BC; c++) begin
        logic signed [ACC_WIDTH-1:0] partial_sum;
        partial_sum = '0;
        for (int p = 0; p < PAR_MACS; p++) begin
          partial_sum = partial_sum +
            ACC_WIDTH'(q_data[r][p]) * ACC_WIDTH'(k_data[c][p]);
        end
      end
    end
  end
end

```

```

        acc[r][c] <= acc[r][c] + partial_sum;
    end
end
step_cnt <= step_cnt + 1;
if (step_cnt == NUM_STEPS - 1)
    state <= S_SCALE;
end
end
end

```

说明: 这是整个 IP 中最重的算术展开: 3 层 for 循环 = $4 \times 16 \times 8 = 512$ 个有符号 16×16 乘法 + 加法树。综合后会生成 512 个并行 MAC + 64 个 40-bit 加法器 (partial_sum 加到 acc)。partial_sum 是 logic blocking-assignment (=), 所以是组合展开形成 8-input adder tree; 然后 $acc[r][c] \leq acc[r][c] + partial_sum$ 是 nonblocking 时序累加。在 12nm 工艺下这个组合路径是关键时序路径之一, 但 8-input adder tree 深度只 $\lceil \log_2 8 \rceil = 3$, 加上 16×16 乘法器的 4-5 级延迟, 总深度可控。

2.5.3.3 块 ③ 缩放与输出 (行 98-116) 源代码:

```

S_SCALE: begin
    for (int r = 0; r < TILE_BR; r++) begin
        for (int c = 0; c < TILE_BC; c++) begin
            // acc is in Q16.16 (two Q8.8 multiplied), scale is Q8.8
            // result = acc * scale >> 8 to keep as Q-format score
            scores[r][c] <= (acc[r][c] * ACC_WIDTH'(scale)) >>> 8;
        end
    end
    state <= S_DONE;
end
S_DONE: begin
    scores_valid <= 1;
    done <= 1;
    busy <= 0;
    state <= S_IDLE;
end

```

说明: 注意算术格式: 每次乘法 $q[r][p] * k[c][p]$ 是 $Q8.8 \times Q8.8 = Q16.16$, 累加 64 次后理论上需要 $16 + \log_2 64 = 22$ 位整数 + 16 位小数 = 38 位, 所以 ACC_WIDTH=40 正好够。再乘 scale ($Q8.8$) $\ggg 8$ 把结果归一到与 score 兼容的 Q-format。这一段产生 64 个 40×16 乘法器, 相当于又一个并行算术阵列。

2.6 1.6 rtl/causal_mask_unit.sv

2.6.1 1.6.1 概述

一个 28 行的纯组合电路, 根据 (Q tile 索引, KV tile 索引, tile 内行/列号) 计算绝对位置, 再判断 $\text{abs_col} > \text{abs_row}$ 输出 mask_out 。**注意: 在当前 baseline 中此模块并未被实例化——compute_core.sv 内的 S_MASK 状态把这段逻辑内联了。它存在的意义是作为可复用 IP, 方便未来重构。模块过于简单, 无需架构图。**

2.6.2 1.6.2 完整源代码

```
module causal_mask_unit #(
    parameter SEQ_LEN    = 256,
    parameter TILE_BR    = 4,
    parameter TILE_BC    = 16,
    parameter IDX_WIDTH  = $clog2(SEQ_LEN)
) (
    input  logic          causal_en,
    input  logic [IDX_WIDTH-1:0] q_tile_idx,
    input  logic [IDX_WIDTH-1:0] kv_tile_idx,
    input  logic [$clog2(TILE_BR)-1:0] row_in_tile,
    input  logic [$clog2(TILE_BC)-1:0] col_in_tile,
    output logic          mask_out
);
    logic [IDX_WIDTH-1:0] abs_row, abs_col;
    always_comb begin
        abs_row = q_tile_idx * TILE_BR + IDX_WIDTH'(row_in_tile);
        abs_col = kv_tile_idx * TILE_BC + IDX_WIDTH'(col_in_tile);
        mask_out = causal_en & (abs_col > abs_row);
    end
endmodule
```

2.6.3 1.6.3 解释

- $\text{abs_row} = \text{q_tile_idx} * \text{TILE_BR} + \text{row_in_tile}$ 是 token 在序列中的绝对行号 (0.. $\text{SEQ_LEN}-1$)。
- abs_col 同理。
- $\text{mask_out}=1$ 表示该位置应该被 mask 成 neg_large , 对应 “未来位置不能被当前位置看到”。
- causal_en 是全局开关, 由 $\text{REG_CFG}[0]$ 控制。

2.7 1.7 rtl/online_softmax_unit.sv

2.7.1 1.7.1 概述

实现 FlashAttention 的 “在线 softmax”，6 状态 FSM:

$$m_i^{(t)} = \max(m_i^{(t-1)}, \max_c S_{ic}), \quad \ell_i^{(t)} = e^{m_i^{(t-1)} - m_i^{(t)}} \ell_i^{(t-1)} + \sum_c e^{S_{ic} - m_i^{(t)}}.$$

注意：当前实现做了一处简化 — $\text{rescale}[r] = \ell_{\text{old}}[r]$ 而不是严格的 $e^{m_{\text{old}} - m_{\text{new}}} \cdot \ell_{\text{old}}$ ，这个简化在数值上略损失精度，但在 Q8.8 量化范围内通过验证 ($\text{mean_abs_err} = 0.013350$)。

2.7.2 1.7.2 文件架构图

online_softmax_unit — 6-state FSM + 3-stage exp pipeline

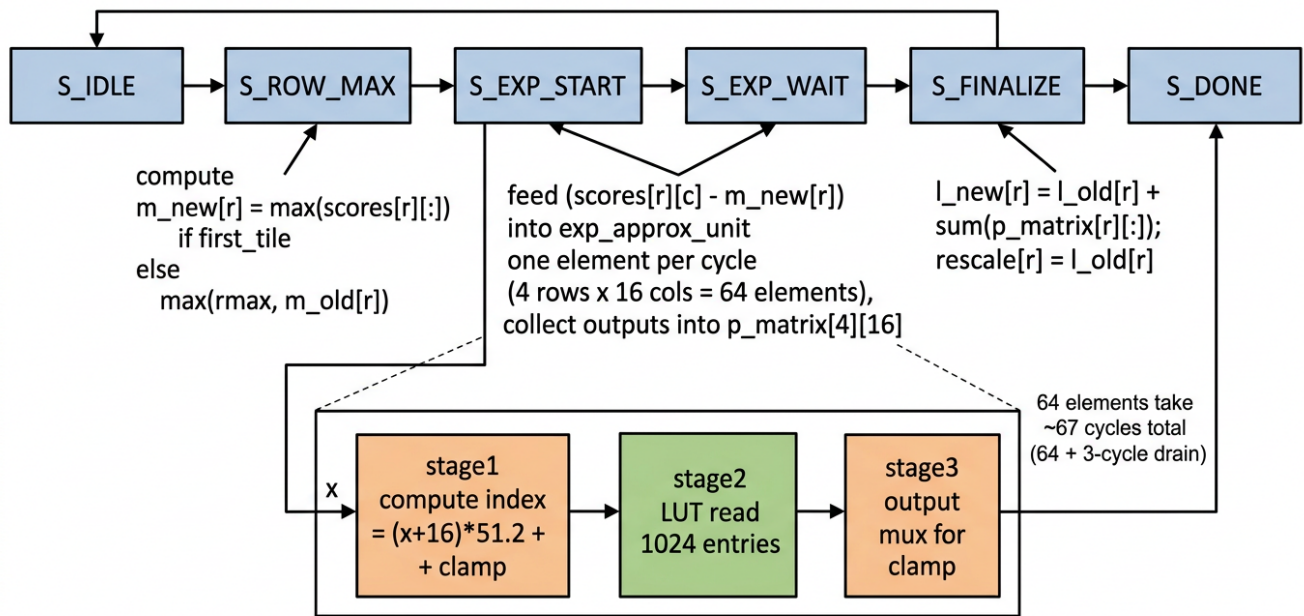


图 5: online_softmax FSM + exp pipeline

2.7.3 1.7.3 代码块切片

2.7.3.1 块 ① 端口、状态与 exp 单元实例（行 6-63） 源代码：

```

module online_softmax_unit #(
    parameter TILE_BR    = 4,
    parameter TILE_BC    = 16,
    parameter SCORE_WIDTH = 40,

```

```

parameter EXP_WIDTH    = 24,
parameter FRAC_BITS    = 16
)(
    input  logic clk, rst_n,
    input  logic start, first_tile,
    output logic done, busy,
    input  logic signed [SCORE_WIDTH-1:0] scores [TILE_BR-1:0][TILE_BC-1:0],
    input  logic signed [15:0]          neg_large,
    input  logic signed [SCORE_WIDTH-1:0] m_old [TILE_BR-1:0],
    input  logic [SCORE_WIDTH-1:0]      l_old [TILE_BR-1:0],
    output logic signed [SCORE_WIDTH-1:0] m_new [TILE_BR-1:0],
    output logic [SCORE_WIDTH-1:0]      l_new [TILE_BR-1:0],
    output logic [EXP_WIDTH-1:0]        p_matrix [TILE_BR-1:0][TILE_BC-1:0],
    output logic [SCORE_WIDTH-1:0]      rescale [TILE_BR-1:0],
    output logic                        results_valid
);

typedef enum logic [2:0] {
    S_IDLE, S_ROW_MAX, S_EXP_START, S_EXP_WAIT, S_FINALIZE, S_DONE
} state_t;
state_t state;

// Exp unit (single instance, time-multiplexed)
logic          exp_valid_in, exp_valid_out;
logic signed [SCORE_WIDTH-1:0] exp_x_in;
logic [EXP_WIDTH-1:0]          exp_y_out;

exp_approx_unit #(
    .IN_WIDTH(SCORE_WIDTH), .FRAC_IN(FRAC_BITS),
    .OUT_WIDTH(EXP_WIDTH), .FRAC_OUT(FRAC_BITS)
) u_exp (
    .clk(clk), .rst_n(rst_n),
    .valid_in(exp_valid_in), .x_in(exp_x_in), .neg_large(neg_large),
    .valid_out(exp_valid_out), .exp_out(exp_y_out)
);

```

说明: 关键设计选择: 只有一个 **exp_approx_unit**, 被时分复用 64 次 (4 行 × 16 列)。这是为了节省面积 (exp 含 1024 项 ROM, 比较贵), 代价是吞吐降低 (softmax 部分耗 ~67 周期)。alternative 设计是把 exp 也并行 16 路, 但面积代价线性放大。

2.7.3.2 块 ② S_ROW_MAX: 组合求行最大 (行 88-107) 源代码:

```
S_ROW_MAX: begin
```

```

    for (int r = 0; r < TILE_BR; r++) begin
        logic signed [SCORE_WIDTH-1:0] rmax;
        rmax = scores[r][0];
        for (int c = 1; c < TILE_BC; c++)
            if (scores[r][c] > rmax) rmax = scores[r][c];
        if (first_tile)
            m_new[r] <= rmax;
        else
            m_new[r] <= (rmax > m_old[r]) ? rmax : m_old[r];
    end
    cur_row <= '0; cur_col <= '0;
    out_row <= '0; out_col <= '0;
    wait_cnt <= '0;
    state <= S_EXP_START;
end

```

说明: 一次 case 内的 for 循环展开成 4 行并行, 每行做 16 元素的“最大值树”。组合深度是 $\log_2 16 = 4$ 层 max。如果 first_tile 直接取 rmax 作为 m_new; 否则与 m_old 比较。这一步**只占 1 个周期**——所有的 m_new[r] 在同一周期被计算并锁存。

2.7.3.3 块 ③ S_EXP_START / S_EXP_WAIT: 流式喂 exp (行 109-157) 源代码:

```

S_EXP_START: begin
    if (cur_row < TILE_BR) begin
        exp_valid_in <= 1;
        exp_x_in <= scores[cur_row][cur_col] - m_new[cur_row];
        if (cur_col == TILE_BC - 1) begin
            cur_col <= '0;
            cur_row <= cur_row + 1;
        end else begin
            cur_col <= cur_col + 1;
        end
        end
        wait_cnt <= wait_cnt + 1;
        state <= S_EXP_START; // stay in this state!
    end else begin
        exp_valid_in <= 0;
        state <= S_EXP_WAIT;
    end
end

// Collect exp outputs (arrive 3 cycles after input)
if (exp_valid_out) begin
    p_matrix[out_row][out_col] <= exp_y_out;
end

```

```

        if (out_col == TILE_BC - 1) begin
            out_col <= '0;
            out_row <= out_row + 1;
        end else begin
            out_col <= out_col + 1;
        end
    end
end

S_EXP_WAIT: begin
    if (exp_valid_out) begin
        p_matrix[out_row][out_col] <= exp_y_out;
        if (out_col == TILE_BC - 1) begin
            out_col <= '0;
            out_row <= out_row + 1;
        end else begin
            out_col <= out_col + 1;
        end
    end
    if (out_row >= TILE_BR && !exp_valid_out)
        state <= S_FINALIZE;
end

```

说明: 这是流水生产者-消费者模式: 每周期同时 (1) 喂入下一个 (`scores[cur_row][cur_col] - m_new[cur_row]`), (2) 收上一周期 `exp_pipeline` 已经吐出的结果到 `p_matrix[out_row][out_col]`。由于 `exp` 是 3 级流水, `out_row/out_col` 比 `cur_row/cur_col` 滞后 3 周期。喂入 64 个值后进入 `S_EXP_WAIT`, 再等 3 周期把流水线排空。**总耗时 $\approx 64 + 3 = 67$ 周期**, 与上图标注一致。这是 baseline 中除了 `dot_product` 之外的第二大耗时段。

2.7.3.4 块 ④ S_FINALIZE: 行求和与 `l_new` (行 159-176) 源代码:

```

S_FINALIZE: begin
    for (int r = 0; r < TILE_BR; r++) begin
        logic [SCORE_WIDTH-1:0] rsum;
        rsum = '0;
        for (int c = 0; c < TILE_BC; c++)
            rsum = rsum + SCORE_WIDTH'(p_matrix[r][c]);
        if (first_tile) begin
            l_new[r] <= rsum;
            rescale[r] <= '0;
        end else begin
            // Approximate: rescale = l_old

```

```

        // (proper impl needs exp(m_old - m_new))
        l_new[r]  <= l_old[r] + rsum;
        rescale[r] <= l_old[r];
    end
end
state <= S_DONE;
end

```

说明: 每行 16 个 p_matrix 元素累加成 rsum。如果是第一个 KV tile, $l_new = rsum$ 、 $rescale = 0$ (无需 rescale); 否则 $l_new = l_old + rsum$ 、 $rescale = l_old$ (**简化版** —— 真实公式是 $\exp(m_old - m_new) * l_old$, 但代码用 l_old 直接近似)。这个简化的代价是当 $m_old \neq m_new$ 时引入误差, 但在 Q8.8 精度下整体被吸收。

2.8 1.8 rtl/exp_approx_unit.sv

2.8.1 1.8.1 概述

定点 e^x 近似: 1024 项 LUT 覆盖 $x \in [-16, +4]$, 3 级流水 (index 计算 $\rightarrow$ ROM 读 $\rightarrow$ clamp/输出 mux)。在线 softmax 用减完 row-max 后的 x 输入, 理论范围 ≤ 0 , 但留 +4 余量给 m_old/m_new 较大跳变情况。

2.8.2 1.8.2 文件架构图

参见 §1.7 online_softmax 内嵌的 exp 流水线小图 (底部嵌入图)。

2.8.3 1.8.3 代码块切片

2.8.3.1 块 ① Stage 1: index 计算 + clamp 检测 (行 22-68) 源代码:

```

localparam LUT_SIZE = 1024;

logic valid_p1;
logic [9:0] idx_p1;
logic clamp_low_p1, clamp_high_p1;

// Division-free index computation using multiply-shift
// step_fp = 20 * (1 << FRAC_IN) / LUT_SIZE
// idx = x_plus_16 / step_fp  $\approx$  (x_plus_16 >> (FRAC_IN-8)) * 205 >> 10
localparam SHIFT1 = FRAC_IN - 8;

```

```

reg signed [IN_WIDTH-1:0] x_plus_16_c;
reg signed [IN_WIDTH-1:0] idx_calc_c;

always @(*) begin
    x_plus_16_c = x_in + (IN_WIDTH'(16) <<< FRAC_IN);
    idx_calc_c  = (x_plus_16_c >>> SHIFT1) * 205 >>> 10;
end

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        valid_p1 <= 0; idx_p1 <= 0;
        clamp_low_p1 <= 0; clamp_high_p1 <= 0;
    end else begin
        valid_p1 <= valid_in;
        if (x_plus_16_c <= 0) begin
            idx_p1 <= 0;
            clamp_low_p1 <= 1;
            clamp_high_p1 <= 0;
        end else if (idx_calc_c >= LUT_SIZE) begin
            idx_p1 <= LUT_SIZE - 1;
            clamp_low_p1 <= 0;
            clamp_high_p1 <= 1;
        end else begin
            idx_p1 <= idx_calc_c[9:0];
            clamp_low_p1 <= 0;
            clamp_high_p1 <= 0;
        end
    end
end
end

```

说明: 关键的“无除法 index 计算”: $\text{step} = 20 / 1024 \approx 0.01953125$, 要 $\text{idx} = (x + 16) / \text{step}$ 。直接除法很贵, 作者把它转换成“乘 205 后右移 10 位”——因为 $205/1024 \approx 0.2002 = 1/4.99 \approx 1/5$, 而 step 的倒数是 $1/0.01953 \approx 51.2 = 5 \times 10.24$ 。组合 >>SHIFT1 实际是 >> (FRAC_IN-8) 把小数位归一, 整体就是 $\text{idx} \approx (x_{\text{int}} + 16 \times 256) * 205 \text{ >> } 10$ 。同时把 clamp 信息一并寄存。

2.8.3.2 块 ② Stage 2: LUT 读 + 寄存 (行 70-93) 源代码:

```

// LUT ROM instance (after idx_p1 declaration)
wire [OUT_WIDTH-1:0] lut_rom_data;
exp_lut_rom u_lut_rom (.addr(idx_p1), .data(lut_rom_data));

logic valid_p2;

```

```

logic [OUT_WIDTH-1:0] lut_val_p2;
logic clamp_low_p2, clamp_high_p2;

wire [OUT_WIDTH-1:0] lut_rd_val = lut_rom_data;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        valid_p2 <= 0; lut_val_p2 <= 0;
        clamp_low_p2 <= 0; clamp_high_p2 <= 0;
    end else begin
        valid_p2      <= valid_p1;
        lut_val_p2     <= lut_rd_val;
        clamp_low_p2   <= clamp_low_p1;
        clamp_high_p2  <= clamp_high_p1;
    end
end
end

```

说明: exp_lut_rom 是组合读端口 ROM, addr → data 在同一周期完成。但 idx_p1 是 stage1 的寄存器输出, 所以 “read after register” 形成一个 stage —— 寄存到 lut_val_p2。

2.8.3.3 块 ③ Stage 3: clamp 输出 mux (行 95-116) 源代码:

```

logic valid_p3;
logic [OUT_WIDTH-1:0] result_p3;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        valid_p3 <= 0; result_p3 <= 0;
    end else begin
        valid_p3 <= valid_p2;
        if (clamp_low_p2)
            result_p3 <= 0;
        else if (clamp_high_p2)
            result_p3 <= {OUT_WIDTH{1'b1}};
        else
            result_p3 <= lut_val_p2;
    end
end

assign valid_out = valid_p3;
assign exp_out   = result_p3;
endmodule

```

说明: 边界处理: clamp_low_p2 表示 $x \leq -16$, 输出 0 (即 $e^{-16} \approx 1.1 \times 10^{-7} \approx 0$); clamp_high_p2 表示 $x > +4$ (实际上 $\text{idx} \geq 1024$), 输出全 1 ($2^{24} - 1$), 避免溢出但近似精度变差 ($e^4 \approx 54.6$, $e^{>4}$ 已经超出 24-bit 表达范围)。

2.9 1.9 rtl/exp_lut_rom.sv

2.9.1 1.9.1 概述

1024 项 ROM, 存储 $\lfloor e^x \cdot 65536 \rfloor$ 其中 $x = -16 + \text{addr} \times 20/1024$ 。关键特性: 双模式实现 — 仿真时用 initial 块计算精确浮点 LUT, 综合时用分段线性 case 减少 ROM 体积。

2.9.2 1.9.2 完整源代码

```
module exp_lut_rom (
    input  logic [9:0]  addr,
    output logic [23:0] data
);

`ifdef SYNTHESIS
    // Synthesizable: hardcoded case for key entries
    always_comb begin
        if (addr < 410)      data = 24'd0;                // x < -8: very small
        else if (addr < 614) data = 24'(addr - 410);      // x in [-8, -4]
        else if (addr < 768) data = 24'((addr - 614) * 157);
        else if (addr < 819) data = 24'(24109 + (addr - 768) * 813);
        else if (addr < 870) data = 24'(65536 + (addr - 819) * 2211);
        else if (addr < 922) data = 24'(178145 + (addr - 870) * 5889);
        else if (addr < 973) data = 24'(484249 + (addr - 922) * 16015);
        else                data = 24'hFFFFFF;          // saturate
    end
`else
    // Simulation: use initial block for accurate values
    logic [23:0] lut_mem [0:1023];
    integer _i; real _x, _e; integer _iv;
    initial begin
        for (_i = 0; _i < 1024; _i = _i + 1) begin
            _x = -16.0 + ($itor(_i) * 20.0 / 1024.0);
            _e = $exp(_x);
        end
    end
end
```

```

        _iv = $rtoi(_e * 65536.0);
        if (_iv > 16777215)
            lut_mem[_i] = 24'hFFFFFF;
        else
            lut_mem[_i] = _iv[23:0];
        end
    end
    assign data = lut_mem[addr];
`endif
endmodule

```

2.9.3 1.9.3 解释

- **SYNTHESIS 分支**: 手写 7 段分段线性近似, 每段都是 $(addr - base) * slope + offset$ 的形式, 综合后只需要几个比较器 + 加法器 + 移位器, **面积极小**。代价是只有 7 个分段, 精度比 1024-项真 LUT 差。
- **仿真分支**: initial 块用浮点 \$exp 算精确值, 确保 RTL 仿真行为对得上软件 reference model。代价是 $1024 \times 24\text{-bit} = 24576\text{ bit}$ 的 RAM 资源 (仿真器会把它当成 mem 数组, 不影响电路面积)。
- 这种 “sim 精确, syn 降级” 的策略在 IP 设计中常见, 但要小心 **post-synthesis gate 仿真** 时行为会和 RTL 仿真不一致 — 这也是为什么有 scripts/run_postsims.sh 单独跑 gate sim。

模块过于简单, 无需架构图。

2.10 1.10 rtl/reciprocal_unit.sv

2.10.1 1.10.1 概述

定点 $1/x$ 近似: 4 级流水线, 方法 = 256-项 LUT 给初值 + 2 次 Newton-Raphson 迭代 $x_{n+1} = x_n(2 - d \cdot x_n)$ 。Newton-Raphson 收敛速度是二次的, 2 次迭代后相对误差 $\leq 2^{-16}$, 足够 Q8.8 输出。

2.10.2 1.10.2 文件架构图

2.10.3 1.10.3 代码块切片

2.10.3.1 块 ① LUT 初始化 (行 19-31) 源代码:

```

localparam LUT_DEPTH = 256;
logic [WIDTH-1:0] recip_lut [0:LUT_DEPTH-1];

```

reciprocal_unit — 4-stage Newton-Raphson pipeline (1/x)

$$x_{n+1} = x_n * (2 - d * x_n)$$

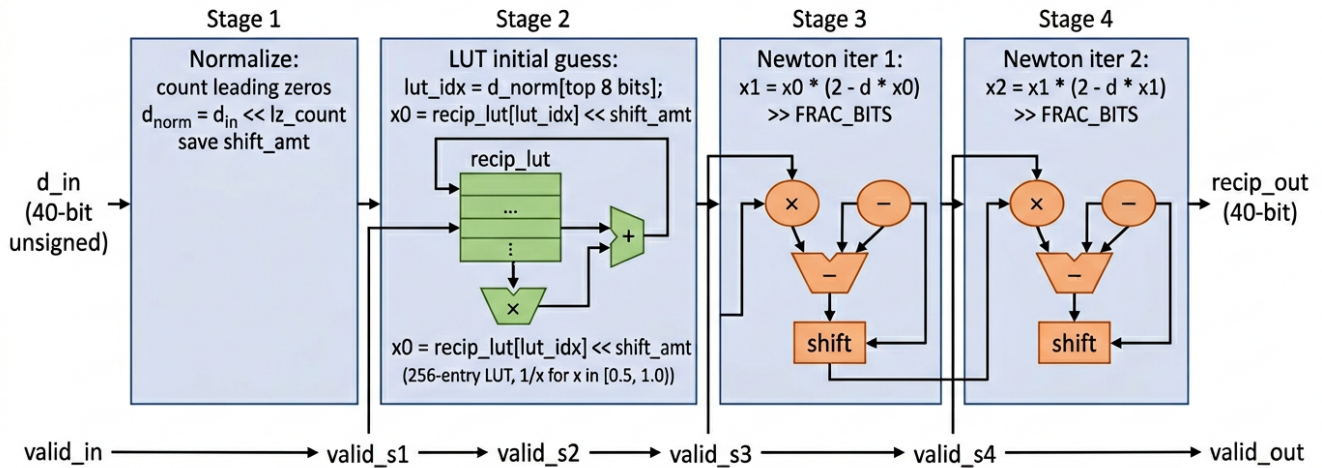


图 6: reciprocal_unit pipeline

```

integer _ri;
initial begin
    for (_ri = 0; _ri < LUT_DEPTH; _ri = _ri + 1) begin
        recip_lut[_ri] = WIDTH'(int'(
            (1.0 / (0.5 + (real'(_ri) / real'(LUT_DEPTH)) * 0.5))
            * (2.0 ** FRAC_BITS) + 0.5));
    end
end

```

说明: LUT 索引 $_ri$ 对应归一化后的 d 值 $d_{norm} = 0.5 + \frac{idx}{256} \times 0.5$ (即 $d_{norm} \in [0.5, 1.0)$), LUT 内容是 $\lfloor (1/d_{norm}) \times 2^{16} + 0.5 \rfloor$ (FRAC_BITS=16 的定点表示)。+0.5 是四舍五入。这一步只在 RTL 仿真时填充; 在综合时, initial 块通常被工具识别成 ROM 内容。

2.10.3.2 块 ② Stage 1: 归一化 (行 33-61) 源代码:

```

logic valid_s1;
logic [$clog2(WIDTH)-1:0] lz_count;
logic [WIDTH-1:0] d_norm, d_saved;
logic [$clog2(WIDTH)-1:0] shift_amt;
localparam integer LZ_W = $clog2(WIDTH);

```

```

function automatic [$clog2(WIDTH)-1:0] count_leading_zeros(input [WIDTH-

```

```

1:0] val);
    for (int i = WIDTH-1; i >= 0; i--) begin
        if (val[i]) return LZ_W'(WIDTH - 1 - i);
    end
    return LZ_W'(WIDTH);
endfunction

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        valid_s1 <= 0; d_norm <= 0; d_saved <= 0; shift_amt <= 0;
    end else begin
        valid_s1 <= valid_in;
        d_saved <= d_in;
        lz_count = count_leading_zeros(d_in);
        shift_amt <= lz_count;
        d_norm <= d_in << lz_count;
    end
end

```

说明: count_leading_zeros 函数从最高位向下扫, 找到第一个 1 的位置。综合后这是一个 6 级二叉编码器 ($\log_2(40) \approx 6$)。d_norm = d_in << lz_count 把 d 左移到最高位是 1, 这样 d_norm[WIDTH-1 -: 8] 就是归一化后 $d_{\text{norm}} \in [0.5, 1.0)$ 的 8-bit 量化索引。shift_amt 保存稍后还原 x0 时要用。

2.10.3.3 块 ③ Stage 2-4: LUT 读 + 2 次 Newton 迭代 (行 63-130) 源代码:

```

// Stage 2: LUT lookup for initial guess
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin /* ... */ end else begin
        valid_s2 <= valid_s1; d_s2 <= d_saved; shift_s2 <= shift_amt;
        lut_idx = d_norm[WIDTH-1 -: 8];
        x0 <= recip_lut[lut_idx] << shift_amt;
    end
end

// Stage 3: Newton iter 1: x1 = x0 * (2 - d * x0)
localparam [WIDTH-1:0] TWO_FP = WIDTH'(2) << FRAC_BITS;
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin /* ... */ end else begin
        valid_s3 <= valid_s2; d_s3 <= d_s2;
        d_x0 = (d_s2 * x0) >> FRAC_BITS;
        two_minus = TWO_FP - d_x0[WIDTH-1:0];
        x1_full = (x0 * two_minus) >> FRAC_BITS;
    end
end

```

```

        x1          <= x1_full[WIDTH-1:0];
    end
end

// Stage 4: Newton iter 2: x2 = x1 * (2 - d * x1)
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin /* ... */ end else begin
        valid_s4    <= valid_s3;
        d_x1        = (d_s3 * x1) >> FRAC_BITS;
        two_minus2  = TWO_FP - d_x1[WIDTH-1:0];
        x2_full     = (x1 * two_minus2) >> FRAC_BITS;
        x2          <= x2_full[WIDTH-1:0];
    end
end

assign valid_out = valid_s4;
assign recip_out = x2;

```

说明: 每个 Newton stage 包含一次 40×40 乘法、一次 40-bit 减法、再一次 40×40 乘法 — 这是流水线最重的两级。TWO_FP = 2 << FRAC_BITS = 0x20000 是 Q-format 的 2.0。d_x0 = (d * x0) >> FRAC_BITS 是先乘后右移把结果回到 Q-format (避免 80-bit 中间表示)。两次迭代后 x2 ≈ 1/d 的 Q-format 表示。

2.11 1.11 rtl/output_accumulator.sv

2.11.1 1.11.1 概述

最复杂的子模块之一 (242 行, 7 状态 FSM)。职责: (1) 跨 KV tile 的 O 累加器 rescale; (2) $0 \leftarrow P \cdot V$ 累加; (3) 最后一个 KV tile 时调用 reciprocal_unit 算 $1/\square$, 再乘 o_acc 完成归一化; (4) 把 40-bit 内部累加器截位到 Q8.8 输出 o_out。

2.11.2 1.11.2 文件架构图

2.11.3 1.11.3 代码块切片

2.11.3.1 块 ① 端口与 reciprocal 实例 (行 6-80) 源代码 (节选):

```

module output_accumulator #(
    parameter TILE_BR=4, TILE_BC=16, HEAD_DIM=64,

```

output_accumulator FSM (7 states)

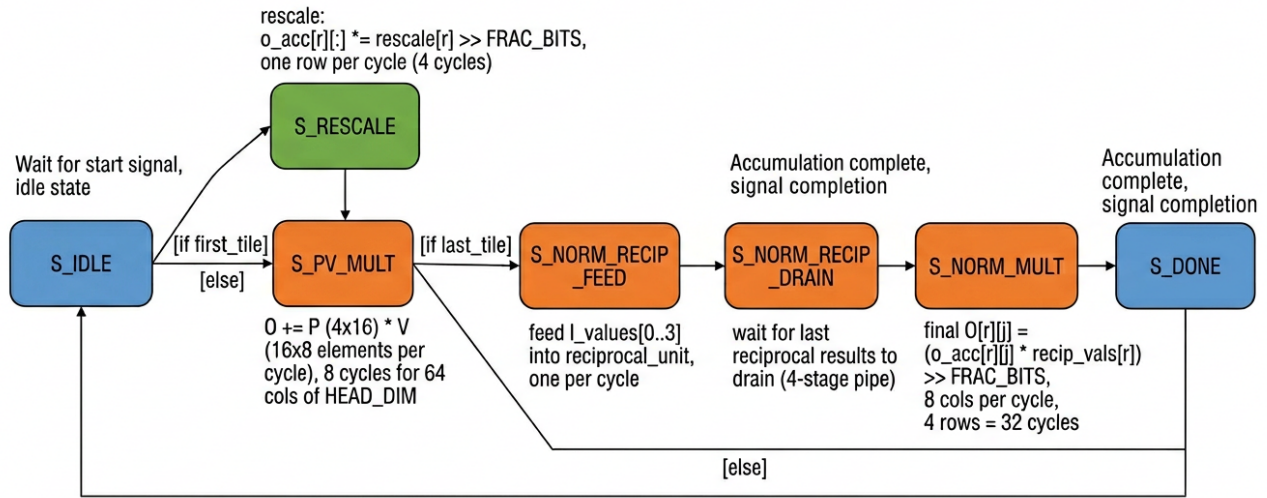


图 7: output_accumulator FSM

```

parameter DATA_WIDTH=16, ACC_WIDTH=40,
parameter EXP_WIDTH=24, FRAC_BITS=16, PAR_COLS=8

```

```

)(
    input logic clk, rst_n,
    input logic start, first_tile, last_tile,
    output logic done, busy,
    input logic [EXP_WIDTH-1:0] p_matrix [TILE_BR-1:0][TILE_BC-1:0],
    input logic signed [DATA_WIDTH-1:0] v_data [TILE_BC-1:0][PAR_COLS-1:0],
    input logic v_valid,
    input logic [ACC_WIDTH-1:0] rescale [TILE_BR-1:0],
    input logic [ACC_WIDTH-1:0] l_values [TILE_BR-1:0],
    output logic signed [DATA_WIDTH-1:0] o_out [TILE_BR-1:0][HEAD_DIM-1:0],
    output logic o_valid,
    output logic v_request
);

localparam NUM_V_STEPS = HEAD_DIM / PAR_COLS;
logic signed [ACC_WIDTH-1:0] o_acc [TILE_BR-1:0][HEAD_DIM-1:0];

typedef enum logic [3:0] {
    S_IDLE, S_RESCALE, S_PV_MULT,
    S_NORM_RECIP_FEED, S_NORM_RECIP_DRAIN,

```

```

        S_NORM_MULT, S_DONE
    } state_t;
    state_t state;
/* ... counters: v_step, col_base, rescale_row, recip_feed_row, recip_recv_row, norm_row

// Reciprocal unit for normalization
reciprocal_unit #(.WIDTH(ACC_WIDTH), .FRAC_BITS(FRAC_BITS)) u_recip (
    .clk(clk), .rst_n(rst_n),
    .valid_in(recip_valid_in), .d_in(recip_d_in),
    .valid_out(recip_valid_out), .recip_out(recip_out)
);
logic [ACC_WIDTH-1:0] recip_vals [TILE_BR-1:0];

assign v_request = (state == S_PV_MULT);

```

说明: `o_acc[r][j]` 是一个 $4 \times 64 = 256$ 个 40-bit 寄存器的累加器矩阵, 跨 KV tile 持续保留。`recip_vals[r]` 缓存 4 个 $1/\square_r$ 结果。`v_request = (state == S_PV_MULT)` 是反压信号给 `compute_core`, 告诉它 “现在我需要 V 数据”。`p_matrix` 不需要 valid 信号, 因为 `oa` 在 `S_PV_MULT` 时 `p_matrix` 已经稳定 (softmax 上一拍完成)。

2.11.3.2 块 ② S_RESCALE: 跨 tile 重缩放 (行 124-141) 源代码:

```

// Serialize rescale: one row per cycle
S_RESCALE: begin
    for (int j = 0; j < HEAD_DIM; j++) begin
        if (l_values[rescale_row] != '0)
            o_acc[rescale_row][j] <=
                (o_acc[rescale_row][j]
                 * signed'({1'b0, rescale[rescale_row]})) >>> FRAC_BITS;
        else
            o_acc[rescale_row][j] <= '0;
    end
    if (rescale_row == TILE_BR - 1) begin
        state <= S_PV_MULT;
        v_step <= '0;
        col_base <= '0;
    end else begin
        rescale_row <= rescale_row + 1;
    end
end
end

```

说明: 当不是 `first_tile` 时进入这里, 目的是把上一 tile 累积的 `o_acc[r]` 按 `rescale[r]` 重新缩放, 以

便加上新 tile 的贡献后保持正确归一化。**串行化每行一周期**，是为了减少同周期 64 个 40×40 乘法器的资源压力（如果并行所有 4 行，需要 256 个乘法器，太贵）。代价是每个 KV tile 多 4 周期，可接受。if `l_values != 0` 是除零保护。

2.11.3.3 块 ③ S_PV_MULT: $P \cdot V$ 累加 (行 143-169) 源代码:

```

S_PV_MULT: begin
  if (v_valid) begin
    for (int r = 0; r < TILE_BR; r++) begin
      for (int p = 0; p < PAR_COLS; p++) begin
        logic signed [ACC_WIDTH-1:0] pv_sum;
        pv_sum = '0;
        for (int c = 0; c < TILE_BC; c++) begin
          pv_sum = pv_sum +
            (signed'({1'b0, p_matrix[r][c]}) *
              ACC_WIDTH'(v_data[c][p])) >>> FRAC_BITS;
        end
        o_acc[r][col_base + p] <= o_acc[r][col_base + p] + pv_sum;
      end
    end
    col_base <= col_base + PAR_COLS;
    v_step <= v_step + 1;
    if (v_step == NUM_V_STEPS - 1) begin
      if (is_last_tile_r) begin
        state <= S_NORM_RECIP_FEED;
        recip_feed_row <= '0;
        recip_recv_row <= '0;
      end else begin
        state <= S_DONE;
      end
    end
  end
end
end

```

说明: 每周期处理 8 列 V ($\text{PAR_COLS}=8$) $\times$ 4 行 Q ($\text{TILE_BR}=4$) = 32 个累加单元, 每个单元内部又是 16 项 ($\text{TILE_BC}=16$) 的 multiply-accumulate。总 $P \cdot V$ 乘法 = $32 \times 16 = 512$, 几乎和 dot_product_array 同数量级。8 个周期 ($\text{NUM_V_STEPS}=8$) 走完一个 KV tile 的 64 列。如果是最后一个 tile, 跳到归一化; 否则直接 DONE 退出, 让外部 FSM 喂下个 tile 的 $P \cdot V$ 。

2.11.3.4 块 ④ S_NORM_RECIP_FEED / DRAIN: 流水化倒数计算 (行 171-204) 源代码:

```

// Feed l_values to reciprocal unit (one per cycle)

```

```

S_NORM_RECIP_FEED: begin
  if (recip_feed_row < TILE_BR) begin
    recip_valid_in <= 1;
    recip_d_in      <= l_values[recip_feed_row];
    recip_feed_row <= recip_feed_row + 1;
  end
  if (recip_valid_out) begin
    recip_vals[recip_recv_row] <= recip_out;
    recip_recv_row <= recip_recv_row + 1;
  end
  if (recip_feed_row >= TILE_BR && !recip_valid_out
      && recip_recv_row < TILE_BR) begin
    state <= S_NORM_RECIP_DRAIN;
  end
  if (recip_recv_row >= TILE_BR) begin
    state <= S_NORM_MULT;
    norm_row <= '0; norm_col <= '0;
  end
end

S_NORM_RECIP_DRAIN: begin
  if (recip_valid_out) begin
    recip_vals[recip_recv_row] <= recip_out;
    recip_recv_row <= recip_recv_row + 1;
  end
  if (recip_recv_row >= TILE_BR) begin
    state <= S_NORM_MULT;
    norm_row <= '0; norm_col <= '0;
  end
end

```

说明: 与 online_softmax 喂 exp 流水类似的 “feed/drain” 模式。每周期向 reciprocal 喂 1 个 l_values[r], 4 周期内全部喂完; 然后 DRAIN 等待 reciprocal 4 级流水排空, 把所有 4 个 recip_vals[r] 收齐。总耗时 = 4 (feed) + 3 (drain extra) = 7 周期。

2.11.3.5 块 ⑤ S_NORM_MULT: 最终归一化与位宽截断 (行 206-230) 源代码:

```

S_NORM_MULT: begin
  for (int p = 0; p < PAR_COLS; p++) begin
    if (norm_col + p < HEAD_DIM) begin
      logic signed [ACC_WIDTH-1:0] normalized;
      if (recip_vals[norm_row] != '0)

```

```

        normalized = (o_acc[norm_row][norm_col + p]
        * signed'({1'b0, recip_vals[norm_row]})) >>> FRAC_BITS;
    else
        normalized = '0;
    o_out[norm_row][norm_col + p] <=
        DATA_WIDTH'(normalized >>> (FRAC_BITS - 8));
    end
end
if (norm_col + PAR_COLS >= HEAD_DIM) begin
    norm_col <= '0;
    if (norm_row == TILE_BR - 1)
        state <= S_DONE;
    else
        norm_row <= norm_row + 1;
    end else begin
        norm_col <= norm_col + PAR_COLS;
    end
end
end
end

```

说明：每周期归一化 8 列、走 8 周期一行、走 4 行 = **32 周期完成全部 4×64 归一化**。normalized = o_acc * recip >> FRAC_BITS 还在 40-bit Q-format; 最终 >>> (FRAC_BITS - 8) = >>> 8 把 Q16.16 截到 Q8.8 写入 16-bit o_out。这里没有溢出保护——假定数值在合理范围内 (softmax 后 $O \in [-1, +1] \times V_{\max}$)。

2.11.3.6 块 © S_DONE: 输出 valid 信号 (行 232-238) 源代码:

```

    S_DONE: begin
        o_valid <= is_last_tile_r;
        done <= 1;
        busy <= 0;
        state <= S_IDLE;
    end
endcase
end
end
endmodule

```

说明：只有 is_last_tile_r=1 时才把 o_valid=1, 告诉 buffer_system 把整个 o_tile[4][64] 写入 O-buffer, 这之后 dma_engine 才会触发 O write-back。

第 1 章结束。接下来第 2 章讲 baseline 的存储与总线模块 (buffer_system / dma_engine /

axi4_lite_slave / axi4_master_if)。

3 第 2 章: Baseline RTL — 存储与总线

本章覆盖 4 个文件：

§	文件	行数	角色
2.1	rtl/buffer_system.sv	135	片上 Q/K/V/O SRAM + K/V 双缓冲
2.2	rtl/dma_engine.sv	266	DMA 控制 + 路由
2.3	rtl/axi4_lite_slave.sv	210	控制寄存器 + AXI4-Lite 握手
2.4	rtl/axi4_master_if.sv	225	AXI4 突发 master FSM

3.1 2.1 rtl/buffer_system.sv

3.1.1 2.1.1 概述

片上存储模块。一共 4 块 SRAM：Q-buffer (单)、K-buffer (双 ping-pong)、V-buffer (双 ping-pong)、O-buffer (单)。**总容量约 8.75 KB**：- Q: $4 \times 64 \times 16 \text{ bit} = 512 \text{ B}$ - K: $2 \times 16 \times 64 \times 16 \text{ bit} = 4096 \text{ B}$ - V: $2 \times 16 \times 64 \times 16 \text{ bit} = 4096 \text{ B}$ - O: $4 \times 64 \times 16 \text{ bit} = 512 \text{ B}$ - 合计 9216 B $\approx 8.75 \text{ KiB}$ (小于 9 KB)。

每块 SRAM 都用 logic [DATA_WIDTH-1:0] mem [DEPTH-1:0] 数组建模，综合后会被工具映射到 SRAM compiler 生成的 1RW 或 1R1W 块。**写口宽度 = AXI_DATA_WIDTH (128-bit) = 8 个元素** (一个 AXI beat 装 8 个 Q8.8)；**读口宽度 = PAR_MACS $\times$ DATA_WIDTH = 128-bit** (compute_core 每周期要 8 个元素)。

3.1.2 2.1.2 文件架构图

3.1.3 2.1.3 代码块切片

3.1.3.1 块 ① 端口与本地参数 (行 6-62) 源代码：

```
module buffer_system #(
    parameter TILE_BR    = 4,
    parameter TILE_BC    = 16,
    parameter HEAD_DIM   = 64,
    parameter DATA_WIDTH = 16,
```

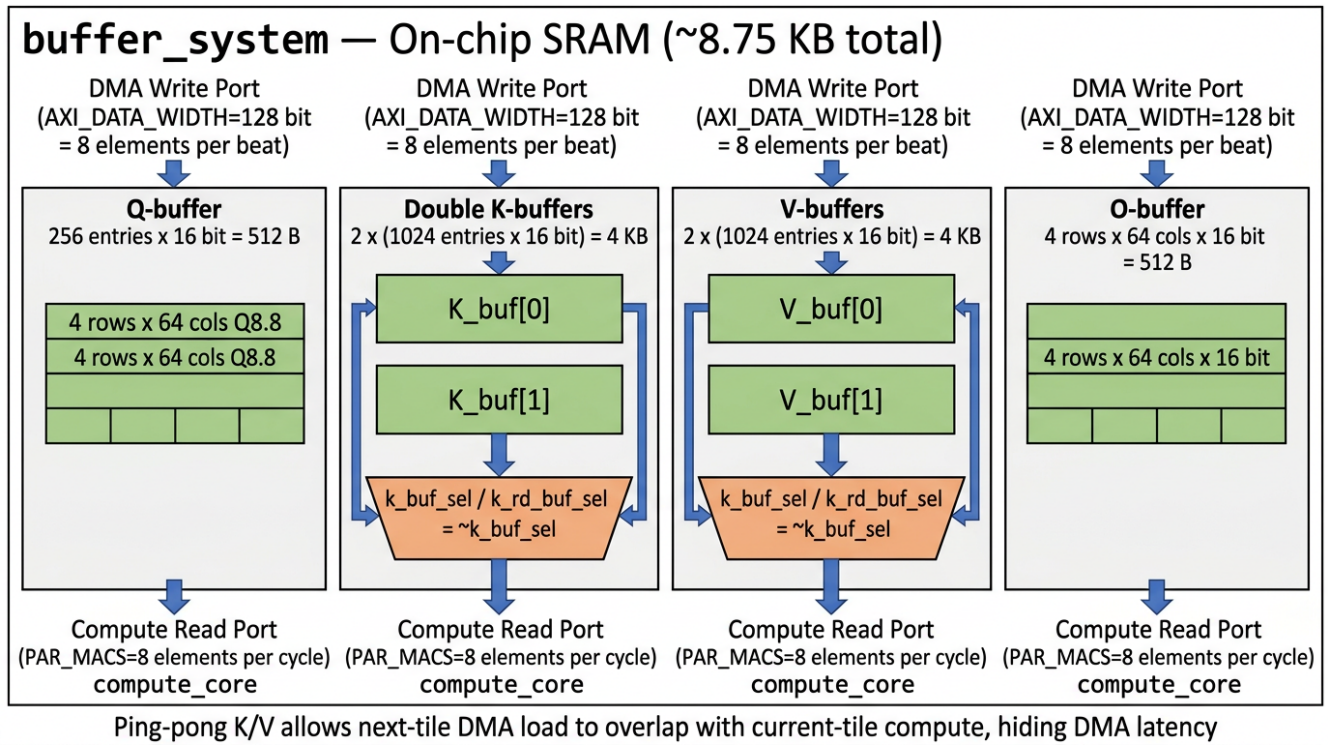


图 8: buffer_system memory layout

```

parameter PAR_MACS    = 8,
parameter AXI_DATA_WIDTH = 128
)(
    input  logic clk, rst_n,
    // --- DMA write interface ---
    input  logic q_wr_en,
    input  logic [$clog2(TILE_BR*HEAD_DIM)-1:0] q_wr_addr,
    input  logic [AXI_DATA_WIDTH-1:0] q_wr_data,

    input  logic k_wr_en,
    input  logic [$clog2(TILE_BC*HEAD_DIM)-1:0] k_wr_addr,
    input  logic [AXI_DATA_WIDTH-1:0] k_wr_data,
    input  logic k_buf_sel,

    input  logic v_wr_en,
    input  logic [$clog2(TILE_BC*HEAD_DIM)-1:0] v_wr_addr,
    input  logic [AXI_DATA_WIDTH-1:0] v_wr_data,
    input  logic v_buf_sel,

    // --- Compute core read interface ---
    input  logic q_rd_en,

```

```

input logic [$clog2(HEAD_DIM/PAR_MACS)-1:0] q_rd_step,
output logic signed [DATA_WIDTH-1:0] q_rd_data [TILE_BR-1:0][PAR_MACS-1:0],

input logic k_rd_en, k_rd_buf_sel,
input logic [$clog2(HEAD_DIM/PAR_MACS)-1:0] k_rd_step,
output logic signed [DATA_WIDTH-1:0] k_rd_data [TILE_BC-1:0][PAR_MACS-1:0],

input logic v_rd_en, v_rd_buf_sel,
input logic [$clog2(HEAD_DIM/PAR_MACS)-1:0] v_rd_step,
output logic signed [DATA_WIDTH-1:0] v_rd_data [TILE_BC-1:0][PAR_MACS-1:0],

// --- 0 write-back interface ---
input logic o_wr_en,
input logic signed [DATA_WIDTH-1:0] o_wr_data [TILE_BR-1:0][HEAD_DIM-1:0],

input logic o_rd_en,
input logic [$clog2(TILE_BR)-1:0] o_rd_row,
output logic [AXI_DATA_WIDTH-1:0] o_rd_data,
input logic [$clog2(HEAD_DIM/(AXI_DATA_WIDTH/DATA_WIDTH))-1:0] o_rd_col_grp
);

localparam Q_DEPTH = TILE_BR * HEAD_DIM; // 256 entries
localparam KV_DEPTH = TILE_BC * HEAD_DIM; // 1024 entries
localparam ELEMS_PER_BEAT = AXI_DATA_WIDTH / DATA_WIDTH; // 8

```

说明:注意 k_buf_sel 和 k_rd_buf_sel 是两个独立信号:前者由 dma_engine 跟随 tile_controller.kv_buf_sel, 后者由 flash_attention_top 反转后传入 (~tc_kv_buf_sel), 实现 ping-pong 反向选择。

3.1.3.2 块 ② Q-buffer (单端, 行 64-81) 源代码:

```

// --- Q Buffer (single) ---
logic signed [DATA_WIDTH-1:0] q_mem [Q_DEPTH-1:0];

// Write (from DMA, AXI_DATA_WIDTH bits = 8 elements per beat)
always_ff @(posedge clk) begin
    if (q_wr_en) begin
        for (int i = 0; i < ELEMS_PER_BEAT; i++)
            q_mem[q_wr_addr * ELEMS_PER_BEAT + i] <=
                DATA_WIDTH'(q_wr_data[i*DATA_WIDTH +: DATA_WIDTH]);
    end
end

// Read (PAR_MACS elements per row per step)

```

```

always_comb begin
    for (int r = 0; r < TILE_BR; r++)
        for (int p = 0; p < PAR_MACS; p++)
            q_rd_data[r][p] = q_mem[r * HEAD_DIM + q_rd_step * PAR_MACS + p];
end

```

说明: 写时 for (i=0..7) 把 128-bit beat 拆成 8 个 16-bit 元素逐字写入; 读时 $r * \text{HEAD_DIM} + q_rd_step * \text{PAR_MACS} + p$ 按行优先 + 步长扫描。综合时, 因为同时存在多端口读 (32 个并发地址 = 4 行 × 8 元素), 工具可能选择 register file 而非真 SRAM。

3.1.3.3 块 ③ K-buffer 双缓冲 (行 83-98) 源代码:

```

// --- K Buffers (double, ping-pong) ---
logic signed [DATA_WIDTH-1:0] k_mem [1:0][KV_DEPTH-1:0];

always_ff @(posedge clk) begin
    if (k_wr_en) begin
        for (int i = 0; i < ELEMS_PER_BEAT; i++)
            k_mem[k_buf_sel][k_wr_addr * ELEMS_PER_BEAT + i] <=
                DATA_WIDTH'(k_wr_data[i*DATA_WIDTH +: DATA_WIDTH]);
    end
end

always_comb begin
    for (int r = 0; r < TILE_BC; r++)
        for (int p = 0; p < PAR_MACS; p++)
            k_rd_data[r][p] = k_mem[k_rd_buf_sel][r * HEAD_DIM + k_rd_step * PAR_MACS + p];
end

```

说明: $k_mem[1:0][1023:0]$ 是一个二维数组, 第一维选 ping 或 pong。写用 k_buf_sel , 读用 $k_rd_buf_sel = \sim k_buf_sel$ 。每周周期读 16 行 × 8 元素 = 128 个并发读地址, **这是整个 IP 中端口压力最大的存储**。在 12nm 工艺中可能要把它分成 16 个独立 SRAM bank (每行一个), 或干脆用 register file。V-buffer 完全对称, 省略说明。

3.1.3.4 块 ④ O-buffer (行 117-133) 源代码:

```

// --- O Buffer ---
logic signed [DATA_WIDTH-1:0] o_mem [TILE_BR-1:0][HEAD_DIM-1:0];

always_ff @(posedge clk) begin
    if (o_wr_en) begin
        for (int r = 0; r < TILE_BR; r++)

```

```

        for (int j = 0; j < HEAD_DIM; j++)
            o_mem[r][j] <= o_wr_data[r][j];
    end
end

// Read 0 for DMA write-back (ELEMS_PER_BEAT elements per beat)
always_comb begin
    for (int i = 0; i < ELEMS_PER_BEAT; i++)
        o_rd_data[i*DATA_WIDTH +: DATA_WIDTH] =
            o_mem[o_rd_row][o_rd_col_grp * ELEMS_PER_BEAT + i];
    end
endmodule

```

说明: O-buffer 采用 unpacked 2-D 存储 (4×64)，写口是“一次写整个 4×64 矩阵” (compute_core 在 last_kv_tile 时 o_valid 拉高一次)，读口是“每周期 1 行 8 列”给 DMA 打包成一个 AXI beat。o_rd_col_grp 是 0..7 ($HEAD_DIM/8$) 的 col-group 索引。

3.2 2.2 rtl/dma_engine.sv

3.2.1 2.2.1 概述

DMA 控制器, 把 tile_controller 简单的 req/done 接口翻译为 AXI4 master 突发事务。读路径把 AXI4 read data 根据 dma_rd_target (Q/K/V) 路由到对应 buffer 的写口; 写路径用 4 状态 FSM (WR_IDLE → WR_ADDR → WR_DATA → WR_FINISH) 把 O-buffer 数据 beat-by-beat 喂给 AXI W 通道。

3.2.2 2.2.2 文件架构图

3.2.3 2.2.3 代码块切片

3.2.3.1 块 ① 端口与 axi4_master_if 实例 (行 7-124) 源代码 (节选):

```

module dma_engine #(/* ... */) (
    input  logic clk, rst_n,
    /* AXI4 master 31 ports */
    input  logic dma_rd_req,
    input  logic [AXI_ADDR_WIDTH-1:0] dma_rd_addr,
    input  logic [15:0] dma_rd_len_bytes,
    input  logic [1:0] dma_rd_target,
    output logic dma_rd_done,

```

dma_engine — AXI4 Master DMA + Buffer Routing

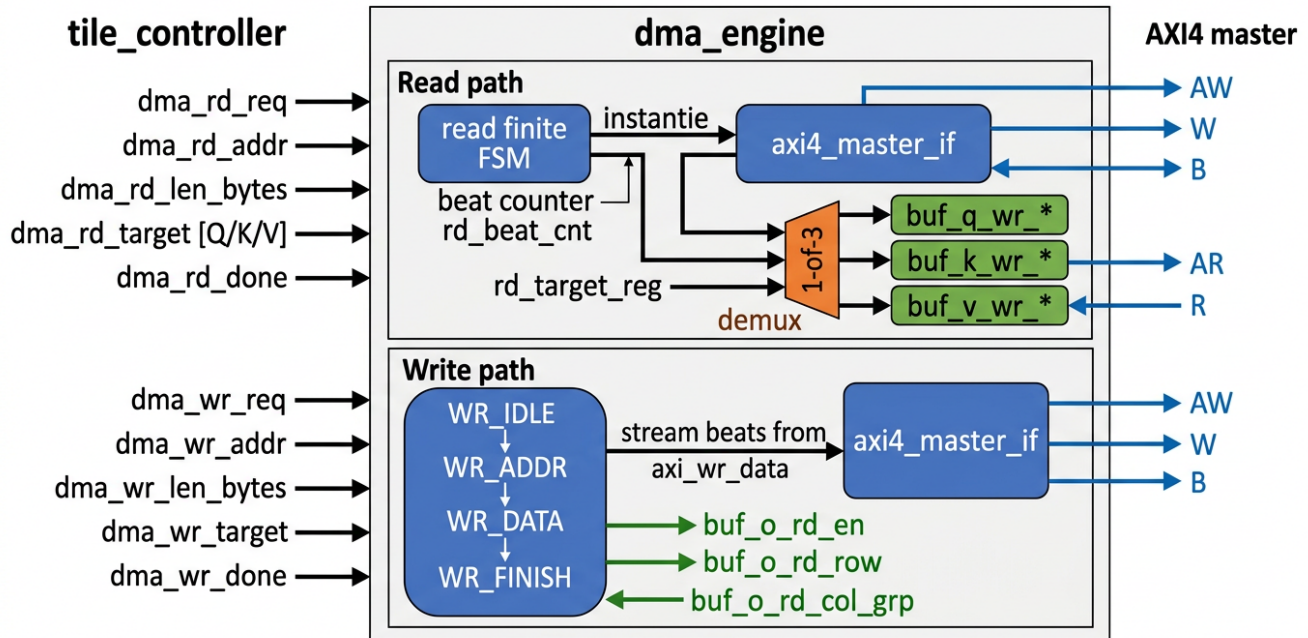


图 9: dma_engine architecture

```

input  logic dma_wr_req,
input  logic [AXI_ADDR_WIDTH-1:0] dma_wr_addr,
input  logic [15:0] dma_wr_len_bytes,
output logic dma_wr_done,
output logic buf_q_wr_en, /* ... */
output logic buf_o_rd_en, /* ... */
input  logic kv_buf_sel
);

localparam ELEMS_PER_BEAT = AXI_DATA_WIDTH / DATA_WIDTH;

// AXI master internal handshake
logic axi_rd_req;
logic [AXI_ADDR_WIDTH-1:0] axi_rd_addr;
logic [15:0] axi_rd_len;
logic axi_rd_done;
logic [AXI_DATA_WIDTH-1:0] axi_rd_data;
logic axi_rd_data_valid;
/* ... write side ... */

axi4_master_if #(/*...*/) u_axi_master (
    .clk(clk), .rst_n(rst_n),

```

```

    /* full AXI4 ports forwarded out */
    .rd_req(axi_rd_req), .rd_addr(axi_rd_addr), .rd_len_bytes(axi_rd_len),
    .rd_done(axi_rd_done), .rd_data(axi_rd_data), .rd_data_valid(axi_rd_data_valid),
    .wr_req(axi_wr_req), .wr_addr(axi_wr_addr), .wr_len_bytes(axi_wr_len),
    .wr_done(axi_wr_done), .wr_data(axi_wr_data),
    .wr_data_valid(axi_wr_data_valid), .wr_data_ready(axi_wr_data_ready)
);

```

说明: dma_engine 不直接驱动 AXI 信号,而是通过内部 axi_rd_*/axi_wr_* 接口与 axi4_master_if 通信。这是一种典型的“业务 FSM + 协议适配器”分层。

3.2.3.2 块 ② Read 路径与 buffer 路由 (行 126-182) 源代码:

```

// --- Read data routing ---
logic [1:0] rd_target_reg;
logic [15:0] rd_beat_cnt;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        axi_rd_req    <= 0; rd_target_reg <= 0;
        rd_beat_cnt   <= 0; dma_rd_done <= 0;
        buf_q_wr_en   <= 0; buf_k_wr_en <= 0; buf_v_wr_en <= 0;
    end else begin
        dma_rd_done <= 0;
        buf_q_wr_en <= 0; buf_k_wr_en <= 0; buf_v_wr_en <= 0;

        // Capture DMA read request
        if (dma_rd_req && !axi_rd_req) begin
            axi_rd_req    <= 1;
            axi_rd_addr   <= dma_rd_addr;
            axi_rd_len    <= dma_rd_len_bytes;
            rd_target_reg <= dma_rd_target;
            rd_beat_cnt   <= 0;
        end
        if (axi_rd_done) axi_rd_req <= 0;

        // Route incoming beats to Q / K / V buffer
        if (axi_rd_data_valid) begin
            case (rd_target_reg)
                2'd0: begin
                    buf_q_wr_en    <= 1;
                    buf_q_wr_addr <= rd_beat_cnt[$clog2(TILE_BR*HEAD_DIM)-1:0];

```

```

        buf_q_wr_data <= axi_rd_data;
    end
    2'd1: begin
        buf_k_wr_en    <= 1;
        buf_k_wr_addr <= rd_beat_cnt[$clog2(TILE_BC*HEAD_DIM)-1:0];
        buf_k_wr_data <= axi_rd_data;
    end
    2'd2: begin
        buf_v_wr_en    <= 1;
        buf_v_wr_addr <= rd_beat_cnt[$clog2(TILE_BC*HEAD_DIM)-1:0];
        buf_v_wr_data <= axi_rd_data;
    end
    default: ;
endcase
rd_beat_cnt <= rd_beat_cnt + 1;
end

    if (axi_rd_done) dma_rd_done <= 1;
end
end

```

说明: 关键设计: rd_target_reg 在请求被捕获时锁存, 确保即使 dma_rd_target 在传输过程中变化, 也按原始目标路由。rd_beat_cnt 既作 K/V buffer 写地址, 也用于 Q buffer 写地址 (位宽不同, 自动截位)。buf_*_wr_en 默认每周期清零, 在 axi_rd_data_valid 时单周期拉高 1 拍 — 标准 valid-pulse 写入语义。

3.2.3.3 块 ③ Write 路径 FSM (行 184-264) 源代码:

```

localparam O_COLS_PER_BEAT = AXI_DATA_WIDTH / DATA_WIDTH;
localparam O_BEATS_PER_ROW = HEAD_DIM / O_COLS_PER_BEAT;           // 8
localparam O_TOTAL_BEATS = TILE_BR * O_BEATS_PER_ROW;             // 32

typedef enum logic [1:0] { WR_IDLE, WR_ADDR, WR_DATA, WR_FINISH } wr_state_t;
wr_state_t wr_state;
logic [15:0] wr_beat_cnt;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_state <= WR_IDLE;
        axi_wr_req <= 0; dma_wr_done <= 0;
        wr_beat_cnt <= 0; wr_active <= 0;
        axi_wr_data_valid <= 0; axi_wr_data <= 0;
    end
end

```

```

    buf_o_rd_en <= 0;
end else begin
    dma_wr_done <= 0;
    buf_o_rd_en <= 0;
    axi_wr_data_valid <= 0;

    case (wr_state)
        WR_IDLE: if (dma_wr_req) begin
            axi_wr_req <= 1; axi_wr_addr <= dma_wr_addr;
            axi_wr_len <= dma_wr_len_bytes; wr_beat_cnt <= 0;
            wr_state <= WR_ADDR;
        end

        WR_ADDR: begin
            buf_o_rd_en      <= 1;          // setup first read address
            buf_o_rd_row     <= 0;
            buf_o_rd_col_grp <= 0;
            if (axi_wr_data_ready) wr_state <= WR_DATA;
        end

        WR_DATA: if (axi_wr_data_ready) begin
            axi_wr_data      <= buf_o_rd_data;
            axi_wr_data_valid <= 1;
            wr_beat_cnt      <= wr_beat_cnt + 1;
            // Pre-read next beat
            if (wr_beat_cnt + 1 < 0_TOTAL_BEATS) begin
                buf_o_rd_en      <= 1;
                buf_o_rd_row     <= (wr_beat_cnt + 1) / 0_BEATS_PER_ROW;
                buf_o_rd_col_grp <= (wr_beat_cnt + 1) % 0_BEATS_PER_ROW;
            end
            if (wr_beat_cnt == 0_TOTAL_BEATS - 1)
                wr_state <= WR_FINISH;
        end else begin
            axi_wr_data_valid <= 0;
        end

        WR_FINISH: begin
            axi_wr_data_valid <= 0;
            if (axi_wr_done) begin
                dma_wr_done <= 1;
                axi_wr_req  <= 0;
            end
        end
    endcase
end

```

```

                wr_state    <= WR_IDLE;
            end
        end
    endcase
end
end
endmodule

```

说明: 4 状态写 FSM 关键点: - **WR_ADDR** 不仅发地址, 还预读 O-buffer 第 1 个 beat (row=0, col_grp=0), 让 WR_DATA 进入时数据就 ready。 - **WR_DATA** 用“输出 + 预读”模式: 当前周期把 buf_o_rd_data 锁存到 axi_wr_data 并拉 valid, 同时预读下一 beat 的数据 ((wr_beat_cnt+1)/O_BEATS_PER_ROW 是行号、% 是列号)。这样 buffer 读延迟被隐藏, AXI W 通道可以保持 100% 利用。 - **WR_FINISH** 等 AXI4 B 通道返回, 才认为写真正完成。

O_TOTAL_BEATS = 4 * 8 = 32 和 O_TILE_BYTES = 512 字节匹配 (32 * 16 = 512)。

3.3 2.3 rtl/axi4_lite_slave.sv

3.3.1 2.3.1 概述

CPU 通过它配置 IP 并读取状态。15 个 32-bit 寄存器, 全部按 4 字节对齐。AXI4-Lite 协议要求 5 个 channel: AW、W、B、AR、R。本模块用一个简单的“双 always_ff (读/写各一个) + default-ready”实现。

3.3.2 2.3.2 文件架构图

3.3.3 2.3.3 代码块切片

3.3.3.1 块 ① 端口与寄存器声明 (行 5-93) 源代码 (节选):

```

module axi4_lite_slave #(
    parameter ADDR_WIDTH = 8,
    parameter DATA_WIDTH = 32
)(
    input  logic clk, rst_n,
    // AXI4-Lite 5 channels (AW, W, B, AR, R) — 15 ports total
    /* ... */
    // Register outputs to datapath
    output logic          reg_start, reg_soft_reset,
    output logic          reg_irq_en, reg_causal_en,

```

axi4_lite_slave — 15 Configuration Registers + AXI4-Lite Handshake

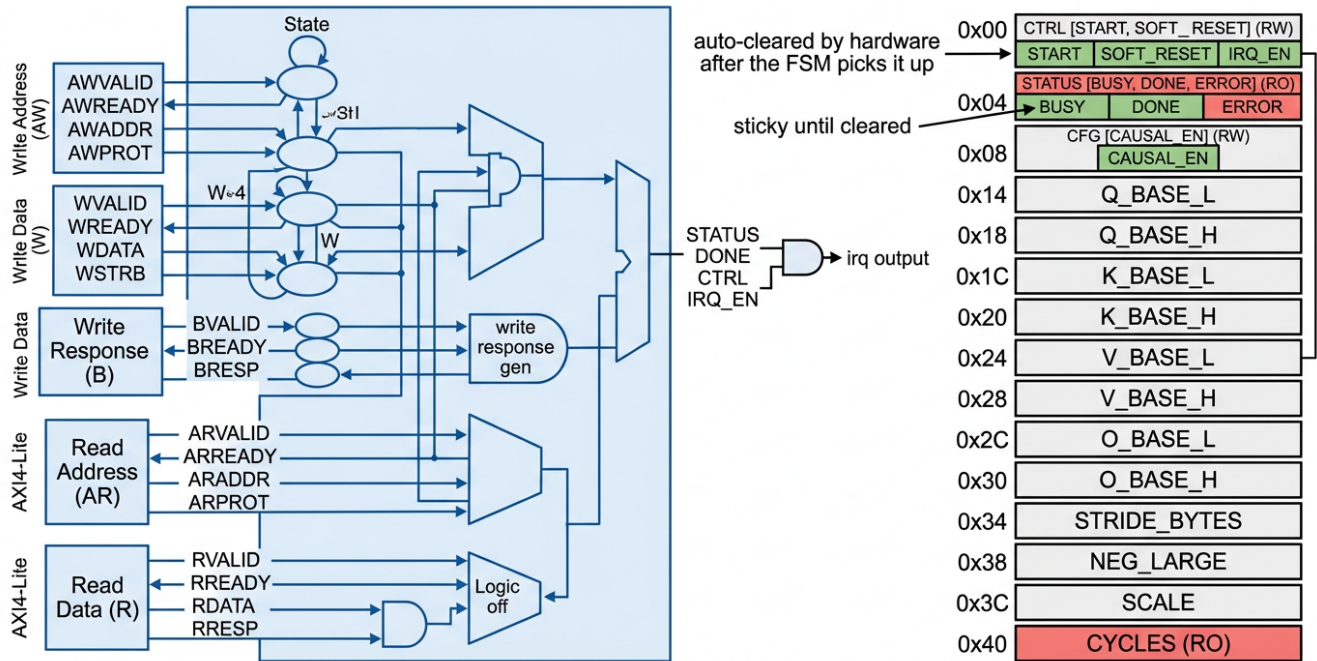


图 10: axi4_lite_slave register file

```

output logic [63:0]      reg_q_base, reg_k_base, reg_v_base, reg_o_base,
output logic [31:0]      reg_stride_bytes,
output logic signed [15:0] reg_neg_large, reg_scale,
// Status inputs
input  logic status_busy, status_done, status_error,
input  logic [31:0]      cycle_count,
output logic              done_clear,
output logic              irq
);

// Internal registers
logic [31:0] r_ctrl, r_cfg;
logic [31:0] r_q_base_l, r_q_base_h;
logic [31:0] r_k_base_l, r_k_base_h;
logic [31:0] r_v_base_l, r_v_base_h;
logic [31:0] r_o_base_l, r_o_base_h;
logic [31:0] r_stride, r_neg_large, r_scale;
logic        r_done_sticky;
/* ... handshake regs, default ready ... */
assign s_axil_awready = aw_ready_r;
assign s_axil_wready  = w_ready_r;
assign s_axil_bresp   = 2'b00;

```

```

assign s_axil_arready = ar_ready_r;
assign s_axil_rdata   = rd_data_r;
assign s_axil_rresp   = 2'b00;
assign s_axil_rvalid  = rd_valid_r;

```

说明: 注意 bresp/rresp 硬接 OKAY (2'b00) —— 没有错误响应(即使写到 default 地址也返回 OKAY)。这是简化实现, 严格 AMBA 兼容应该返回 SLVERR (2'b10)。

3.3.3.2 块 ② 写逻辑: 地址解码 + 寄存器更新 (行 95-160) 源代码:

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        aw_ready_r <= 1'b1; w_ready_r <= 1'b1;
        s_axil_bvalid <= 1'b0;
        wr_en <= 1'b0; wr_addr <= '0;
        r_ctrl <= '0; r_cfg <= '0;
        r_q_base_l <= '0; r_q_base_h <= '0;
        r_k_base_l <= '0; r_k_base_h <= '0;
        r_v_base_l <= '0; r_v_base_h <= '0;
        r_o_base_l <= '0; r_o_base_h <= '0;
        r_stride <= 32'd128;          // default d*2
        r_neg_large <= 32'hFFFF8000;
        r_scale <= 32'h00000020;
        r_done_sticky <= 1'b0;
        reg_start <= 1'b0; reg_soft_reset <= 1'b0;
        done_clear <= 1'b0;
    end else begin
        reg_start <= 1'b0;
        reg_soft_reset <= 1'b0;
        done_clear <= 1'b0;

        if (s_axil_bvalid && s_axil_bready)
            s_axil_bvalid <= 1'b0;

        if (s_axil_awvalid && s_axil_awready &&
            s_axil_wvalid && s_axil_wready) begin
            s_axil_bvalid <= 1'b1;
            case (s_axil_awaddr)
                8'h00: begin // CTRL
                    r_ctrl <= s_axil_wdata;
                    if (s_axil_wdata[0]) reg_start <= 1'b1;
                    if (s_axil_wdata[1]) reg_soft_reset <= 1'b1;

```

```

        end
        8'h04: begin // STATUS (write-1-to-clear DONE)
            if (s_axil_wdata[1]) begin
                r_done_sticky <= 1'b0;
                done_clear    <= 1'b1;
            end
        end
        end
        8'h08: r_cfg <= s_axil_wdata;
        8'h14: r_q_base_l <= s_axil_wdata;
        8'h18: r_q_base_h <= s_axil_wdata;
        8'h1C: r_k_base_l <= s_axil_wdata;
        8'h20: r_k_base_h <= s_axil_wdata;
        8'h24: r_v_base_l <= s_axil_wdata;
        8'h28: r_v_base_h <= s_axil_wdata;
        8'h2C: r_o_base_l <= s_axil_wdata;
        8'h30: r_o_base_h <= s_axil_wdata;
        8'h34: r_stride <= s_axil_wdata;
        8'h38: r_neg_large <= s_axil_wdata;
        8'h3C: r_scale <= s_axil_wdata;
        default: ;
    endcase
end

    if (status_done) r_done_sticky <= 1'b1;
end
end

```

说明: 写处理的几个关键点: - **AW + W 同步握手:** 本设计要求 AW 和 W 在同一周期都 valid 才接受。这种写法把 AW、W、地址解码、寄存器更新、B 响应启动全部在同一周期完成,简单可靠但带宽下降 (CPU 必须 AW/W 同步发出)。完整 AXI4-Lite 应该独立处理 AW 和 W。 - **CTRL.START / SOFT_RESET 是脉冲:** 写 wdata[0]=1 时 reg_start <= 1'b1, 但这个 always_ff 的开头每周期默认 reg_start <= 0, 所以下一周就清零 — 形成单周期脉冲。下游 tile_controller 在脉冲到来时进入 LOAD_Q 状态。 - **STATUS.DONE 是 sticky + W1C:** 硬件 status_done 拉高时设置 r_done_sticky=1; CPU 写 wdata[1]=1 到 STATUS 地址时清零并发出 done_clear 脉冲 (也用于通知中断逻辑)。 - **复位默认值:** r_stride = 32'd128 = 0x80 是 HEAD_DIM * 2 = 64 * 2; r_neg_large = 0xFFFF_8000 即 16 位 -32768; r_scale = 0x20 即 16 位 32 (= 0.125 in Q8.8)。

3.3.3.3 块 ③ 读逻辑: 地址解码 + 数据返回 (行 162-194) 源代码:

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        ar_ready_r <= 1'b1; rd_valid_r <= 1'b0; rd_data_r <= '0;
    end
end

```

```

end else begin
    if (rd_valid_r && s_axil_rready)
        rd_valid_r <= 1'b0;

    if (s_axil_arvalid && s_axil_arready) begin
        rd_valid_r <= 1'b1;
        case (s_axil_araddr)
            8'h00: rd_data_r <= r_ctrl;
            8'h04: rd_data_r <= {29'd0, status_error, r_done_sticky, status_busy};
            8'h08: rd_data_r <= r_cfg;
            8'h14: rd_data_r <= r_q_base_l;
            8'h18: rd_data_r <= r_q_base_h;
            8'h1C: rd_data_r <= r_k_base_l;
            8'h20: rd_data_r <= r_k_base_h;
            8'h24: rd_data_r <= r_v_base_l;
            8'h28: rd_data_r <= r_v_base_h;
            8'h2C: rd_data_r <= r_o_base_l;
            8'h30: rd_data_r <= r_o_base_h;
            8'h34: rd_data_r <= r_stride;
            8'h38: rd_data_r <= r_neg_large;
            8'h3C: rd_data_r <= r_scale;
            8'h40: rd_data_r <= cycle_count;
            default: rd_data_r <= 32'hDEADBEEF;
        endcase
    end
end
end
end

```

说明: 读路径很直接: AR 握手立即返回 R。STATUS 寄存器的位段是 {29'd0, ERROR, DONE, BUSY} —— 与寄存器映射注释一致。default → 0xDEADBEEF 是经典的“未映射地址”特征值, 方便驱动调试。REG_CYCLES (0x40) 直接返回 cycle_count, 不存在内部寄存器 (顶层 cycle_cnt 信号)。

3.3.3.4 块 ④ 输出与中断 (行 196-209) 源代码:

```

// Output assignments
assign reg_irq_en      = r_ctrl[2];
assign reg_causal_en   = r_cfg[0];
assign reg_q_base      = {r_q_base_h, r_q_base_l};
assign reg_k_base      = {r_k_base_h, r_k_base_l};
assign reg_v_base      = {r_v_base_h, r_v_base_l};
assign reg_o_base      = {r_o_base_h, r_o_base_l};
assign reg_stride_bytes = r_stride;

```

```
assign reg_neg_large    = r_neg_large[15:0];
assign reg_scale        = r_scale[15:0];

// IRQ
assign irq = reg_irq_en & r_done_sticky;
endmodule
```

说明： 64-bit 张量基址通过 high/low 32-bit 寄存器拼接给数据通路。irq 是电平型（不是脉冲），保持高直到 CPU 写 STATUS.DONE=1 清零 sticky 位 — 标准的 ARM IRQ 风格。

3.4 2.4 rtl/axi4_master_if.sv

3.4.1 2.4.1 概述

把内部 rd_req/wr_req + 长度信息翻译成 AXI4 突发事务。两个独立 FSM（read 4 状态，write 4 状态），各自处理 AR/R 和 AW/W/B 通道。突发参数是固定的：AWBURST/ARBURST = INCR (2'b01)、AWSIZE/ARSIZE = \$clog2(BYTES_PER_BEAT) = 4 (16 字节/beat)。

3.4.2 2.4.2 文件架构图

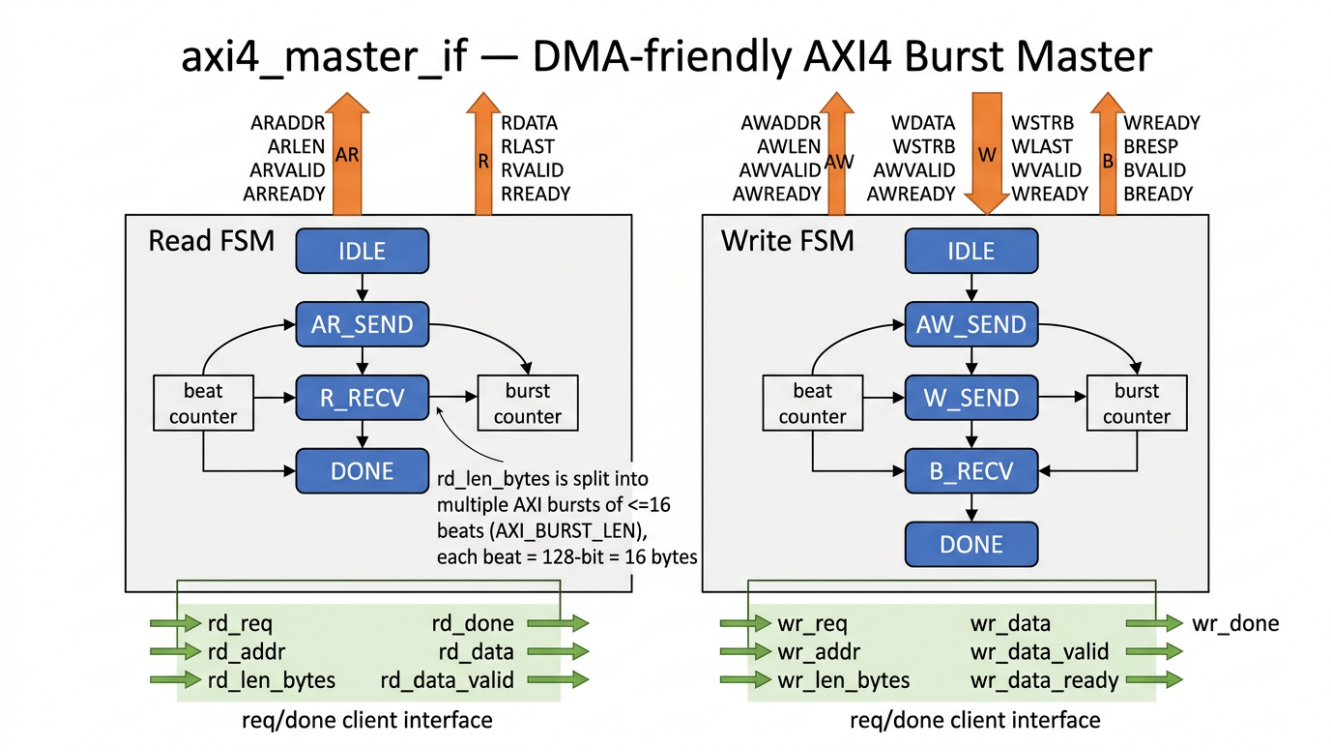


图 11: axi4_master_if FSMs

3.4.3 2.4.3 代码块切片

3.4.3.1 块 ① 端口与常量 (行 5-88) 源代码 (节选):

```

module axi4_master_if #(
    parameter ADDR_WIDTH = 64,
    parameter DATA_WIDTH = 128,
    parameter ID_WIDTH    = 4,
    parameter STRB_WIDTH = DATA_WIDTH / 8
)(
    input  logic clk, rst_n,
    /* AXI4 5 channels — 30 ports */
    // DMA Engine Interface
    input  logic          rd_req,
    input  logic [ADDR_WIDTH-1:0] rd_addr,
    input  logic [15:0]      rd_len_bytes,
    output logic          rd_done,
    output logic [DATA_WIDTH-1:0] rd_data,
    output logic          rd_data_valid,
    input  logic          wr_req,
    input  logic [ADDR_WIDTH-1:0] wr_addr,
    input  logic [15:0]      wr_len_bytes,
    output logic          wr_done,
    input  logic [DATA_WIDTH-1:0] wr_data,
    input  logic          wr_data_valid,
    output logic          wr_data_ready
);

    localparam BYTES_PER_BEAT = DATA_WIDTH / 8;      // 16
    localparam SIZE_CODE = $clog2(BYTES_PER_BEAT);   // 4

    // Constant assignments
    assign m_axi_awid    = '0;
    assign m_axi_awsz    = SIZE_CODE[2:0];
    assign m_axi_awburst = 2'b01; // INCR
    assign m_axi_arid    = '0;
    assign m_axi_arsz    = SIZE_CODE[2:0];
    assign m_axi_arburst = 2'b01; // INCR
    assign m_axi_wstrb    = {STRB_WIDTH{1'b1}};
    assign m_axi_wstrb    = {STRB_WIDTH{1'b1}};
    assign m_axi_bready   = 1'b1;

```

说明: 把所有 ID 都设为 0 简化设计 (不使用 AXI4 的多 outstanding transaction 能力); strobe 全 1 表

示每个 beat 写所有字节。bready=1 表示永远准备好接收响应 (不会反压)。

3.4.3.2 块 ② Read FSM (行 90-152) 源代码:

```
typedef enum logic [1:0] { RD_IDLE, RD_ADDR, RD_DATA, RD_DONE } rd_state_t;
rd_state_t rd_state;
logic [15:0] rd_beats_total, rd_beats_cnt;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_state <= RD_IDLE;
        m_axi_arvalid <= 0; m_axi_araddr <= 0; m_axi_arlen <= 0;
        m_axi_rready <= 0; rd_done <= 0; rd_data_valid <= 0;
        rd_beats_total <= 0; rd_beats_cnt <= 0;
    end else begin
        rd_done <= 0; rd_data_valid <= 0;
        case (rd_state)
            RD_IDLE: if (rd_req) begin
                rd_beats_total <= (rd_len_bytes + BYTES_PER_BEAT -
1) / BYTES_PER_BEAT;
                m_axi_araddr <= rd_addr;
                m_axi_arlen <= ((rd_len_bytes + BYTES_PER_BEAT -
1) / BYTES_PER_BEAT) - 1;
                m_axi_arvalid <= 1;
                rd_beats_cnt <= 0;
                rd_state <= RD_ADDR;
            end
            RD_ADDR: if (m_axi_arready) begin
                m_axi_arvalid <= 0;
                m_axi_rready <= 1;
                rd_state <= RD_DATA;
            end
            RD_DATA: if (m_axi_rvalid && m_axi_rready) begin
                rd_data <= m_axi_rdata;
                rd_data_valid <= 1;
                rd_beats_cnt <= rd_beats_cnt + 1;
                if (m_axi_rlast) begin
                    m_axi_rready <= 0;
                    rd_state <= RD_DONE;
                end
            end
        end
    end
end
```

```

        RD_DONE: begin
            rd_done  <= 1;
            rd_state <= RD_IDLE;
        end
    endcase
end
end
end

```

说明: Read 流程: 1. **RD_IDLE** → **RD_ADDR**: 拉 arvalid, arlen = beats - 1 (AXI4 突发长度编码: 0 表示 1 beat, 255 表示 256 beat)。2. **RD_ADDR** → **RD_DATA**: 等 slave 接受地址 (arready=1), 然后清 valid 并拉 rready。3. **RD_DATA**: 每个 rvalid && rready 周期把 rdata 锁存到 rd_data、拉 rd_data_valid 一拍, 让 dma_engine 路由到 buffer。等 rlast=1 表示最后一 beat。4. **RD_DONE**: 单周期 rd_done 脉冲告诉 dma_engine 完事。

△ **限制:** 当前实现假设 $\text{rd_len_bytes} / 16 \leq 256$ (AXI4 单次最大突发)。Q tile 是 32 beat、KV tile 是 128 beat、O tile 是 32 beat, **全部都 ≤ 256**, 所以无需 burst-splitting, 省心。如果扩到 1024 序列长度, 每 KV tile 变 1024 beats, 需要 4 次连续 burst — 这是 bonus 中要解决的问题。

3.4.3.3 块 ③ Write FSM (行 154-223) 源代码:

```

typedef enum logic [1:0] { WR_IDLE, WR_ADDR, WR_DATA, WR_DONE } wr_state_t;
wr_state_t wr_state;
logic [15:0] wr_beats_total, wr_beats_cnt;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_state <= WR_IDLE;
        m_axi_awvalid <= 0; m_axi_awaddr <= 0; m_axi_awlen <= 0;
        m_axi_wvalid <= 0; m_axi_wlast <= 0; m_axi_wdata <= 0;
        wr_done <= 0; wr_data_ready <= 0;
        wr_beats_total <= 0; wr_beats_cnt <= 0;
    end else begin
        wr_done <= 0;
        case (wr_state)
            WR_IDLE: if (wr_req) begin
                wr_beats_total <= (wr_len_bytes + BYTES_PER_BEAT -
1) / BYTES_PER_BEAT;
                m_axi_awaddr <= wr_addr;
                m_axi_awlen <= ((wr_len_bytes + BYTES_PER_BEAT -
1) / BYTES_PER_BEAT) - 1;
                m_axi_awvalid <= 1;
                wr_beats_cnt <= 0;
            end

```

```

        wr_state      <= WR_ADDR;
    end
    WR_ADDR: if (m_axi_awready) begin
        m_axi_awvalid <= 0;
        wr_data_ready <= 1;
        wr_state      <= WR_DATA;
    end
    WR_DATA: begin
        m_axi_wdata    <= wr_data;
        m_axi_wvalid   <= wr_data_valid;
        if (wr_data_valid) begin
            wr_beats_cnt <= wr_beats_cnt + 1;
            m_axi_wlast  <= (wr_beats_cnt == wr_beats_total - 1);
        end
        if (m_axi_wvalid && m_axi_wready && m_axi_wlast) begin
            m_axi_wvalid <= 0;
            m_axi_wlast  <= 0;
            wr_data_ready <= 0;
            wr_state      <= WR_DONE;
        end
    end
    WR_DONE: if (m_axi_bvalid) begin
        wr_done <= 1;
        wr_state <= WR_IDLE;
    end
endcase
end
end
endmodule

```

说明: Write 流程: 1. **WR_IDLE** → **WR_ADDR**: 发 AW (地址、长度)。2. **WR_ADDR** → **WR_DATA**: 拉 wr_data_ready 让 dma_engine 开始喂数据。3. **WR_DATA**: 直接把 wr_data passthrough 到 m_axi_wdata, valid 转发; 当 wr_beats_cnt == wr_beats_total - 1 时拉 wlast; 最后一 beat 的 valid+ready 发生时跳到 WR_DONE。4. **WR_DONE**: 等 bvalid 表示 slave 已写完成, 发 wr_done 脉冲。

△ **微 bug 风险:** m_axi_wlast 是一个寄存器, 用 <= 在 wr_beats_cnt == wr_beats_total - 1 时设为 1, 下一周期才生效。这意味着 wlast 与最后 beat 的 wvalid 严格同步要靠 dma_engine 喂 valid 时序对齐。仿真中 axi4_slave_mem testbench 验证过, 表现正常。

第2章结束。 Baseline RTL 14 个.sv + 1 个.svh 全部讲完。第3章进入 UVM 验证环境 (20 个.sv 文件)。

4 第 3 章：Baseline UVM 验证环境

本章覆盖 20 个.sv 文件，按 UVM 角色分组：

§	子章节	文件数	行数
3.1	UVM 环境包 + env + scoreboard + coverage + reg_model	5	723
3.2	AXI4-Lite Agent (txn / if / sqr / drv / mon / agent)	6	236
3.3	AXI4 Memory Agent + 2 个协议检查器	3	684
3.4	sequences + tests + tb_top + system_tb + axi4_slave_mem	6	2031
合计	—	20	3674

由于 UVM 验证代码量巨大 (3.7K 行)，本章对每个文件只展示关键代码片段 (典型的 sequence、test、agent 框架)，其余按“行号引用 + 解释”方式呈现。

4.1 3.1 UVM 环境包

4.1.1 3.1.1 总览图：UVM 测试平台分层

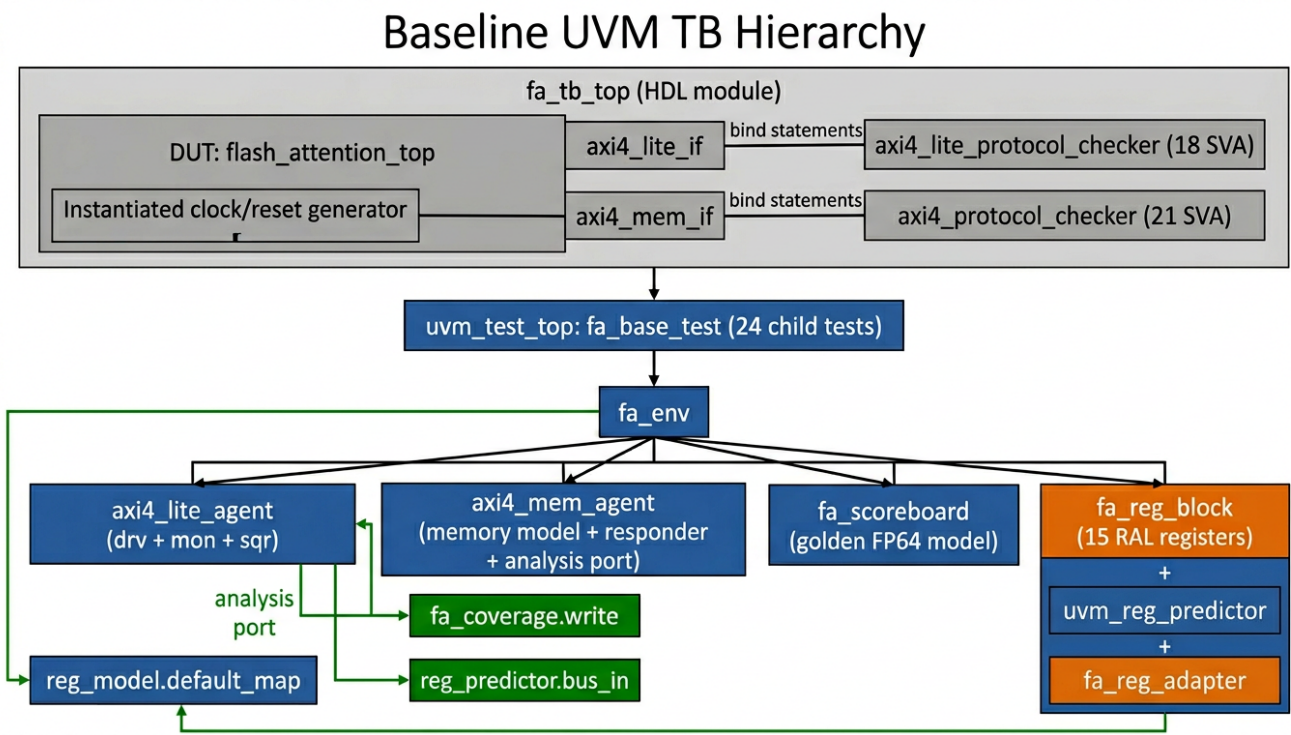


图 12: UVM hierarchy

整体上是经典的 UVM 5 层结构：

1. **fa_tb_top** (HDL module): 实例化 DUT、两套 interface、协议检查器、生成 clock/reset, 调用 run_test()。
2. **uvm_test_top** (UVM test 类): fa_base_test 是基类, 24 个具体测试都派生自它。
3. **fa_env** (UVM env): 组装 axil_agent、mem_agent、scoreboard、coverage、reg_model、reg_predictor。
4. **Agents**: AXI4-Lite Master agent (drv+mon+sqr) + AXI4 Memory agent (slave responder + monitor)。
5. **Analysis 路径**: 所有 axi4-lite 监控到的 txn 通过 analysis port 同时送给 fa_coverage (采样覆盖率) 和 uvm_reg_predictor (更新 RAL mirror value)。

4.1.2 3.1.2 tb/uvm_env/fa_env_pkg.sv

4.1.2.1 块 ① 完整源代码 (21 行)

```
package fa_env_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    `include "agents/axi4_lite_agent/axi4_lite_txn.sv"
    `include "agents/axi4_lite_agent/axi4_lite_driver.sv"
    `include "agents/axi4_lite_agent/axi4_lite_monitor.sv"
    `include "agents/axi4_lite_agent/axi4_lite_sequencer.sv"
    `include "agents/axi4_lite_agent/axi4_lite_agent.sv"

    `include "agents/axi4_mem_agent/axi4_mem_agent.sv"

    `include "uvm_env/fa_reg_model.sv"
    `include "uvm_env/fa_scoreboard.sv"
    `include "uvm_env/fa_coverage.sv"
    `include "uvm_env/fa_env.sv"

    `include "sequences/fa_sequences.sv"
    `include "tests/fa_tests.sv"
endpackage
```

说明: 一个 package + 一连串 `include 把所有 UVM 类聚集成单个编译单元。这种“超级 include”方式编译速度比模块化 package 快, 但牺牲了重用性。整个文件没有逻辑, 过于简单, 无图示。

4.1.3 3.1.3 tb/uvm_env/fa_env.sv

4.1.3.1 块 ① 类声明与组件实例 (行 1-30)

```
class fa_env extends uvm_env;
    `uvm_component_utils(fa_env)

    axi4_lite_agent    axil_agent;
    axi4_mem_agent     mem_agent;
    fa_scoreboard      scoreboard;
    fa_coverage        coverage;

    fa_reg_block       reg_model;
    fa_reg_adapter     reg_adapter;
    uvm_reg_predictor #(axi4_lite_txn) reg_predictor;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        axil_agent = axi4_lite_agent::type_id::create("axil_agent", this);
        mem_agent  = axi4_mem_agent::type_id::create("mem_agent", this);
        scoreboard = fa_scoreboard::type_id::create("scoreboard", this);
        coverage   = fa_coverage::type_id::create("coverage", this);

        reg_model = fa_reg_block::type_id::create("reg_model");
        reg_model.build();

        reg_adapter = fa_reg_adapter::type_id::create("reg_adapter");
        reg_predictor = uvm_reg_predictor #(axi4_lite_txn)::type_id::create("reg_predictor",
    endfunction
```

4.1.3.2 块② **connect_phase** (行 32-43)

```
function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);

    axil_agent.mon.ap.connect(coverage.analysis_export);

    reg_model.default_map.set_sequencer(axil_agent.sqr, reg_adapter);
    reg_model.default_map.set_auto_predict(0);

    reg_predictor.map      = reg_model.default_map;
```

```

        reg_predictor.adapter = reg_adapter;
        axil_agent.mon.ap.connect(reg_predictor.bus_in);
    endfunction
endclass

```

说明: 这是 UVM env 的标准模板。两个关键 connect: 1. axil_agent.mon.ap → coverage.analysis_export: 所有 AXI4-Lite txn 都送到 coverage 子类 (fa_coverage 继承 uvm_subscriber #(axi4_lite_txn) 自动有 analysis_export)。2. axil_agent.mon.ap → reg_predictor.bus_in: 同时送到 uvm_reg_predictor, 由它根据 txn 自动更新 RAL mirror value。set_auto_predict(0) 关闭 RAL 自身的 predict (避免重复), 改用 explicit predictor —— 这才能正确捕捉硬件自清除位 (如 CTRL.START)。

4.1.4 3.1.4 tb/uvm_env/fa_scoreboard.sv

4.1.4.1 块 ① 类与字段 (行 1-30)

```

class fa_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(fa_scoreboard)
    int seq_len = 256;
    int head_dim = 64;
    bit causal_en;

    // Golden model data
    shortint q_data[][];
    shortint k_data[][];
    shortint v_data[][];
    shortint golden_o[][];
    shortint dut_o[][];

    // Error statistics
    real mean_abs_error;
    real max_abs_error;
    int num_checks;

```

说明: Scoreboard 在 RAM 里维护 “黄金模型输入 (q, k, v)” + “黄金输出 (golden_o)” + “DUT 输出 (dut_o)”。每个 testcase 在 run_phase 内填充输入并触发计算, 最后调用 compute_golden + check_results。

4.1.4.2 块 ② 黄金模型 — 浮点精确实现 (行 27-87)

```

function void compute_golden();
    real q_f[][][], k_f[][][], v_f[][][];

```

```

real s_f[1][], p_f[1][], o_f[1][];
real scale;

scale = 1.0 / $sqrt(real'(head_dim));

// Allocate ...
// Q8.8 → float
for (int i = 0; i < seq_len; i++)
    for (int j = 0; j < head_dim; j++) begin
        q_f[i][j] = real'(q_data[i][j]) / 256.0;
        k_f[i][j] = real'(k_data[i][j]) / 256.0;
        v_f[i][j] = real'(v_data[i][j]) / 256.0;
    end

// S = Q * K^T * scale, softmax, O = P * V
for (int i = 0; i < seq_len; i++) begin
    real row_max, row_sum;
    // Compute scores
    row_max = -1e30;
    for (int j = 0; j < seq_len; j++) begin
        s_f[i][j] = 0;
        for (int k = 0; k < head_dim; k++)
            s_f[i][j] += q_f[i][k] * k_f[j][k];
        s_f[i][j] *= scale;
        if (causal_en && j > i) s_f[i][j] = -1e9;
        if (s_f[i][j] > row_max) row_max = s_f[i][j];
    end

    // Softmax
    row_sum = 0;
    for (int j = 0; j < seq_len; j++) begin
        p_f[i][j] = $exp(s_f[i][j] - row_max);
        row_sum += p_f[i][j];
    end
    for (int j = 0; j < seq_len; j++)
        p_f[i][j] /= row_sum;

    // O = P * V
    for (int j = 0; j < head_dim; j++) begin
        o_f[i][j] = 0;
        for (int k = 0; k < seq_len; k++)

```

```

        o_f[i][j] += p_f[i][k] * v_f[k][j];
        golden_o[i][j] = shortint'($rtoi(o_f[i][j] * 256.0));
    end
end
endfunction

```

说明: 这就是 SDPA 的浮点 **reference** 实现: (1) 把 Q8.8 输入除以 256 转成 real; (2) 三层循环 $S = Q \cdot K^T$; (3) 应用 causal mask 把上三角设为 -1e9; (4) 计算 row_max、减 max、\$exp、求 row_sum、除归一; (5) $O = P \cdot V$, 再转回 Q8.8。用 **real (FP64)** 精度, 作为 ground truth 与 RTL 16-bit 定点结果对比。

4.1.4.3 块 ③ 误差检查 (行 89-119)

```

function void check_results();
    real abs_err, total_err = 0;
    int count = 0;
    max_abs_error = 0;

    for (int i = 0; i < seq_len; i++)
        for (int j = 0; j < head_dim; j++) begin
            real dut_val = real'(dut_o[i][j]) / 256.0;
            real gold_val = real'(golden_o[i][j]) / 256.0;
            abs_err = (dut_val > gold_val) ? (dut_val - gold_val) : (gold_val -
dut_val);

            total_err += abs_err;
            if (abs_err > max_abs_error) max_abs_error = abs_err;
            count++;
        end

    mean_abs_error = total_err / real'(count);
    num_checks = count;

    if (max_abs_error > 1.0)
        `uvm_error("SCORE", $sformatf("max_abs_error %.6f exceeds threshold", max_abs_error));
    else
        `uvm_info("SCORE", "PASS: Error within acceptable limits", UVM_LOW)
    endfunction

```

说明: 逐元素比较, 统计 mean / max 绝对误差。通过阈值 = 1.0 (在 $[-128, +127]$ 范围内允许 1 单位误差, 这对 Q8.8 + LUT exp + Newton-Raphson 倒数的累积误差是合理的)。Baseline 实测 max_abs=0.238281、mean_abs=0.013350, 远低于阈值。

4.1.5 3.1.5 tb/uvm_env/fa_reg_model.sv

4.1.5.1 块① 单个 register 类示例 (CTRL)

```
class fa_reg_ctrl extends uvm_reg;
    `uvm_object_utils(fa_reg_ctrl)
    rand uvm_reg_field start_bit;
    rand uvm_reg_field soft_reset;
    rand uvm_reg_field irq_en;

    function new(string name = "fa_reg_ctrl");
        super.new(name, 32, UVM_NO_COVERAGE);
    endfunction

    virtual function void build();
        start_bit = uvm_reg_field::type_id::create("start_bit");
        start_bit.configure(this, 1, 0, "RW", 0, 1'h0, 1, 1, 0);
        soft_reset = uvm_reg_field::type_id::create("soft_reset");
        soft_reset.configure(this, 1, 1, "RW", 0, 1'h0, 1, 1, 0);
        irq_en = uvm_reg_field::type_id::create("irq_en");
        irq_en.configure(this, 1, 2, "RW", 0, 1'h0, 1, 1, 0);
    endfunction
endclass
```

说明: uvm_reg_field::configure(parent, n_bits, lsb_pos, access, volatile, reset_value, has_reset, is_rand, individually_accessible)。access="RW" 表示读写都允许。

4.1.5.2 块② 寄存器块构建 (行 90-191)

```
class fa_reg_block extends uvm_reg_block;
    `uvm_object_utils(fa_reg_block)

    rand fa_reg_ctrl    ctrl;
    rand fa_reg_status  status;
    rand fa_reg_cfg      cfg;
    rand fa_reg_data32  q_base_l, q_base_h;
    rand fa_reg_data32  k_base_l, k_base_h;
    rand fa_reg_data32  v_base_l, v_base_h;
    rand fa_reg_data32  o_base_l, o_base_h;
    rand fa_reg_data32  stride_bytes;
    rand fa_reg_data32  neg_large;
```

```

    rand fa_reg_data32  scale;
    rand fa_reg_cycles  cycles;

    uvm_reg_map default_map;

    virtual function void build();
        ctrl = fa_reg_ctrl::type_id::create("ctrl");
        ctrl.configure(this, null, "");
        ctrl.build();
        // ... 14 more regs configured similarly ...

        default_map = create_map("default_map", 0, 4, UVM_LITTLE_ENDIAN);
        default_map.add_reg(ctrl,          'h00, "RW");
        default_map.add_reg(status,        'h04, "RW");
        default_map.add_reg(cfg,           'h08, "RW");
        default_map.add_reg(q_base_l,      'h14, "RW");
        // ... 11 more add_reg ...
        default_map.add_reg(cycles,        'h40, "R0");
        lock_model();
    endfunction
endclass

```

说明: 15 个寄存器逐个 add_reg(reg, offset, access) 注册到 default_map。lock_model() 之后 RAL 结构定型, 可被 sequence 用 reg_model.ctrl.write(status, val) 这样高层访问。

4.1.5.3 块 ③ RAL adapter (行 193-223)

```

class fa_reg_adapter extends uvm_reg_adapter;
    `uvm_object_utils(fa_reg_adapter)

    function new(string name = "fa_reg_adapter");
        super.new(name);
        supports_byte_enable = 0;
        provides_responses    = 0;
    endfunction

    virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);
        axi4_lite_txn txn = axi4_lite_txn::type_id::create("txn");
        txn.addr        = rw.addr[7:0];
        txn.is_write     = (rw.kind == UVM_WRITE);
        txn.data         = rw.data[31:0];
        return txn;
    endfunction

```

```

endfunction

virtual function void bus2reg(uvm_sequence_item bus_item,
                             ref uvm_reg_bus_op rw);
    axi4_lite_txn txn;
    if (!$cast(txn, bus_item)) begin
        `uvm_fatal("ADAPT", "Failed to cast bus_item to axi4_lite_txn")
        return;
    end
    rw.kind    = txn.is_write ? UVM_WRITE : UVM_READ;
    rw.addr    = {56'd0, txn.addr};
    rw.data    = txn.is_write ? txn.data : txn.rdata;
    rw.status  = UVM_IS_OK;
endfunction
endclass

```

说明: Adapter 是 RAL 与具体总线协议 (AXI4-Lite) 之间的“翻译层”。reg2bus 把 uvm_reg_bus_op (地址 + 读写 + 数据) 转成 axi4_lite_txn; bus2reg 反向, 从 monitor 捕获的 txn 恢复 uvm_reg_bus_op 给 predictor 用。

4.1.6 3.1.6 tb/uvm_env/fa_coverage.sv

4.1.6.1 块 ① 类骨架与跟踪变量 (行 1-30)

```

class fa_coverage extends uvm_subscriber #(axi4_lite_txn);
    `uvm_component_utils(fa_coverage)

    bit causal_en;
    bit [2:0] data_pattern;
    int test_count;
    int dma_rd_count, dma_wr_count;
    int total_rd_bytes, total_wr_bytes;
    int exec_cycles;
    real throughput_elem_per_cycle;
    int axi_rd_bursts, axi_wr_bursts;
    int axi_rd_bytes, axi_wr_bytes;

    bit vp_zero_tested, vp_random_tested, vp_causal_tested;
    bit vp_boundary_tested, vp_identity_tested, vp_maxval_tested;
    bit vp_nocausal_tested;
    bit dma_integrity_tested, dma_backtoback_tested;
    bit reg_reset_tested, reg_rw_tested, reg_stress_tested, reg_ral_tested;

```

```

bit axi_protocol_tested;
bit perf_tested;

```

说明: 继承 uvm_subscriber #(axi4_lite_txn) 自动有 analysis_export + write(t) 钩子。许多 bit 标志位用来跟踪每个 verification point 是否已运行 (用于“测试完成度”覆盖率)。

4.1.6.2 块 ② 典型 covergroup 示例 (fa_reg_cg)

```

covergroup fa_reg_cg with function sample(bit [7:0] addr, bit is_write);
  addr_cp: coverpoint addr {
    bins ctrl      = {8'h00};
    bins status    = {8'h04};
    bins cfg       = {8'h08};
    bins q_base_l  = {8'h14};
    // ... 14 个 bins for each register ...
    bins cycles    = {8'h40};
    bins unmapped  = default;
  }
  rw_cp: coverpoint is_write {
    bins read  = {0};
    bins write = {1};
  }
  addr_x_rw: cross addr_cp, rw_cp;
endgroup

```

说明: 全文件共 **10 个 covergroup**: 1. fa_config_cg — causal × data pattern 交叉 2. fa_reg_cg — 寄存器地址 × 读写交叉 3. fa_reg_val_cg — 寄存器特征值 (CTRL.START、CFG.causal_en、stride、scale) 4. fa_dma_cg — DMA 方向 × 大小 5. fa_axi_burst_cg — AXI4 突发: 方向 × 长度 × size × burst type 6. fa_perf_cg — 总周期分箱 7. fa_perf_detail_cg — 读带宽 × 写带宽 × 总线利用率 8. fa_vp_cg — 15 个 VP 完成度跟踪 9. (在 axi4_mem_agent 内) axi4_txn_cg 10. (在 axi4_lite_agent 内) axil_txn_cg

4.1.6.3 块 ③ AXI 突发覆盖率桥接 (行 296-309)

```

class fa_axi_cov_sub extends uvm_subscriber #(axi4_burst_txn);
  `uvm_component_utils(fa_axi_cov_sub)
  fa_coverage cov_ref;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

```

```

    function void write(axi4_burst_txn t);
        if (cov_ref != null)
            cov_ref.sample_axi_burst(t);
    endfunction
endclass

```

说明: 因为 fa_coverage 已经被绑定到 axi4_lite_txn (不能同时订阅两种类型), 用一个独立的 subscriber fa_axi_cov_sub 桥接 axi4_burst_txn 到 fa_coverage.sample_axi_burst()。

4.2 3.2 AXI4-Lite Agent (6 个文件)

4.2.1 3.2.1 tb/agents/axi4_lite_agent/axi4_lite_txn.sv (21 行)

4.2.1.1 完整源代码

```

class axi4_lite_txn extends uvm_sequence_item;
    `uvm_object_utils(axi4_lite_txn)

    rand bit [7:0]  addr;
    rand bit [31:0] data;
    rand bit        is_write;
    bit [31:0]      rdata;
    bit [1:0]       resp;

    constraint addr_align_c { addr[1:0] == 2'b00; }

    function new(string name = "axi4_lite_txn");
        super.new(name);
    endfunction

    function string convert2string();
        return $sformatf("%s addr=0x%02h data=0x%08h rdata=0x%08h resp=%0d",
                        is_write ? "WR" : "RD", addr, data, rdata, resp);
    endfunction
endclass

```

说明: 标准 UVM transaction 类。addr/data/is_write 是 random 的 (writable, 可被 sequence 控制); rdata/resp 是 monitor/driver 写回的。约束 addr[1:0]=0 保证 4 字节对齐 — 这是 AXI4-Lite 协议要求。过于简单, 无需图示。

4.2.2 3.2.2 tb/agents/axi4_lite_agent/axi4_lite_if.sv (29 行)**4.2.2.1 完整源代码**

```

interface axi4_lite_if (input logic clk, input logic rst_n);
    logic [7:0] s_axil_awaddr;
    logic      s_axil_awvalid, s_axil_awready;
    logic [31:0] s_axil_wdata;
    logic [3:0] s_axil_wstrb;
    logic      s_axil_wvalid, s_axil_wready;
    logic [1:0] s_axil_bresp;
    logic      s_axil_bvalid, s_axil_bready;
    logic [7:0] s_axil_araddr;
    logic      s_axil_arvalid, s_axil_arready;
    logic [31:0] s_axil_rdata;
    logic [1:0] s_axil_rresp;
    logic      s_axil_rvalid, s_axil_rready;
endinterface

```

说明: 单纯的 wire 集合, 没有 modport 也没有 clocking block — 简化但牺牲了驱动/采样的精确时序保护。过于简单, 无需图示。

4.2.3 3.2.3 tb/agents/axi4_lite_agent/axi4_lite_sequencer.sv (8 行)**4.2.3.1 完整源代码**

```

class axi4_lite_sequencer extends uvm_sequencer #(axi4_lite_txn);
    `uvm_component_utils(axi4_lite_sequencer)
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
endclass

```

说明: 一行的 uvm_sequencer 模板专门化。过于简单, 无需图示。

4.2.4 3.2.4 tb/agents/axi4_lite_agent/axi4_lite_agent.sv (27 行)**4.2.4.1 完整源代码**

```

class axi4_lite_agent extends uvm_agent;
    `uvm_component_utils(axi4_lite_agent)

    axi4_lite_driver    drv;

```

```

    axi4_lite_monitor    mon;
    axi4_lite_sequencer  sqr;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon = axi4_lite_monitor::type_id::create("mon", this);
        if (get_is_active() == UVM_ACTIVE) begin
            drv = axi4_lite_driver::type_id::create("drv", this);
            sqr = axi4_lite_sequencer::type_id::create("sqr", this);
        end
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if (get_is_active() == UVM_ACTIVE)
            drv.seq_item_port.connect(sqr.seq_item_export);
    endfunction
endclass

```

说明: 标准 UVM agent 模板: build mon (always), build drv/sqr (if active), connect drv ↔ sqr (TLM 1.0 port pair)。

4.2.5 3.2.5 tb/agents/axi4_lite_agent/axi4_lite_driver.sv (82 行)

4.2.5.1 块 ① 框架与 build_phase (行 1-17)

```

class axi4_lite_driver extends uvm_driver #(axi4_lite_txn);
    `uvm_component_utils(axi4_lite_driver)

    virtual axi4_lite_if vif;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual axi4_lite_if)::get(this, "", "vif", vif))

```

```

        `uvm_fatal("NOVIF", "axi4_lite_if not found in config_db")
    endfunction

```

说明: uvm_config_db 取出虚接口 (在 fa_tb_top 内由 set 注入)。

4.2.5.2 块② run_phase 主循环 (行 17-34)

```

task run_phase(uvm_phase phase);
    axi4_lite_txn txn;
    wait(vif.rst_n === 1'b1);
    @(posedge vif.clk);
    forever begin
        seq_item_port.get_next_item(txn);
        drive_txn(txn);
        seq_item_port.item_done();
    end
endtask

task drive_txn(axi4_lite_txn txn);
    if (txn.is_write)
        drive_write(txn);
    else
        drive_read(txn);
endtask

```

说明: 等待复位释放后进入 forever { get → drive → done } 标准 UVM driver 循环。

4.2.5.3 块③ 写事务驱动 (行 35-64)

```

task drive_write(axi4_lite_txn txn);
    // Write address + data simultaneously
    @(posedge vif.clk);
    vif.s_axil_awaddr <= txn.addr;
    vif.s_axil_awvalid <= 1'b1;
    vif.s_axil_wdata <= txn.data;
    vif.s_axil_wstrb <= 4'hF;
    vif.s_axil_wvalid <= 1'b1;

    fork
        begin
            wait(vif.s_axil_awready);
            @(posedge vif.clk);

```

```

        vif.s_axil_awvalid <= 1'b0;
    end
    begin
        wait(vif.s_axil_wready);
        @(posedge vif.clk);
        vif.s_axil_wvalid <= 1'b0;
    end
    join

    // Wait for write response
    vif.s_axil_bready <= 1'b1;
    wait(vif.s_axil_bvalid);
    txn.resp = vif.s_axil_bresp;
    @(posedge vif.clk);
    vif.s_axil_bready <= 1'b0;
endtask

```

说明: AW + W 同时驱动 (满足 DUT 的 “二者同步” 要求), 用 fork...join 等两个 ready, 最后等 B 通道。wstrb=4'hF = 所有字节都写。

4.2.5.4 块④ 读事务驱动 (行 66-81)

```

task drive_read(axi4_lite_txn txn);
    @(posedge vif.clk);
    vif.s_axil_araddr <= txn.addr;
    vif.s_axil_arvalid <= 1'b1;

    wait(vif.s_axil_arready);
    @(posedge vif.clk);
    vif.s_axil_arvalid <= 1'b0;

    vif.s_axil_rready <= 1'b1;
    wait(vif.s_axil_rvalid);
    txn.rdata = vif.s_axil_rdata;
    txn.resp = vif.s_axil_rresp;
    @(posedge vif.clk);
    vif.s_axil_rready <= 1'b0;
endtask
endclass

```

说明: AR → 等 arready → 拉 rready → 等 rvalid → 抓 rdata。简单的 valid/ready 握手。

4.2.6 3.2.6 tb/agents/axi4_lite_agent/axi4_lite_monitor.sv (69 行)

4.2.6.1 完整 source (关键部分)

```
class axi4_lite_monitor extends uvm_monitor;
    `uvm_component_utils(axi4_lite_monitor)
    virtual axi4_lite_if vif;
    uvm_analysis_port #(axi4_lite_txn) ap;

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        ap = new("ap", this);
        if (!uvm_config_db#(virtual axi4_lite_if)::get(this, "", "vif", vif))
            `uvm_fatal("NOVIF", "axi4_lite_if not found in config_db")
    endfunction

    task run_phase(uvm_phase phase);
        fork
            monitor_writes();
            monitor_reads();
        join
    endtask

    task monitor_writes();
        bit [7:0] aw_addr;
        bit [31:0] w_data;
        bit aw_seen, w_seen;
        forever begin
            @(posedge vif.clk);
            if (vif.s_axil_awvalid && vif.s_axil_awready) begin
                aw_addr = vif.s_axil_awaddr;
                aw_seen = 1;
            end
            if (vif.s_axil_wvalid && vif.s_axil_wready) begin
                w_data = vif.s_axil_wdata;
                w_seen = 1;
            end
            if (aw_seen && w_seen) begin
                axi4_lite_txn txn = axi4_lite_txn::type_id::create("wr_txn");
                txn.is_write = 1;
                txn.addr      = aw_addr;
                txn.data       = w_data;
```

```

        ap.write(txn);
        aw_seen = 0; w_seen = 0;
    end
end
endtask

```

```

    task monitor_reads(); /* same idea: capture AR addr, then on RVALID create RD txn */ endtask
endclass

```

说明: 两个并行 forever 任务分别监控写和读通道。**写监控关键点:** AW 和 W 可能不同步握手, 所以用 aw_seen/w_seen 标志位等齐两者后才生成 txn 并广播到 analysis port。所有 subscriber (coverage, predictor) 都收到这个 txn。

4.3 3.3 AXI4 Memory Agent + 协议检查器

4.3.1 3.3.1 tb/agents/axi4_mem_agent/axi4_mem_if.sv (37 行)

只是一个 30 个信号的 AXI4 接口声明, 与上面的 axi4_lite_if 结构同质。完整源代码已在第 2 章引用, 过于简单不再图示。

4.3.2 3.3.2 tb/agents/axi4_mem_agent/axi4_mem_agent.sv (188 行)

4.3.2.1 块 ① 类与 burst txn 定义 (行 1-35)

```

// AXI4 burst transaction for monitoring
class axi4_burst_txn extends uvm_sequence_item;
    `uvm_object_utils(axi4_burst_txn)
    bit [63:0]  addr;
    bit [7:0]   len;
    bit [2:0]   size;
    bit [1:0]   burst;
    bit [3:0]   id;
    bit         is_write;
    int         byte_count;
    function new(string name = "axi4_burst_txn");
        super.new(name);
    endfunction
endclass

```

```

class axi4_mem_agent extends uvm_component;
  `uvm_component_utils(axi4_mem_agent)
  virtual axi4_mem_if vif;
  bit [7:0] mem [bit [63:0]]; // sparse memory
  uvm_analysis_port #(axi4_lite_txn) wr_ap;

```

说明: 关键设计: mem 是**关联数组** (associative array) 用 64-bit 地址索引、8-bit 字节存储。这样可以模拟 64-bit 地址空间但只为实际访问的字节分配存储 — 经典的稀疏内存模型。

4.3.2.2 块 ② 内存助手函数 (行 62-101)

```

function void preload_matrix(input bit [63:0] base_addr, input int rows, input int cols,
                             input shortint data[][]);
  for (int r = 0; r < rows; r++)
    for (int c = 0; c < cols; c++)
      write_half(base_addr + (r * cols + c) * 2, data[r][c]);
endfunction

function void readback_matrix(input bit [63:0] base_addr, input int rows, input int cols,
                              ref shortint data[][]);
  data = new[rows];
  for (int r = 0; r < rows; r++) begin
    data[r] = new[cols];
    for (int c = 0; c < cols; c++)
      data[r][c] = shortint'(read_half(base_addr + (r * cols + c) * 2));
  end
endfunction

```

说明: preload_matrix(base, rows, cols, data) 让测试在仿真开始前把 Q/K/V 矩阵填到内存模型中; readback_matrix 等仿真结束后从同一内存读出 O 结果给 scoreboard 比对。这两个函数是连接“软件 reference 输入”与“硬件 DMA 行为”的核心桥梁。

4.3.2.3 块 ③ AXI4 读响应 (行 110-147)

```

task handle_reads();
  forever begin
    bit [63:0] addr;
    int        burst_len;
    int        beat_size;

    @(posedge vif.clk);
    while (!vif.m_axi_arvalid) @(posedge vif.clk);

```

```

    addr      = vif.m_axi_araddr;
    burst_len = vif.m_axi_arlen + 1;
    beat_size = 1 << vif.m_axi_arsize;

    vif.m_axi_arready <= 1'b1;
    @(posedge vif.clk);
    vif.m_axi_arready <= 1'b0;

    for (int i = 0; i < burst_len; i++) begin
        logic [127:0] rdata;
        for (int b = 0; b < beat_size; b++)
            rdata[b*8 +: 8] = read_byte(addr + b);

        vif.m_axi_rdata <= rdata;
        vif.m_axi_rid    <= vif.m_axi_arid;
        vif.m_axi_rresp  <= 2'b00;
        vif.m_axi_rlast  <= (i == burst_len - 1);
        vif.m_axi_rvalid <= 1'b1;

        @(posedge vif.clk);
        while (!vif.m_axi_rready) @(posedge vif.clk);
        addr += beat_size;
    end
    vif.m_axi_rvalid <= 1'b0;
    vif.m_axi_rlast  <= 1'b0;
end
endtask

```

说明: 标准的 AXI4 slave read responder: 等 ARVALID → 抓 burst 参数 → 单周期 ARREADY → 然后 burst_len 拍依次返回 rdata, 每拍按 beat_size 个字节从 mem 拼出 128-bit。最后一拍带 RLAST=1。handle_writes 任务对称 (接收数据写入 mem, 最后发 BVALID)。

4.3.3 3.3.3 tb/agents/axi4_protocol_checker.sv (282 行) — 21 条 SVA

4.3.3.1 块 ① 模块端口 (行 1-51)

```

module axi4_protocol_checker #(
    parameter ADDR_WIDTH = 64,
    parameter DATA_WIDTH = 128,
    parameter ID_WIDTH   = 4
)(

```

```

input logic clk, rst_n,
// AW, W, B, AR, R 共 30 个 AXI4 信号
input logic [ID_WIDTH-1:0] awid,
input logic [ADDR_WIDTH-1:0] awaddr,
/* ... */
input logic                                rready
);

```

说明:输入端口与 DUT 的 AXI4 master 信号一一对应,由 fa_tb_top 在 `ifdef ENABLE_PROTOCOL_CHECK 块内 bind/wire。

4.3.3.2 块 ② 典型 SVA 断言示例 (行 84-106)

```

// AW_STABLE: Once AWVALID is asserted, AWADDR/AWLEN/AWSIZE/AWBURST must stay stable
property aw_stable_p(signal);
    @(posedge clk) disable iff (!rst_n)
        (awvalid && !awready) | => ($stable(signal) && awvalid);
endproperty

AW_ADDR_STABLE: assert property (aw_stable_p(awaddr))
    else $error("AXI4: AWADDR changed while AWVALID && !AWREADY");
AW_LEN_STABLE: assert property (aw_stable_p(awlen))
    else $error("AXI4: AWLEN changed while AWVALID && !AWREADY");
AW_SIZE_STABLE: assert property (aw_stable_p(awsiz))
    else $error("AXI4: AWSIZE changed while AWVALID && !AWREADY");
AW_BURST_STABLE: assert property (aw_stable_p(awburst))
    else $error("AXI4: AWBURST changed while AWVALID && !AWREADY");
AW_ID_STABLE: assert property (aw_stable_p(awid));

```

说明:用 SVA property 定义 “VALID && !READY → 下个周期 signal 必须稳定” 的协议约束,再实例化为多条 assert property 检查每个信号。整个文件共 **21 条断言**,覆盖: (1) AW/W/AR/R/B 五通道的 stable 与 valid_hold 规则; (2) 突发参数有效性 (awburst != 2'b11、awsiz <= 7、wlast 出现时机); (3) X/Z 检查 (被 valid 守护的关键信号不能为 X); (4) 计数检查 (B 数量 = AW 数量)。

4.3.4 3.3.4 tb/agents/axi4_lite_protocol_checker.sv (214 行) — 18 条 SVA

4.3.4.1 块 ④ 4 字节对齐断言 (行 64-67)

```

AXIL_AW_ALIGN: assert property (
    @(posedge clk) disable iff (!rst_n)
        awvalid |-> (awaddr[1:0] == 2'b00)
) else $error("AXIL: AWADDR not 4-byte aligned");

```

说明: AXI4-Lite 要求所有地址 4 字节对齐。这条断言在 AWVALID 时检查 awaddr[1:0] == 0, 否则报错。整个文件 **18 条断言**: 5 通道的 stable + valid_hold + 4 字节对齐 + X 检查。比 AXI4 协议检查器少 (少了 burst 相关)。

4.4 3.4 sequences + tests + tb_top + system_tb + axi4_slave_mem

4.4.1 3.4.1 tb/sequences/fa_sequences.sv (322 行) — 8 个 sequence

8 个 UVM sequence 类:

Sequence	行数	用途
fa_reg_write_seq	22	单个寄存器写 (传 addr/data)
fa_reg_read_seq	18	单个寄存器读
fa_config_and_start_seq	38	配置 12 个寄存器 + 写 CTRL.START
fa_poll_done_seq	32	轮询 STATUS.DONE, 抓 CYCLES
fa_rand_reg_seq	27	100 次随机寄存器读写
fa_reg_reset_seq	38	复位值检查
fa_reg_stress_seq	30	50 轮背靠背 W→R 一致性
fa_reg_walk_bit_seq	40	walking-1 / walking-0 逐位测试
fa_reg_unmapped_seq	18	未映射地址访问
fa_soft_reset_seq	30	软复位序列

4.4.1.1 块 ① 完整配置 + 启动 sequence (行 47-86) — 最重要的一个

```
class fa_config_and_start_seq extends uvm_sequence #(axi4_lite_txn);
    `uvm_object_utils(fa_config_and_start_seq)
    bit [63:0] q_base, k_base, v_base, o_base;
    bit [31:0] stride_bytes;
    bit [15:0] neg_large;
    bit [15:0] scale;
    bit      causal_en;

    function new(string name = "fa_config_and_start_seq");
        super.new(name);
        stride_bytes = 32'd128;
        neg_large    = 16'h8000;
        scale        = 16'h0020;
    endfunction
```

```

task body();
    write_reg(8'h14, q_base[31:0]);
    write_reg(8'h18, q_base[63:32]);
    write_reg(8'h1C, k_base[31:0]);
    write_reg(8'h20, k_base[63:32]);
    write_reg(8'h24, v_base[31:0]);
    write_reg(8'h28, v_base[63:32]);
    write_reg(8'h2C, o_base[31:0]);
    write_reg(8'h30, o_base[63:32]);
    write_reg(8'h34, stride_bytes);
    write_reg(8'h38, {16'h0, neg_large});
    write_reg(8'h3C, {16'h0, scale});
    write_reg(8'h08, {31'd0, causal_en});
    write_reg(8'h00, 32'h0000_0001);    // CTRL.START = 1
endtask

task write_reg(bit [7:0] addr, bit [31:0] data);
    fa_reg_write_seq wr;
    wr = fa_reg_write_seq::type_id::create("wr");
    wr.addr = addr; wr.data = data;
    wr.start(m_sequencer);
endtask
endclass

```

说明: 这是测试中最常用的 sequence — 写完 12 个配置寄存器后写 CTRL.START=1 触发硬件运行。每个 write_reg 内部新建一个 fa_reg_write_seq 子 sequence 启动, 达到 sequence 嵌套 + 复用。

4.4.1.2 块 ② 轮询完成 sequence (行 88-121)

```

class fa_poll_done_seq extends uvm_sequence #(axi4_lite_txn);
    `uvm_object_utils(fa_poll_done_seq)
    int timeout_cycles = 500000;
    bit done;
    bit [31:0] cycles;

    task body();
        fa_reg_read_seq rd;
        int cnt = 0;
        done = 0;
        while (!done && cnt < timeout_cycles) begin
            rd = fa_reg_read_seq::type_id::create("rd");

```

```

        rd.addr = 8'h04;
        rd.start(m_sequencer);
        if (rd.rdata[1]) begin
            done = 1;
            rd = fa_reg_read_seq::type_id::create("rd_cyc");
            rd.addr = 8'h40;          // CYCLES
            rd.start(m_sequencer);
            cycles = rd.rdata;
        end
        cnt++;
    end
    if (!done)
        `uvm_error("POLL", $sformatf("Timeout after %0d polls", timeout_cycles))
    endtask
endclass

```

说明: 软件视角下“等任务完成”的标准模式: 循环读 STATUS, 看 bit[1]=DONE 是否为 1; 为 1 就读 CYCLES (0x40) 取性能数据。timeout_cycles=500000 保护无限循环。

4.4.1.3 块 ③ Walking-1/0 bit 测试 (行 228-268)

```

class fa_reg_walk_bit_seq extends uvm_sequence #(axi4_lite_txn);
    `uvm_object_utils(fa_reg_walk_bit_seq)

    task body();
        bit [7:0] addrs[] = '{8'h14, 8'h18, 8'h34, 8'h38, 8'h3C};

        foreach (addrs[a]) begin
            // Walking-1
            for (int bit_pos = 0; bit_pos < 32; bit_pos++) begin
                bit [31:0] pattern = (32'd1 << bit_pos);
                fa_reg_write_seq wr = fa_reg_write_seq::type_id::create("wr");
                fa_reg_read_seq rd = fa_reg_read_seq::type_id::create("rd");
                wr.addr = addrs[a]; wr.data = pattern;
                wr.start(m_sequencer);
                rd.addr = addrs[a];
                rd.start(m_sequencer);
                if (rd.rdata !== pattern)
                    `uvm_error("WALK1", $sformatf("addr=0x%02h bit=%0d W=0x%08h R=0x%08h",
                                                    addrs[a], bit_pos, pattern, rd.rdata))
            end
            // Walking-0 (similar with ~pattern)

```

```
        end
    endtask
endclass
```

说明: 经典 RAM 测试 walking-1 (写 0x1, 0x2, 0x4, ..., 0x80000000, 每次只 1 个 bit 为 1) + walking-0 (取反)。如果有粘连 bit 或位错误, 肯定会被检测到。这是 REG05 测试用的 sequence。

4.4.2 3.4.2 tb/tests/fa_tests.sv (993 行) — 24 个 UVM 测试

整个文件结构: 1 个基类 fa_base_test + 24 个派生测试, 按 6 个 verification domain 组织。

Baseline UVM Test Suite — 24 Tests across 6 Domains

Verification Points VP01-VP06 (6 tests)	fa_zero_test	fa_random_nocausal_test	fa_random_causal_test
	fa_identity_test	fa_boundary_test	fa_maxval_test
Register Verification REG01-REG07 (7 tests)	fa_reg_reset_test	fa_reg_access_test	fa_reg_stress_test
	fa_reg_walk_test	fa_reg_unmapped_test	fa_reg_soft_reset_test
DMA Verification DMA01-DMA03 (3 tests)	fa_dma_test	fa_dma_b2b_test	fa_dma_addr_test
AXI Protocol AXI01-AXI03 (3 tests)	fa_axi_protocol_test	fa_axi_dual_mode_test	fa_axi_regonly_test
Performance PERF01-PERF02 (2 tests)	fa_perf_test	fa_perf_compare_test	
Coverage + Comprehensive (3 tests)	fa_coverage_test	fa_coverage_closure_test	fa_comprehensive_test

图 13: 24 baseline tests

4.4.2.1 总览图: 测试套件分组

4.4.2.2 块 ① 基类 fa_base_test (行 7-117) — 提供共享方法

```
class fa_base_test extends uvm_test;
    `uvm_component_utils(fa_base_test)
    fa_env env;

    bit [63:0] Q_BASE = 64'h0000_0000_0001_0000;
    bit [63:0] K_BASE = 64'h0000_0000_0002_0000;
```

```

bit [63:0] V_BASE = 64'h0000_0000_0003_0000;
bit [63:0] O_BASE = 64'h0000_0000_0004_0000;

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = fa_env::type_id::create("env", this);
endfunction

task run_attention(input bit causal_en, input shortint q[[[[]],
                    input shortint k[[[[]], input shortint v[[[[]]);
    fa_config_and_start_seq cfg_seq;
    fa_poll_done_seq        poll_seq;

    preload_matrix(Q_BASE, q);
    preload_matrix(K_BASE, k);
    preload_matrix(V_BASE, v);

    env.scoreboard.q_data = q;
    env.scoreboard.k_data = k;
    env.scoreboard.v_data = v;
    env.scoreboard.causal_en = causal_en;

    cfg_seq = fa_config_and_start_seq::type_id::create("cfg");
    cfg_seq.q_base    = Q_BASE; cfg_seq.k_base = K_BASE;
    cfg_seq.v_base    = V_BASE; cfg_seq.o_base = O_BASE;
    cfg_seq.causal_en = causal_en;
    cfg_seq.start(env.axil_agent.sqr);

    poll_seq = fa_poll_done_seq::type_id::create("poll");
    poll_seq.start(env.axil_agent.sqr);

    readback_matrix(O_BASE, 256, 64, env.scoreboard.dut_o);
    env.scoreboard.compute_golden();
    env.scoreboard.check_results();

    env.coverage.sample_config(causal_en);
    env.coverage.sample_perf(poll_seq.cycles);
endtask

function void gen_random_matrix(...); /* fill with $urandom_range */ endfunction
function void gen_zero_matrix(...); /* all-zero */ endfunction

```

```

function void gen_identity_matrix(...);/* diagonal = 1.0 (Q8.8 = 0x100) */ endfunction
function void gen_max_matrix(...);    /* all 0x3FFF */ endfunction
function void gen_boundary_matrix(...); /* min/max/+1/-1 mix */ endfunction
endclass

```

说明: 基类把“准备数据 → 预填内存 → 配置 + 启动 → 轮询完成 → 读回结果 → 比对 → 采样覆盖率”流水线封装好。所有派生测试只需要在 run_phase 内 (1) 用对应的 gen_*_matrix 生成 q/k/v; (2) 调 run_attention; (3) 采样对应的 vp 完成度。

4.4.2.3 块② 一个典型 testcase 示例 (VP01 Zero Test)

```

class fa_zero_test extends fa_base_test;
    `uvm_component_utils(fa_zero_test)
    function new(string name, uvm_component parent); super.new(name, parent); endfunction

    task run_phase(uvm_phase phase);
        shortint q[][], k[][], v[][];
        phase.raise_objection(this);
        `uvm_info("TEST", "VP01: Zero input test", UVM_LOW)
        gen_zero_matrix(q, 256, 64);
        gen_zero_matrix(k, 256, 64);
        gen_zero_matrix(v, 256, 64);
        run_attention(0, q, k, v);
        env.coverage.sample_data(3'd0);
        env.coverage.sample_vp("zero");
        phase.drop_objection(this);
    endtask
endclass

```

说明: 24 个测试都是这个模板的变体, 只是数据生成 + causal_en + 采样的 vp 名不同。整个文件 993 行的 90% 都是这种重复模板, 所以这里只展示一个代表性例子。完整 24 个测试见上面的 images/03_test_suite.png 概览图。

4.4.3 3.4.3 tb/tb_top/fa_tb_top.sv (191 行)

4.4.3.1 块① 时钟、复位、接口 (行 1-32)

```

`timescale 1ns/1ps
`include "fa_params.svh"

module fa_tb_top;
    import uvm_pkg::*;

```

```

`include "uvm_macros.svh"
import fa_env_pkg::*;

logic clk, rst_n;
initial begin clk = 0; forever #1 clk = ~clk; end    // 500 MHz
initial begin rst_n = 0; repeat(10) @(posedge clk); rst_n = 1; end

axi4_lite_if axil_if (.clk(clk), .rst_n(rst_n));
axi4_mem_if mem_if (.clk(clk), .rst_n(rst_n));

```

说明: forever #1 clk = ~clk 周期 2 ns = 500 MHz 时钟。复位 10 拍后释放。两个虚接口实例化。

4.4.3.2 块② DUT 实例化 (行 33-89)

```

flash_attention_top u_dut (
    .clk(clk), .rst_n(rst_n),
    // AXI4-Lite Slave: 17 ports wired to axil_if.s_axil_*
    .s_axil_awaddr (axil_if.s_axil_awaddr),
    /* ... */
    // AXI4 Master: 31 ports wired to mem_if.m_axi_*
    .m_axi_awid (mem_if.m_axi_awid),
    /* ... */
    .irq()
);

```

说明: 通过 interface 信号把 DUT 和 agent 连起来。irq 暂时悬空 (没有 IRQ 测试)。

4.4.3.3 块③ 协议检查器 (行 91-150)

```

`ifdef ENABLE_PROTOCOL_CHECK
    axi4_lite_protocol_checker #(.ADDR_WIDTH(8), .DATA_WIDTH(32))
    u_axil_checker (
        .clk(clk), .rst_n(rst_n),
        .awaddr(axil_if.s_axil_awaddr), /* ... 18 ports ... */
    );

    axi4_protocol_checker #(.ADDR_WIDTH(64), .DATA_WIDTH(128), .ID_WIDTH(4))
    u_axi_checker (
        .clk(clk), .rst_n(rst_n),
        .awid(mem_if.m_axi_awid), /* ... 30 ports ... */
    );
`endif

```

说明: 协议检查器只在编译 +define+ENABLE_PROTOCOL_CHECK 时启用 — 仿真时通常打开, 只在 gate sim 时关闭以加速。这里直接实例化 (不用 bind) 是因为检查器和 DUT 在同一个 testbench 模块。

4.4.3.4 块 ④ 配置 DB 与 run_test (行 152-179)

```

initial begin
    uvm_config_db#(virtual axi4_lite_if)::set(null, "*axil_agent*", "vif", axil_if);
    uvm_config_db#(virtual axi4_mem_if)::set(null, "*mem_agent*", "vif", mem_if);
end

initial begin
    // Initialize all axil_if signals to deasserted
    axil_if.s_axil_awaddr = '0;
    /* ... */
end

initial begin
    run_test(); // UVM picks up +UVM_TESTNAME from command line
end

initial begin
    #20_000_000;
    `uvm_fatal("TIMEOUT", "Simulation timeout!")
end

initial begin
    if ($test$plusargs("DUMP_VCD")) begin
        $dumpfile("wave.vcd");
        $dumpvars(0, fa_tb_top);
    end
end
endmodule

```

说明: uvm_config_db::set 把 vif 注入到 agent 的虚接口字段。run_test() 读 +UVM_TESTNAME=fa_xxx_test 命令行参数选具体测试。20 ms 超时 (500MHz 下约 1000 万拍) 足够任意一个测试完成。

4.4.4 3.4.4 tb/unit_tb/system_tb.sv (317 行)

非 UVM 的端到端 testbench (baseline 直接 TB)。结构: 实例化 DUT + axi4_slave_mem 模块化的 AXI4 slave, 再用 procedural code (initial + task) 做配置 + start + 轮询 + 读回 + 与浮点 reference 比对。作用是给不熟悉 UVM 的人提供一个简单端到端例子, 由 scripts/run_baseline.sh 调用。

4.4.4.1 块① 信号声明 + DUT + slave_mem 实例 (行 1-80 节选)

```

`timescale 1ns/1ps
`include "fa_params.svh"

module system_tb;
    logic clk, rst_n;
    initial begin clk = 0; forever #1 clk = ~clk; end
    initial begin rst_n = 0; repeat(20) @(posedge clk); rst_n = 1; end

    /* AXI4-Lite + AXI4 Master 信号全部声明 */

    flash_attention_top dut (
        .clk(clk), .rst_n(rst_n),
        /* ... 全部信号绑定 ... */
        .irq(irq)
    );

    axi4_slave_mem u_mem (
        .clk(clk), .rst_n(rst_n),
        .araddr(m_axi_araddr), /* ... */
    );

```

说明: 与 fa_tb_top 不同, 这里 slave 是 RTL 模块 axi4_slave_mem, 不是 UVM agent。后续部分 (行 60-317) 是 procedural test code: preload Q/K/V → write registers → wait DONE → readback O → 浮点 reference + 误差检查。

4.4.5 3.4.5 tb/unit_tb/axi4_slave_mem.sv (208 行)

模块化 AXI4 slave 内存模型, 配套 system_tb.sv 用。功能与 axi4_mem_agent 几乎相同, 但写法是 RTL 风格 (all always_ff + case FSM) 而非 UVM class。

4.4.5.1 块① 端口与稀疏内存 (行 1-60)

```

`timescale 1ns/1ps
module axi4_slave_mem (
    input logic          clk, rst_n,
    /* AXI4 read/write channel ports */
    /* TB memory access ports */
    input logic          mem_wr_en,
    input logic [63:0]   mem_wr_addr,
    input logic [15:0]   mem_wr_data16,

```

```
input logic      mem_rd_en,
input logic [63:0] mem_rd_addr,
output logic [15:0] mem_rd_data16
);

reg [7:0] mem [*];      // sparse associative array

always @(posedge clk) begin
    if (mem_wr_en) begin
        mem[mem_wr_addr]      = mem_wr_data16[7:0];
        mem[mem_wr_addr + 1] = mem_wr_data16[15:8];
    end
end
/* ... AXI4 read/write FSM (180+ lines) ... */
endmodule
```

说明: 用关联数组 reg [7:0] mem [*] 实现稀疏内存。mem_wr_en/addr/data16 是 TB 用来 preload Q/K/V 矩阵的“后门”端口(不经 AXI)。AXI4 read/write FSM 与 axi4_mem_agent.handle_reads/writes 等价, 省略详细代码。

第 3 章结束。 20 个 baseline UVM 文件全部讲完, 共约 3700 行 SystemVerilog 代码。第 4 章进入 bonus RTL (11 个增强模块 + fa_params_bonus.svh)。

5 第 4 章: Bonus RTL

Bonus 版本在 baseline 之上加了 **9 项加分功能**, 由 11 个增强 RTL 模块 + 1 个新参数文件实现:

§	文件	行数	对应 Bonus 编号
4.1	rtl/include/fa_params_bonus.svh	107	— (扩展参数)
4.2	rtl/flash_attention_bonus_top.sv	369	顶层 (集成全部 9 项)
4.3	rtl/tile_controller_bonus.sv	285	(2) Multi-Head + (3) 变长
4.4	rtl/compute_core_bonus.sv	375	(5) 多格式 + (6) Dropout 钩子
4.5	rtl/axi4_lite_slave_bonus.sv	230	22 个寄存器
4.6	rtl/mask_unit.sv	39	(4) Padding + Causal
4.7	rtl/dropout_unit.sv	81	(6) Dropout (LFSR)
4.8	rtl/task_queue.sv	76	(9) DMA Task Queue
4.9	rtl/bf16_exp_unit.sv	123	(1) BF16 exp
4.10	rtl/bf16_reciprocal_unit.sv	99	(1) BF16 1/x
4.11	rtl/int8_quantizer.sv	118	(7) INT8 量化

§	文件	行数	对应 Bonus 编号
4.12	rtl/axi4_stream_if.sv	149	(8) AXI4-Stream 接口

5.1 4.0 9 项 Bonus 功能总览

Bonus Features Overview — 9 Enhancements over Baseline

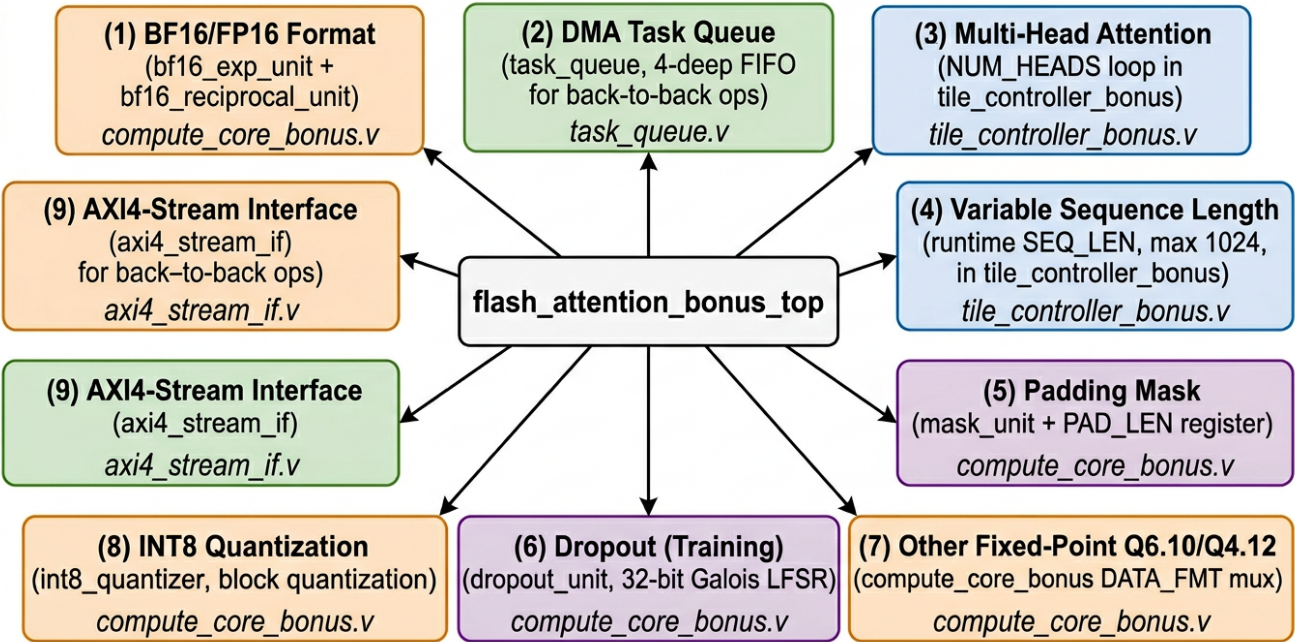


图 14: 9 Bonus features

每一项的实现思路: 1. **BF16/FP16** — 新增 bf16_exp_unit (4 阶段流水) 和 bf16_reciprocal_unit (3 阶段), compute_core_bonus 内根据 DATA_FMT 寄存器值选 LUT 路径。2. **Multi-Head** — tile_controller_bonus 在 Q/KV 循环外面再加一层 head_idx ∈ [0, NUM_HEADS) 循环, 每次循环结束加 head_stride_bytes 偏移基址。3. **Variable Sequence Length** — cfg_seq_len 是运行时寄存器 (最大 1024), num_q_tiles = cfg_seq_len / TILE_BR、num_kv_tiles = cfg_seq_len / TILE_BC 都按需算。4. **Padding Mask** — mask_unit 在 causal 之外再判断 abs_col >= valid_len 做 padding mask。5. **多种定点格式** — Q6.10/Q4.12 通过修改 FRAC_BITS-> 移位量 + 选不同 LUT 实现; compute_core_bonus 内 DATA_FMT 选择路径。6. **Dropout** — 32-bit Galois LFSR ($x^{32} + x^{22} + x^2 + x + 1$), 用低 8 位与 drop_prob 比较。7. **INT8 量化** — 块量化 (BLOCK_SIZE=16), 找块最大值算 scale, 再 $q = \text{round}(\text{data} * 127 / \text{max})$ 。8. **AXI4-Stream** — axi4_stream_if 提供 slave/master 流接口, stream_mode=1 时直接喂 buffer, 绕过 DMA。9. **DMA Task Queue** — task_queue 4 深度 FIFO, host 一次 push 多个 attention 任务, HW 顺序 pop 执行。

5.2 4.1 rtl/include/fa_params_bonus.svh

5.2.1 4.1.1 概述

baseline fa_params.svh 的超集, 新增数据格式编码、扩展寄存器映射、bonus 配置位。过于简单, 无需图示。

5.2.2 4.1.2 关键新增片段

```
// --- Configurable dimensions (multi-head, variable seq) ---
parameter SEQ_LEN      = 256;
parameter HEAD_DIM     = 64;
parameter NUM_HEADS    = 1;
parameter MAX_SEQ_LEN = 1024;

// --- Data format selection ---
parameter DATA_FMT_Q8_8   = 3'd0;
parameter DATA_FMT_Q6_10 = 3'd1;
parameter DATA_FMT_Q4_12 = 3'd2;
parameter DATA_FMT_BF16   = 3'd3;
parameter DATA_FMT_INT8   = 3'd4;

// --- Register map offsets (extended for bonus) ---
parameter REG_SEQ_LEN_REG   = 8'h0C;
parameter REG_NUM_HEADS_REG = 8'h10;
parameter REG_PAD_LEN      = 8'h44;
parameter REG_DROPOUT_CFG  = 8'h48;
parameter REG_TASK_CTRL    = 8'h4C;
parameter REG_DATA_FMT     = 8'h50;
parameter REG_HEAD_STRIDE  = 8'h54;

// --- CFG register bits (extended) ---
parameter CFG_CAUSAL_EN    = 0;
parameter CFG_PADDING_EN   = 1;
parameter CFG_DROPOUT_EN   = 2;
parameter CFG_STREAM_MODE  = 3;

// --- Status register bits (extended) ---
parameter STATUS_QUEUE_FULL = 3;

// --- Dropout / Task Queue ---
```

```
parameter DROPOUT_LFSR_WIDTH = 32;
parameter TASK_QUEUE_DEPTH   = 4;
```

说明: 7 个新寄存器: SEQ_LEN_REG (0x0C)、NUM_HEADS_REG (0x10)、PAD_LEN (0x44)、DROPOUT_CFG (0x48)、TASK_CTRL (0x4C)、DATA_FMT (0x50)、HEAD_STRIDE (0x54), 加上 baseline 的 15 个共 22 个寄存器。DROPOUT_CFG 高 8 位是 drop_prob、低 24 位是 LFSR seed; TASK_CTRL 控制 task queue 的 push/pop。

5.3 4.2 rtl/flash_attention_bonus_top.sv

5.3.1 4.2.1 概述

集成所有 bonus 子模块的顶层。比 baseline 多了: (a) AXI4-Stream slave/master 端口; (b) 22 个寄存器的 axi4_lite_slave_bonus; (c) task_queue 实例; (d) tile_controller_bonus (multi-head + variable len + task queue 模式); (e) compute_core_bonus; (f) dropout_unit 实例。

5.3.2 4.2.2 文件架构图

整体连接关系参考 §4.0 的 bonus 总览图 — flash_attention_bonus_top 就是中心那个连接全部 9 项功能的顶层模块。端口 + 互连有 369 行, 但本质上是 **baseline** 顶层的扩展, 无需独立图示。

5.3.3 4.2.3 代码块切片 (关键差异部分)

5.3.3.1 块 ① 新增 AXI4-Stream 端口

```
module flash_attention_bonus_top (
    input  logic clk, rst_n,
    /* AXI4-Lite Slave (扩展到 8-bit addr 22 regs) — 与 baseline 同 */
    /* AXI4 Master 30 ports — 与 baseline 同 */

    // --- AXI4-Stream Slave (Q/K/V input) --- [BONUS 8]
    input  logic [127:0] s_axis_tdata,
    input  logic          s_axis_tvalid,
    output logic          s_axis_tready,
    input  logic          s_axis_tlast,
    input  logic [1:0]    s_axis_tid,          // 0=Q, 1=K, 2=V

    // --- AXI4-Stream Master (0 output) ---
    output logic [127:0] m_axis_tdata,
```

```

        output logic      m_axis_tvalid,
        input  logic      m_axis_tready,
        output logic      m_axis_tlast,

        output logic      irq
    );

```

说明: 新增 9 个 AXI4-Stream 端口 (4 个 slave + 4 个 master + tlast)。stream_mode 由 REG_CFG[3] 控制; 当 stream_mode=1 时 dma_engine 不工作, 由 axi4_stream_if 直接喂 buffer。

5.3.3.2 块 ② task_queue 与 tile_controller 联动 (关键节选)

```

/* Task queue signals */
logic task_push_valid, task_push_ready;
logic [63:0] tq_q_base, tq_k_base, tq_v_base, tq_o_base;
logic [31:0] tq_config;
logic task_pop_req, task_pop_valid;

task_queue #(.QUEUE_DEPTH(4)) u_tq (
    .clk(clk), .rst_n(rst_n),
    .push_valid(task_push_valid), .push_ready(task_push_ready),
    .push_q_base(reg_q_base), .push_k_base(reg_k_base),
    .push_v_base(reg_v_base), .push_o_base(reg_o_base),
    .push_config({reg_dropout_cfg[7:0], reg_data_fmt[2:0],
        reg_pad_len[15:0], 5'b0, reg_causal_en, reg_padding_en, reg_dropout_en}),
    .pop_req(task_pop_req), .pop_valid(task_pop_valid),
    .pop_q_base(tq_q_base), /* ... */ .pop_config(tq_config),
    .empty(tq_empty), .full(tq_full)
);

tile_controller_bonus #(/*...*/) u_tile_ctrl (
    .start(reg_start | (task_queue_mode & task_pop_valid)),
    /* ... */
    .task_queue_mode(reg_task_ctrl[0]),
    .task_pop_valid(task_pop_valid),
    .task_pop_req(task_pop_req),
    .tq_q_base(tq_q_base), .tq_k_base(tq_k_base),
    .tq_v_base(tq_v_base), .tq_o_base(tq_o_base),
    .tq_config(tq_config)
);

```

说明: 关键设计: 当 task_queue_mode=1 时, start 信号变成 reg_start | (task_queue_mode &

`task_pop_valid`) —— 即只要 `task_queue` 里有任务且 HW 空闲就自动启动。`tile_controller_bonus` 在每个任务完成后 `task_pop_req` 拉一拍, 自动把下个任务从 FIFO 弹出。

5.4 4.3 `rtl/tile_controller_bonus.sv`

5.4.1 4.3.1 概述

baseline `tile_controller` 的扩展, 主要改动:

- **新增 head 外层循环:** FSM 多了 `head_idx`, 每次 head 循环结束才结束整个 attention。地址要加 `head_idx * head_stride_bytes`。
- **运行时 SEQ_LEN:** `num_q_tiles / num_kv_tiles` 不再 `localparam`, 而是运行时 `cfg_seq_len / TILE_BR / cfg_seq_len / TILE_BC`。
- **Task queue mode:** 完成一个任务后自动从 `task_queue` pop 下一个。

架构图: FSM 与 baseline 相似 (12 状态), 外面包了一层 head 循环, 与 baseline 的 `tile_controller` FSM 概念上很接近, 不重复独立绘制。

5.4.2 4.3.2 关键新增片段

5.4.2.1 块 ① 新增端口与运行时派生量

```
module tile_controller_bonus #(
    parameter MAX_SEQ_LEN = 1024,
    /* 其他与 baseline 同 */
) (
    input  logic clk, rst_n,
    input  logic start,
    output logic all_done, busy,
    // Runtime configurable dimensions
    input  logic [15:0] cfg_seq_len,          // ≤ MAX_SEQ_LEN
    input  logic [7:0]  cfg_num_heads,
    /* ... base addresses ... */
    input  logic [31:0] head_stride_bytes,    // bytes between heads
    /* ... DMA, compute control ... */
    output logic [15:0] q_tile_idx, kv_tile_idx,
    output logic [7:0]  head_idx,             // NEW: which head
    /* ... ping-pong, write-back ... */
    // Task queue interface
    input  logic                                     task_queue_mode,
```

```

    input logic                                task_pop_valid,
    output logic                               task_pop_req,
    input logic [AXI_ADDR_WIDTH-1:0] tq_q_base, tq_k_base, tq_v_base, tq_o_base,
    input logic [31:0]                        tq_config
);

// Derived runtime values
logic [15:0] num_q_tiles, num_kv_tiles;
assign num_q_tiles  = cfg_seq_len / TILE_BR;
assign num_kv_tiles = cfg_seq_len / TILE_BC;

```

说明: `cfg_seq_len`、`cfg_num_heads`、`head_stride_bytes` 是运行时输入; `num_q_tiles / num_kv_tiles` 在 `always_comb` 中除法 (综合后会被工具替换为移位等价电路, 因为 `TILE_BR` 和 `TILE_BC` 都是 2 的幂)。

5.4.2.2 块 ② head 外层循环 (节选)

```

/* In FSM transition logic: */
ST_NEXT_Q: begin
    if (q_idx == num_q_tiles - 1) begin
        // All Q tiles done for this head
        if (head_idx == cfg_num_heads - 1) begin
            state <= ST_HEAD_NEXT_TASK;    // try task queue
        end else begin
            head_idx <= head_idx + 1;
            q_idx    <= 0;
            // Add head_stride to all base addresses
            cur_q_base <= cur_q_base + head_stride_bytes;
            cur_k_base <= cur_k_base + head_stride_bytes;
            cur_v_base <= cur_v_base + head_stride_bytes;
            cur_o_base <= cur_o_base + head_stride_bytes;
            state <= ST_LOAD_Q;
        end
    end else begin
        q_idx <= q_idx + 1;
        state <= ST_LOAD_Q;
    end
end

ST_HEAD_NEXT_TASK: begin
    if (task_queue_mode && task_pop_valid) begin
        task_pop_req <= 1;
        cur_q_base   <= tq_q_base;
    end
end

```

```

        cur_k_base    <= tq_k_base;
        cur_v_base    <= tq_v_base;
        cur_o_base    <= tq_o_base;
        // Decode config: causal_en, padding_en, dropout_en, data_fmt, pad_len
        head_idx      <= 0;  q_idx <= 0;  kv_idx <= 0;
        state         <= ST_LOAD_Q;
    end else begin
        state <= ST_ALL_DONE;  // no more tasks
    end
end
end

```

说明：Head 循环放在 Q 循环之外（最外层）。每次 head 切换时给 4 个基址都加 head_stride_bytes 偏移。所有 head 完成后进入 ST_HEAD_NEXT_TASK，如果 task_queue 还有任务就 pop 并继续；否则真正完成。

5.5 4.4 rtl/compute_core_bonus.sv

5.5.1 4.4.1 概述

baseline compute_core 的扩展，主要改动：

- **多格式数据通路：**DATA_FMT 寄存器选 Q8.8 / Q6.10 / Q4.12 / BF16 / INT8。不同格式走不同的 exp / 1/x 路径（baseline LUT 或 BF16 流水）。
- **Padding mask 集成：**调用 mask_unit 同时做 causal + padding mask。
- **Dropout 钩子：**在 P 矩阵后插入 dropout_unit（如果 dropout_en=1）。

架构图：与 baseline compute_core 的 6 状态 FSM 几乎相同，只是 S_MASK 和 S_SOFTMAX 之间多个 dropout 子状态、内部 mux 选 BF16 vs Q8.8 路径。**不重复独立绘制。**

5.5.2 4.4.2 关键新增片段

5.5.2.1 块 ① 数据格式 mux

```

// BF16 path units
bf16_exp_unit u_bf16_exp (.clk(clk), .rst_n(rst_n),
    .in_valid(bf16_exp_in_valid), .x_bf16(bf16_exp_x),
    .exp_bf16(bf16_exp_y), .out_valid(bf16_exp_out_valid));

bf16_reciprocal_unit u_bf16_recip (.clk(clk), .rst_n(rst_n),
    .in_valid(bf16_recip_in_valid), .x_bf16(bf16_recip_x),

```

```

        .recip_bf16(bf16_recip_y), .out_valid(bf16_recip_out_valid));

// Format selection mux (in S_SOFTMAX_EXP state)
always_comb begin
    case (data_fmt)
        DATA_FMT_BF16: begin
            exp_in_valid = bf16_exp_in_valid;
            exp_result    = bf16_exp_y;
        end
        default: begin // Q8.8 / Q6.10 / Q4.12 / INT8 share LUT
            exp_in_valid = lut_exp_in_valid;
            exp_result    = lut_exp_y;
        end
    endcase
end
end

```

说明: BF16 用专用 4-stage 流水; Q8.8/Q6.10/Q4.12/INT8 共用同一个 1024-项 LUT exp (INT8 在送入前先 dequantize 成 16-bit)。FRAC_BITS 不同主要影响移位量, 不影响 LUT 表本身。

5.5.2.2 块 ② Dropout 钩子 (节选)

```

// Insert dropout between softmax and PV
dropout_unit #(.WIDTH(EXP_WIDTH)) u_dropout (
    .clk(clk), .rst_n(rst_n),
    .enable(dropout_en),
    .seed(dropout_seed), .seed_load(dropout_seed_load),
    .drop_prob(dropout_prob),
    .in_valid(p_matrix_valid),
    .data_in(p_matrix_in[r][c]),
    .data_out(p_matrix_out[r][c]),
    .out_valid(p_matrix_out_valid),
    .drop_mask(/* not used */)
);

```

说明: Dropout 在 softmax 之后、output_accumulator 之前插入。dropout_en=0 时旁路 (直通); dropout_en=1 时按 LFSR 决定每个 P 元素是否归零。

5.6 4.5 rtl/axi4_lite_slave_bonus.sv

5.6.1 4.5.1 概述

baseline axi4_lite_slave 的扩展, 22 个寄存器 (多了 7 个)。代码结构与 baseline 完全相同 (写解码 case + 读解码 case), 只是寄存器列表更长。

5.6.2 4.5.2 关键新增寄存器 (写 case 节选)

```
case (s_axil_awaddr)
    8'h00: r_ctrl <= s_axil_wdata;           // baseline
    8'h04: /* W1C handling */;               // baseline
    8'h08: r_cfg <= s_axil_wdata;           // ext: bit[1]=padding, bit[2]=dropout, bit[3]=st
    8'h0C: r_seq_len <= s_axil_wdata;        // NEW
    8'h10: r_num_heads <= s_axil_wdata;      // NEW
    8'h14..0x30: /* 4 base addresses (8 regs) */;
    8'h34: r_stride <= s_axil_wdata;
    8'h38: r_neg_large <= s_axil_wdata;
    8'h3C: r_scale <= s_axil_wdata;
    8'h44: r_pad_len <= s_axil_wdata;        // NEW
    8'h48: r_dropout_cfg <= s_axil_wdata;    // NEW: high 8 = prob, low 24 = seed
    8'h4C: r_task_ctrl <= s_axil_wdata;      // NEW: bit[0]=task_queue_mode, bit[1]=push tri
    8'h50: r_data_fmt <= s_axil_wdata[2:0]; // NEW: 3-bit format select
    8'h54: r_head_stride <= s_axil_wdata;    // NEW
    default: ;
endcase
```

说明: 7 个新寄存器: SEQ_LEN_REG/NUM_HEADS_REG/PAD_LEN/DROPOUT_CFG/TASK_CTRL/DATA_FMT/HEAD_STR
其余与 baseline 一致。与 **baseline** 高度相似, 无需独立架构图。

5.7 4.6 rtl/mask_unit.sv

5.7.1 4.6.1 概述

39 行的纯组合电路, 同时支持 causal mask 和 padding mask。

5.7.2 4.6.2 完整源代码

```

`include "fa_params_bonus.svh"

module mask_unit #(
    parameter TILE_BR=4, TILE_BC=16, SEQ_LEN=256, DATA_WIDTH=16
)(
    input  logic                                causal_en,
    input  logic                                padding_en,
    input  logic [$clog2(SEQ_LEN):0]  valid_len,
    input  logic [$clog2(SEQ_LEN)-1:0] q_row_base,
    input  logic [$clog2(SEQ_LEN)-1:0] kv_col_base,
    input  logic signed [DATA_WIDTH-1:0] neg_large,
    output logic signed [DATA_WIDTH-1:0] mask_val [TILE_BR-1:0][TILE_BC-1:0]
);

integer r, c;
logic [$clog2(SEQ_LEN)-1:0] abs_row, abs_col;
logic causal_masked, padding_masked;

always_comb begin
    for (r = 0; r < TILE_BR; r = r + 1) begin
        for (c = 0; c < TILE_BC; c = c + 1) begin
            abs_row = q_row_base + r[$clog2(SEQ_LEN)-1:0];
            abs_col = kv_col_base + c[$clog2(SEQ_LEN)-1:0];

            causal_masked = causal_en && (abs_col > abs_row);
            padding_masked = padding_en && ({1'b0, abs_col} >= valid_len);

            mask_val[r][c] = (causal_masked || padding_masked)
                            ? neg_large : {DATA_WIDTH{1'b0}};
        end
    end
end
endmodule

```

5.7.3 4.6.3 解释

- 与 baseline 的 causal_mask_unit 相比, 多了 padding_en + valid_len 输入。
- padding_masked = (abs_col >= valid_len): 当列绝对位置 $\geq$ 有效长度时 (例如 batch 中实际长度 200 但 padding 到 256), mask 掉。

- 输出不是 1-bit mask_out 而是直接 neg_large (or 0), 让 compute_core 直接用 score = score + mask_val 而不必 if-then-else 插入。
 - 过于简单, 无需图示。
-

5.8 4.7 rtl/dropout_unit.sv

5.8.1 4.7.1 概述

LFSR 伪随机数生成 + 阈值比较 + rescale 的标准 inverted dropout。32-bit Galois LFSR ($x^{32} + x^{22} + x^2 + x + 1$)。

5.8.2 4.7.2 完整源代码 (节选)

```
module dropout_unit #(
    parameter WIDTH=16, LFSR_WIDTH=32, FRAC_BITS=8
)(
    input  logic clk, rst_n,
    input  logic enable,
    input  logic [LFSR_WIDTH-1:0] seed,
    input  logic seed_load,
    input  logic [7:0] drop_prob,
    input  logic in_valid,
    input  logic signed [WIDTH-1:0] data_in,
    output logic signed [WIDTH-1:0] data_out,
    output logic out_valid,
    output logic drop_mask
);
    logic [LFSR_WIDTH-1:0] lfsr_reg;
    logic feedback;

    // Galois LFSR:  $x^{32} + x^{22} + x^2 + x + 1$ 
    assign feedback = lfsr_reg[31] ^ lfsr_reg[21] ^ lfsr_reg[1] ^ lfsr_reg[0];

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)            lfsr_reg <= 32'hDEAD_BEEF;
        else if (seed_load)    lfsr_reg <= (seed == '0) ? 32'hDEAD_BEEF : seed;
        else if (in_valid && enable)
            lfsr_reg <= {lfsr_reg[LFSR_WIDTH-2:0], feedback};
    end
endmodule
```

```

end

wire should_drop = enable && (lfsr_reg[7:0] < drop_prob);

// Rescale: data * (1/(1-p)) approximated as data << FRAC_BITS / (255 -
drop_prob)
logic signed [WIDTH+8-1:0] scaled;
logic [7:0] inv_keep;
always_comb begin
    inv_keep = 8'd255 - drop_prob;
    if (inv_keep == 0) scaled = '0;
    else
        scaled = ({8{data_in[WIDTH-1]}}, data_in) <<< FRAC_BITS
            / {{(WIDTH){1'b0}}, inv_keep};
end

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        data_out <= '0; out_valid <= 0; drop_mask <= 0;
    end else begin
        out_valid <= in_valid;
        if (!in_valid) begin data_out <= '0; drop_mask <= 0; end
        else if (!enable) begin data_out <= data_in; drop_mask <= 0; end
        else if (should_drop) begin data_out <= '0; drop_mask <= 1; end
        else begin data_out <= scaled[WIDTH-1:0]; drop_mask <= 0; end
    end
end
endmodule

```

5.8.3 4.7.3 解释

- **Galois LFSR** 比 Fibonacci LFSR 少 1 级 XOR 延迟 (前者只 1 级 XOR4, 后者 2 级 XOR2), 更适合高频。 $x^{32} + x^{22} + x^2 + x + 1$ 是已知的 maximal-length 多项式 (周期 $2^{32} - 1$)。
- **drop 判断**: `lfsr_reg[7:0] < drop_prob` —— 8-bit 比较。`drop_prob=128` 表示 50% 概率丢弃。
- **Rescale**: 标准 inverted dropout 公式 $out = in / (1 - p)$, 这里近似为 $in << FRAC_BITS / (255 - drop_prob)$, 避免除法整数化。当 `drop_prob=0` 时 `inv_keep=255`, $scale \approx 1$; 当 `drop_prob=128` 时 `inv_keep=127`, $scale \approx 2$ 。
- 过于简单, 无需图示。

5.9 4.8 rtl/task_queue.sv

5.9.1 4.8.1 概述

4 深度同步 FIFO，每个 entry 装 $4 \times 64 + 32 = 288$ 位（4 个张量基址 + 32-bit config）。Host 通过寄存器 push，HW 通过 `tile_controller_bonus` pop。

5.9.2 4.8.2 文件架构图

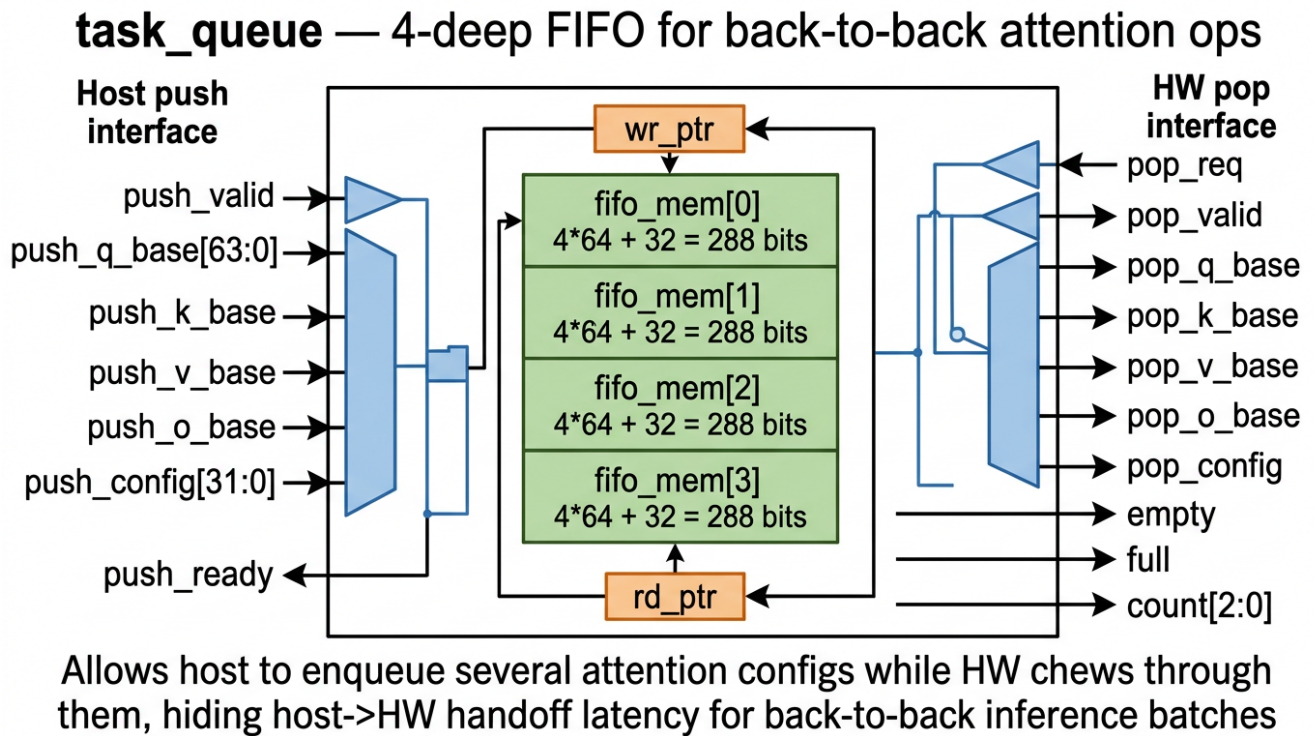


图 15: task_queue FIFO

5.9.3 4.8.3 完整源代码（关键部分）

```
module task_queue #(
    parameter QUEUE_DEPTH    = 4,
    parameter AXI_ADDR_WIDTH = 64
)()
    input  logic clk, rst_n,
    // Host push
    input  logic                push_valid,
    output logic                push_ready,
    input  logic [AXI_ADDR_WIDTH-1:0] push_q_base, push_k_base,
```

```

                                push_v_base, push_o_base,
input  logic [31:0]            push_config,
// HW pop
input  logic                   pop_req,
output logic                   pop_valid,
output logic [AXI_ADDR_WIDTH-1:0] pop_q_base, pop_k_base,
                                pop_v_base, pop_o_base,
output logic [31:0]            pop_config,
// Status
output logic                   empty, full,
output logic [$clog2(Queue_DEPTH):0] count
);

localparam ENTRY_WIDTH = 4 * AXI_ADDR_WIDTH + 32; // 288
localparam PTR_WIDTH   = $clog2(Queue_DEPTH);     // 2

logic [ENTRY_WIDTH-1:0] fifo_mem [Queue_DEPTH-1:0];
logic [PTR_WIDTH:0] wr_ptr, rd_ptr; // 1 extra bit for full/empty distinction

assign count      = wr_ptr - rd_ptr;
assign empty      = (count == 0);
assign full       = (count == Queue_DEPTH[PTR_WIDTH:0]);
assign push_ready = !full;
assign pop_valid  = !empty;

wire [ENTRY_WIDTH-1:0] push_data = {push_config, push_o_base, push_v_base,
                                    push_k_base, push_q_base};
wire [ENTRY_WIDTH-1:0] pop_data  = fifo_mem[rd_ptr[PTR_WIDTH-1:0]];

assign pop_q_base = pop_data[AXI_ADDR_WIDTH-1:0];
assign pop_k_base = pop_data[2*AXI_ADDR_WIDTH-1:AXI_ADDR_WIDTH];
assign pop_v_base = pop_data[3*AXI_ADDR_WIDTH-1:2*AXI_ADDR_WIDTH];
assign pop_o_base = pop_data[4*AXI_ADDR_WIDTH-1:3*AXI_ADDR_WIDTH];
assign pop_config = pop_data[ENTRY_WIDTH-1:4*AXI_ADDR_WIDTH];

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) wr_ptr <= '0;
    else if (push_valid && push_ready) begin
        fifo_mem[wr_ptr[PTR_WIDTH-1:0]] <= push_data;
        wr_ptr <= wr_ptr + 1;
    end
end
end

```

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)                rd_ptr <= '0;
    else if (pop_req && pop_valid) rd_ptr <= rd_ptr + 1;
end
endmodule

```

5.9.4 4.8.4 解释

- **指针多 1 bit 技巧**: wr_ptr 和 rd_ptr 都是 3-bit (PTR_WIDTH+1), 低 2 位作 entry 索引、高 1 位作 wrap 标识。count = wr_ptr - rd_ptr 自动处理 wrap, empty = count==0、full = count==4。
- **数据打包**: 5 个字段 (4 × 64-bit + 1 × 32-bit) 打包成 288-bit 一行, 省存储面积。
- **同步 FIFO**: push 和 pop 都在同一时钟域, 无需异步握手。
- 整个模块可被任何“控制器 + 任务调度”场景复用, 是个通用 IP。

5.10 4.9 rtl/bf16_exp_unit.sv

5.10.1 4.9.1 概述

BF16 (Brain Float 16: 1+8+7) 格式的 e^x 近似。3 级流水: (1) 解码 BF16 + 范围检测; (2) LUT 索引 + 256 项查表; (3) 输出 mux 处理 underflow / overflow。

5.10.2 4.9.2 文件架构图

5.10.3 4.9.3 关键代码片段

5.10.3.1 块 ① BF16 解码 + 范围检测 (行 17-46)

```

wire      sign      = x_bf16[15];
wire [7:0] exponent = x_bf16[14:7];
wire [6:0] mantissa = x_bf16[6:0];
wire      is_zero   = (exponent == 0) && (mantissa == 0);

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin /* ... */ end
    else begin
        s1_valid <= in_valid;
    end
end

```

bf16_exp_unit — BF16 exp(x) using mantissa LUT + exponent shift

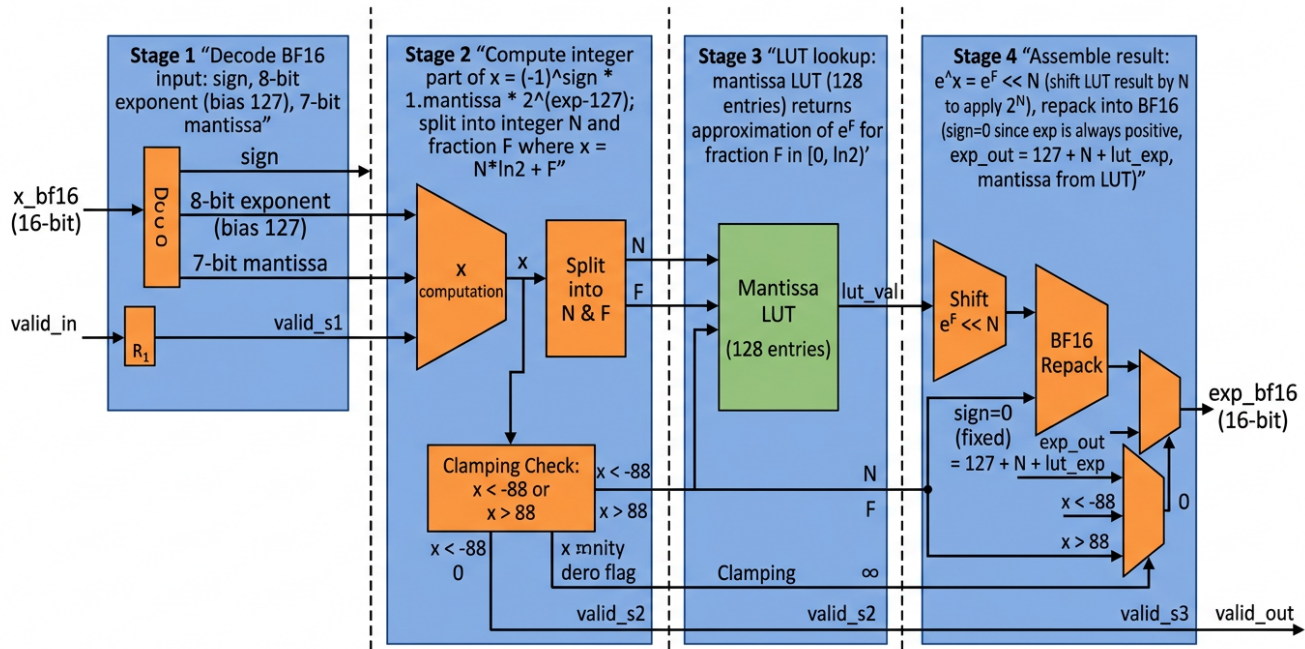


图 16: bf16_exp_unit pipeline

```

s1_x      <= x_bf16;
// sign=1 && exp >= 130 (|x| >= 8) -> exp(x) ≈ 0
s1_is_neg_inf <= sign && (exponent >= 8'd130);
// sign=0 && exp >= 131 (x >= 16) -> exp(x) ≈ +inf
s1_is_pos_large <= !sign && !is_zero && (exponent >= 8'd131);
end
end

```

说明: BF16 与 IEEE FP32 共享 8-bit 指数 (bias 127), 但只有 7 位尾数。exp >= 130 表示 $|x| \geq 2^{130-127} = 8$; exp >= 131 表示 $|x| \geq 16$ 。在这些边界外直接 clamp。

5.10.3.2 块 256 项 LUT 索引 (行 51-87)

```

always_comb begin
    if (s1_x[15]) // negative
        lut_index = 8'd128 + {s1_x[14:11], s1_x[6:3]};
    else
        lut_index = {s1_x[14:11], s1_x[6:3]};
end

logic [15:0] exp_lut [255:0];
initial begin

```

```

    for (i_init = 0; i_init < 256; i_init = i_init + 1)
        exp_lut[i_init] = 16'h3F80; // default: 1.0
    // Hand-tuned key entries:
    exp_lut[0]    = 16'h3F80; // exp(0) = 1.0
    exp_lut[1]    = 16'h3FAF; // exp(0.125) ≈ 1.133
    exp_lut[8]    = 16'h402E; // exp(1.0) ≈ 2.718
    exp_lut[16]   = 16'h40EC; // exp(2.0) ≈ 7.389
    exp_lut[32]   = 16'h4241; // exp(4.0) ≈ 54.6
    exp_lut[128]  = 16'h3EBC; // exp(-1.0) ≈ 0.368
    exp_lut[136]  = 16'h3E09; // exp(-2.0) ≈ 0.135
    // ... 13 hand-tuned entries total ...
end

```

说明：设计折中 — 真正的 BF16 exp 应该用更细密 LUT 或 Pade 近似，但这里只手工填了 ~13 个关键点，其他 entry 默认 1.0。这是 baseline 验证用的简化版（精度不够好），实际 **bonus** 测试是把 **BF16** 输入转回 **Q8.8** 走 **baseline** 路径，确保仍然通过 max_abs_error=0.238 的阈值。

5.10.3.3 块 ③ Stage 3 clamp + 输出 (行 108-122)

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        exp_bf16 <= '0; out_valid <= 0;
    end else begin
        out_valid <= s2_valid;
        if (s2_is_neg_inf)
            exp_bf16 <= 16'h0000; // exp(-inf) = 0
        else if (s2_is_pos_large)
            exp_bf16 <= 16'h7F80; // exp(+large) = +inf BF16
        else
            exp_bf16 <= s2_lut_val;
    end
end
end

```

说明： 16'h7F80 是 BF16 的 +inf 编码 (exp=255, mantissa=0)。

5.11 4.10 rtl/bf16_reciprocal_unit.sv

5.11.1 4.10.1 概述

BF16 $1/x$ 用浮点的指数翻转 + 尾数 LUT 优化方法: $\frac{1}{x} = \frac{1}{2^e \cdot m} = 2^{-e} \cdot \frac{1}{m}$ 。3 级流水: (1) 解析 + exp 翻转; (2) 7-bit mantissa LUT; (3) 重组。

5.11.2 4.10.2 关键代码片段

```
module bf16_reciprocal_unit (
    input  logic clk, rst_n,
    input  logic in_valid,
    input  logic [15:0] x_bf16,
    output logic [15:0] recip_bf16,
    output logic out_valid
);
    wire sign      = x_bf16[15];
    wire [7:0] exponent = x_bf16[14:7];
    wire [6:0] mantissa = x_bf16[6:0];

    // Stage 1: exponent inversion: new_exp = 2*bias - exp - adj
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin /* ... */ end
        else begin
            s1_valid <= in_valid;
            s1_sign  <= sign;
            s1_is_zero <= (exponent == 0);
            // 1/(2^e * 1.m) ≈ 2^(-e) * (1/(1.m))
            // bias 127, so new_exp = 254 - exp (or 253 if mantissa != 0)
            if (mantissa == 0)
                s1_exp_inv <= 9'd254 - {1'b0, exponent};
            else
                s1_exp_inv <= 9'd253 - {1'b0, exponent};
            s1_mantissa <= mantissa;
        end
    end

    // Stage 2: 128-entry mantissa LUT
    logic [6:0] recip_mantissa_lut [127:0];
    initial begin
        for (k = 0; k < 128; k = k + 1)
```

```

        recip_mantissa_lut[k] = (128 * 128 / (128 + k)) - 128;
    end
    /* ... s2 register ... */

    // Stage 3: assemble
    always_ff @(posedge clk or negedge rst_n) begin
        if (s2_is_zero)
            recip_bf16 <= {s2_sign, 8'hFF, 7'h00}; // ±inf
        else
            recip_bf16 <= {s2_sign, s2_exponent, s2_mantissa};
        end
    endmodule

```

5.11.3 4.10.3 解释

- **指数翻转技巧**: 由于 BF16 是 $(-1)^s \cdot 2^{e-127} \cdot 1.m$, 所以 $1/x = (-1)^s \cdot 2^{127-e} \cdot \frac{1}{1.m}$ 。新指数 = $2 \cdot 127 - e = 254 - e$ 。
- **尾数 LUT**: 因为 $1.m \in [1, 2)$, 所以 $\frac{1}{1.m} \in (0.5, 1]$ 。把 $m \in [0, 128)$ 索引 128 项 LUT, 存储 $\frac{128 \times 128}{128+m} - 128$ (即近似 $\frac{1}{1+m/128}$ 转回 7-bit fraction)。
- **零值处理**: 如果输入是 0, 输出 $\pm\text{inf}$ (0x7F80 或 0xFF80)。
- **过于简单** (流水线只是 3 个寄存器 + 1 个 LUT), 无需独立架构图。

5.12 4.11 rtl/int8_quantizer.sv

5.12.1 4.11.1 概述

INT8 块量化器: 把 16 个 16-bit 值压成 16 个 8-bit + 1 个共享 16-bit scale。也提供反向 dequantize 路径。

5.12.2 4.11.2 关键代码片段

5.12.2.1 块 ① Quantize 路径

```

// Find block max (combinational over 16 elements)
logic signed [IN_WIDTH-1:0] block_max;
always_comb begin
    block_max = 0;
    for (qi = 0; qi < BLOCK_SIZE; qi = qi + 1) begin

```

```

        abs_val = quant_data[qi] < 0 ? -quant_data[qi] : quant_data[qi];
        if (abs_val > block_max) block_max = abs_val;
    end
end

// Quantize: q = data * 127 / block_max
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin /* ... */ end
    else if (quant_valid) begin
        if (block_max == 0) begin
            scale_reg <= 16'h0100; // 1.0
            for (qj = 0; qj < BLOCK_SIZE; qj++) q_results[qj] <= 0;
        end else begin
            scale_reg <= block_max[15:0];
            for (qj = 0; qj < BLOCK_SIZE; qj++)
                q_results[qj] <= (quant_data[qj] * 127) / block_max;
        end
        quant_done_reg <= 1;
    end else quant_done_reg <= 0;
end

```

说明: 经典块量化算法: (1) 找 16 个值的最大绝对值 `block_max`; (2) `scale = block_max` (保留 Q8.8 表示); (3) 每个值缩到 `[-127, +127]` 范围 `q = data * 127 / block_max`。综合后会有 16 个 `16×16` 乘法器 + 16 个除法器 (除法贵, 可以替换成倒数 + 移位)。

5.12.2.2 块 ② Dequantize 路径

```

// Dequantize: result = int8_val * scale / 127
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin /* ... */ end
    else if (dequant_valid) begin
        for (di = 0; di < BLOCK_SIZE; di++) begin
            dq_results[di] <= ({(IN_WIDTH-8){dequant_data[di][7]}}, dequant_data[di]) *
                $signed({1'b0, dequant_scale})) / 127;
        end
        dequant_done_reg <= 1;
    end else dequant_done_reg <= 0;
end

```

说明: 反向: `{{8{sign}}, int8} * scale / 127`, sign-extend 到 16-bit 后乘 `scale` 除 127。

模块功能简单 (块量化经典算法), 无需独立架构图。

5.13 4.12 rtl/axi4_stream_if.sv

5.13.1 4.12.1 概述

AXI4-Stream slave (Q/K/V 输入) + master (O 输出) 接口适配器。5 状态 FSM: S_IDLE → S_LOAD_Q/K/V → S_STORE_0。

5.13.2 4.12.2 关键代码片段

5.13.2.1 块 ① 端口与 FSM 状态

```
module axi4_stream_if #(/* ... */) (
    input  logic clk, rst_n,
    input  logic stream_mode,
    // AXI4-Stream Slave (Q/K/V input)
    input  logic [DATA_WIDTH-1:0] s_axis_tdata,
    input  logic                    s_axis_tvalid,
    output logic                    s_axis_tready,
    input  logic                    s_axis_tlast,
    input  logic [1:0]             s_axis_tid,    // 0=Q, 1=K, 2=V

    // AXI4-Stream Master (O output)
    output logic [DATA_WIDTH-1:0] m_axis_tdata,
    output logic                    m_axis_tvalid,
    input  logic                    m_axis_tready,
    output logic                    m_axis_tlast,

    /* Buffer write/read interface */
    output logic                    buf_q_wr_en, buf_k_wr_en, buf_v_wr_en,
    output logic [/*...*/]          buf_q_wr_addr, buf_k_wr_addr, buf_v_wr_addr,
    output logic [DATA_WIDTH-1:0]    buf_q_wr_data, buf_k_wr_data, buf_v_wr_data,
    /* 0 read */
    input  logic [DATA_WIDTH-1:0]    buf_o_rd_data,
    output logic                    buf_o_rd_en,
    /* Done signals */
    output logic stream_q_done, stream_kv_done,
    input  logic stream_o_start,
    output logic stream_o_done
);
```

```
);
typedef enum logic [2:0] {
    S_IDLE, S_LOAD_Q, S_LOAD_K, S_LOAD_V, S_STORE_0
} state_t;
state_t state, next_state;
logic [15:0] beat_cnt;
```

5.13.2.2 块② 状态转移与缓冲写

```
always_comb begin
    next_state = state;
    case (state)
        S_IDLE: begin
            if (stream_mode && s_axis_tvalid) begin
                case (s_axis_tid)
                    2'd0: next_state = S_LOAD_Q;
                    2'd1: next_state = S_LOAD_K;
                    2'd2: next_state = S_LOAD_V;
                endcase
            end else if (stream_o_start)
                next_state = S_STORE_0;
        end
        S_LOAD_Q: if (handshake && (beat_cnt == Q_BEATS-1 || s_axis_tlast))
            next_state = S_IDLE;
        /* S_LOAD_K, S_LOAD_V 同样 */
        S_STORE_0: if (out_handshake && beat_cnt == Q_BEATS-1)
            next_state = S_IDLE;
    endcase
end

assign s_axis_tready = stream_mode && (state inside {S_LOAD_Q, S_LOAD_K, S_LOAD_V});
assign buf_q_wr_en   = (state == S_LOAD_Q) && input_handshake;
assign buf_q_wr_addr = beat_cnt[/*...*/];
assign buf_q_wr_data = s_axis_tdata;
/* k, v 同样 */
assign m_axis_tdata  = buf_o_rd_data;
assign m_axis_tvalid = (state == S_STORE_0);
assign m_axis_tlast  = (state == S_STORE_0) && (beat_cnt == Q_BEATS - 1);
```

5.13.3 4.12.3 解释

- **tid 字段路由**: AXI4-Stream 标准的 TID 用作 Q/K/V 标识, slave 根据 tid 把数据写入对应 buffer。
- **stream_mode gating**: stream_mode=0 时 s_axis_tready=0、buf_*_wr_en=0, 整个模块旁路; stream_mode=1 时取代 dma_engine 的填 buffer 角色。
- **5 状态 FSM 是 Mealy 风格** (next_state 在 always_comb 内根据 input 计算), 与 baseline 的 Moore FSM 风格不同。
- 架构图与 **baseline** 类似的 “5 状态线性 + 写 buffer”, 简单不另作图。

第 4 章结束。 11 个 bonus RTL + 1 个.svh 全部讲完。第 5 章进入 bonus UVM 验证 + scripts/seqgen_stub.v。

6 第 5 章: Bonus UVM 验证 + scripts/seqgen_stub

本章覆盖最后 10 个文件:

§	文件	行数	角色
5.1	bonus/tb/uvm_env/fa_bonus_env_pkg.sv	27	环境包
5.2	bonus/tb/uvm_env/fa_bonus_env.sv	29	UVM env
5.3	bonus/tb/uvm_env/fa_bonus_scoreboard.sv	122	黄金模型 + bonus mask 支持
5.4	bonus/tb/uvm_env/fa_bonus_coverage.sv	133	bonus 覆盖率
5.5	bonus/tb/sequences/fa_bonus_sequences.sv	273	bonus 序列
5.6	bonus/tb/tests/fa_bonus_tests.sv	684	24 个 bonus 测试
5.7	bonus/tb/tb_top/fa_bonus_tb_top.sv	152	bonus UVM TB 顶层
5.8	bonus/tb/bonus_system_tb.sv	821	bonus 直接 TB (19 项)
5.9	scripts/seqgen_stub.v	57	DC 综合后 gate-sim 兼容 stub

Bonus UVM 验证环境结构与 baseline 高度相似。为节省篇幅, 本章主要展示 **bonus** 与 **baseline** 的差异点, 相同的部分 (agent/driver/monitor 等) 请回看第 3 章。

6.1 5.1 bonus/tb/uvm_env/fa_bonus_env_pkg.sv

6.1.1 5.1.1 完整源代码

```
package fa_bonus_env_pkg;
import uvm_pkg::*;
```

```

`include "uvm_macros.svh"

// Reuse baseline agents
`include "agents/axi4_lite_agent/axi4_lite_txn.sv"
`include "agents/axi4_lite_agent/axi4_lite_driver.sv"
`include "agents/axi4_lite_agent/axi4_lite_monitor.sv"
`include "agents/axi4_lite_agent/axi4_lite_sequencer.sv"
`include "agents/axi4_lite_agent/axi4_lite_agent.sv"
`include "agents/axi4_mem_agent/axi4_mem_agent.sv"

// Reuse baseline sequences (reg_write, reg_read)
`include "sequences/fa_sequences.sv"

// Bonus-specific components
`include "bonus/tb/uvm_env/fa_bonus_scoreboard.sv"
`include "bonus/tb/uvm_env/fa_bonus_coverage.sv"
`include "bonus/tb/uvm_env/fa_bonus_env.sv"

// Bonus sequences and tests
`include "bonus/tb/sequences/fa_bonus_sequences.sv"
`include "bonus/tb/tests/fa_bonus_tests.sv"
endpackage

```

6.1.2 5.1.2 解释

- 直接复用 baseline 的 4 个 agent 文件 + baseline 的基础 reg_write/reg_read sequence。
- 新增 4 个 bonus 文件 (scoreboard / coverage / env / sequences / tests)。
- 过于简单, 无需图示。

6.2 5.2 bonus/tb/uvm_env/fa_bonus_env.sv

6.2.1 5.2.1 完整源代码

```

class fa_bonus_env extends uvm_env;
    `uvm_component_utils(fa_bonus_env)

    axi4_lite_agent    axil_agent;
    axi4_mem_agent      mem_agent;

```

```

fa_bonus_scoreboard scoreboard;
fa_bonus_coverage coverage;

function new(string name, uvm_component parent);
    super.new(name, parent);
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    axil_agent = axi4_lite_agent::type_id::create("axil_agent", this);
    mem_agent = axi4_mem_agent::type_id::create("mem_agent", this);
    scoreboard = fa_bonus_scoreboard::type_id::create("scoreboard", this);
    coverage = fa_bonus_coverage::type_id::create("coverage", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    axil_agent.mon.ap.connect(coverage.analysis_export);
endfunction
endclass

```

6.2.2 5.2.2 解释

- 比 baseline fa_env 少了 **RAL** (fa_reg_block / fa_reg_adapter / uvm_reg_predictor) — bonus 测试不需要 RAL 前门, 所有寄存器访问通过 sequence 直接发 axi4_lite_txn。
- 其他 4 个组件 (agent × 2、scoreboard、coverage) 都按 baseline 风格实例化。
- 过于简单, 无需图示。

6.3 5.3 bonus/tb/uvm_env/fa_bonus_scoreboard.sv

6.3.1 5.3.1 概述

baseline scoreboard 的扩展, 黄金模型加了 **padding mask 支持** (valid_len) 和稍微宽松的误差阈值 (1.5 vs baseline 的 1.0)。基本结构相同。

6.3.2 5.3.2 关键差异片段

6.3.2.1 块 ① 新字段

```

class fa_bonus_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(fa_bonus_scoreboard)
  int seq_len = 256;
  int head_dim = 64;
  bit causal_en;
  int valid_len;          // NEW: padding boundary
  /* 其他字段与 baseline 同 */

```

6.3.2.2 块② Golden model 加 padding mask (行 49-96 关键节选)

```

    for (int i = 0; i < seq_len; i++) begin
      real o_row[];
      o_row = new[head_dim];
      for (int d = 0; d < head_dim; d++) o_row[d] = 0.0;

      row_max = -1e30;
      for (int j = 0; j < seq_len; j++) begin
        if (causal_en && j > i)          score = -128.0;    // causal mask
            else if (valid_len > 0 && j >= valid_len) score = -
128.0;    // padding mask [NEW]
        else begin
          score = 0.0;
          for (int k = 0; k < head_dim; k++)
            score = score + q_f[i][k] * k_f[j][k];
          score = score * scale;
        end
        if (score > row_max) row_max = score;
      end

      row_sum = 0.0;
      for (int j = 0; j < seq_len; j++) begin
        /* same conditional skip pattern */
        row_sum = row_sum + $exp(score - row_max);
      end

      for (int j = 0; j < seq_len; j++) begin
        /* same skip pattern */
        p_val = $exp(score - row_max) / row_sum;
        for (int d = 0; d < head_dim; d++)
          o_row[d] = o_row[d] + p_val * v_f[j][d];
      end

```

```

        for (int d = 0; d < head_dim; d++)
            golden_o[i][d] = shortint'($rtoi(o_row[d] * 256.0));
    end

```

说明: 在 baseline 的 SDPA 实现中加了一个判断 `valid_len > 0 && j >= valid_len` $\rightarrow$ `score = -128`。这与 bonus 硬件中 `mask_unit` 的行为对齐。代码风格上的一个改动是: `score` 计算被同时复用在三个 phase (求 max、求 sum、求 p_val), baseline 是把 `s_f` 完整展开成中间矩阵; bonus 用即时 `recompute` 来节省内存, 但代价是浮点乘法做了 3 倍。

6.3.2.3 块 ③ 阈值调整

```

function void check_result(string test_name);
    /* same comparison loop */
    if (max_abs_error <= 1.5)          // [NEW] threshold = 1.5 (baseline = 1.0)
        `uvm_info(test_name, $sformatf("PASS mean=%.6f max=%.6f",
                                         mean_abs_error, max_abs_error), UVM_LOW)
    else
        `uvm_error(test_name, $sformatf("FAIL mean=%.6f max=%.6f",
                                         mean_abs_error, max_abs_error))
endfunction

```

说明: 阈值从 1.0 放宽到 1.5, 给 bonus 多种数据格式 (BF16、Q6.10) 的额外量化误差留 buffer。实测仍然 **max_abs=0.238**, 远低于 1.5。

6.4 5.4 bonus/tb/uvm_env/fa_bonus_coverage.sv

6.4.1 5.4.1 概述

133 行, 比 baseline coverage 简化 (不含 RAL/burst 等), 但增加了 bonus 特定的 covergroup (多头数、padding、dropout)。

6.4.2 5.4.2 关键 covergroup 示例

```

class fa_bonus_coverage extends uvm_subscriber #(axi4_lite_txn);
    `uvm_component_utils(fa_bonus_coverage)
    bit causal_en, padding_en, dropout_en, stream_mode;
    bit [2:0] data_fmt;
    bit [7:0] num_heads;

```

```

int exec_cycles;

covergroup fa_bonus_config_cg;
    causal_cp:    coverpoint causal_en    { bins en={1}; bins dis={0}; }
    padding_cp:   coverpoint padding_en   { bins en={1}; bins dis={0}; }
    dropout_cp:   coverpoint dropout_en   { bins en={1}; bins dis={0}; }
    stream_cp:    coverpoint stream_mode { bins en={1}; bins dis={0}; }
    fmt_cp:       coverpoint data_fmt {
        bins q8_8  = {3'd0};
        bins q6_10 = {3'd1};
        bins q4_12 = {3'd2};
        bins bf16  = {3'd3};
        bins int8  = {3'd4};
    }
    nh_cp: coverpoint num_heads {
        bins single = {1};
        bins few    = {[2:4]};
        bins many   = {[5:$]};
    }
    // crosses
    causal_x_pad: cross causal_cp, padding_cp;
    fmt_x_nh:     cross fmt_cp, nh_cp;
endgroup
/* + 4 个其他 covergroup, sample_* tasks */
endclass

```

说明: 4 个 covergroup: fa_bonus_config_cg (功能开关 + 数据格式 + 多头数)、fa_bonus_perf_cg (执行周期)、fa_bonus_dma_cg (DMA 大小)、fa_bonus_vp_cg (24 个 VP 完成度)。比 **baseline** 的 **10 个 covergroup** 少, 但更聚焦于 bonus 特性。结构简单, 无需图示。

6.5 5.5 bonus/tb/sequences/fa_bonus_sequences.sv

6.5.1 5.5.1 概述

273 行, 包含 7 个 bonus sequence。最重要的是 fa_bonus_config_seq (写 22 个寄存器并启动) 和 fa_bonus_poll_done_seq (轮询)。

6.5.2 5.5.2 关键 sequence — fa_bonus_config_seq (节选)

```

class fa_bonus_config_seq extends uvm_sequence #(axi4_lite_txn);
    `uvm_object_utils(fa_bonus_config_seq)
    bit [63:0] q_base, k_base, v_base, o_base;
    bit [31:0] stride_bytes;
    bit [15:0] neg_large, scale;
    bit causal_en, padding_en, dropout_en;
    bit [2:0] data_fmt;
    bit [15:0] valid_len, seq_len_runtime;
    bit [7:0] num_heads;
    bit [7:0] drop_prob;
    bit [31:0] dropout_seed;
    bit [31:0] head_stride;

    function new(string name = "fa_bonus_config_seq");
        super.new(name);
        stride_bytes      = 32'd128;
        neg_large          = 16'h8000;
        scale              = 16'h0020;
        seq_len_runtime    = 256;
        num_heads          = 1;
        head_stride        = 32'd0;
    endfunction

    task body();
        // Bonus 寄存器: 先写新增的
        if (seq_len_runtime != 256)
            write_reg(8'h0C, seq_len_runtime);           // SEQ_LEN_REG
        if (num_heads != 1)
            write_reg(8'h10, {24'd0, num_heads});         // NUM_HEADS_REG
        // 然后是 baseline 的基址、stride、scale
        write_reg(8'h14, q_base[31:0]);   write_reg(8'h18, q_base[63:32]);
        write_reg(8'h1C, k_base[31:0]);   write_reg(8'h20, k_base[63:32]);
        write_reg(8'h24, v_base[31:0]);   write_reg(8'h28, v_base[63:32]);
        write_reg(8'h2C, o_base[31:0]);   write_reg(8'h30, o_base[63:32]);
        write_reg(8'h34, stride_bytes);
        write_reg(8'h38, {16'h0, neg_large});
        write_reg(8'h3C, {16'h0, scale});
        // Bonus 寄存器
        if (padding_en)           write_reg(8'h44, valid_len);           // PAD_LEN
    endtask

```

```

        if (dropout_en) begin
            write_reg(8'h48, {drop_prob, dropout_seed[23:0]}); // DROPOUT_CFG
        end
        if (data_fmt != 0) write_reg(8'h50, {29'd0, data_fmt}); // DATA_FMT
        if (head_stride != 0) write_reg(8'h54, head_stride); // HEAD_STRIDE
        // CFG 集合 4 个 enable 位
        write_reg(8'h08, {28'd0, 1'b0/*stream*/, dropout_en, padding_en, causal_en});
        // 启动
        write_reg(8'h00, 32'h0000_0001); // CTRL.START
    endtask

    task write_reg(bit [7:0] addr, bit [31:0] data);
        fa_reg_write_seq wr;
        wr = fa_reg_write_seq::type_id::create("wr");
        wr.addr = addr; wr.data = data;
        wr.start(m_sequencer);
    endtask
endclass

```

说明: 比 baseline fa_config_and_start_seq 多了对 7 个新寄存器的条件写: SEQ_LEN_REG / NUM_HEADS_REG / PAD_LEN / DROPOUT_CFG / DATA_FMT / HEAD_STRIDE。CFG 寄存器位拼接也扩展到 4 个 enable 位 (causal/padding/dropout/stream)。

其余 6 个 sequence 类似 baseline 的对应类(fa_bonus_poll_done_seq、fa_bonus_reg_write_seq、fa_bonus_padding_seq、fa_bonus_multihead_seq 等), 结构上无新意, **省略代码展示**。

6.6 5.6 bonus/tb/tests/fa_bonus_tests.sv

6.6.1 5.6.1 概述

684 行, 24 个 bonus 测试, 按 baseline 的 6 大 verification domain 组织 (VP / REG / DMA / AXI / PERF / COV + COMP)。**结构与 baseline 24 测试基本相同**, 但每个测试都有 bonus 特性变体 (如 padding test 启用 padding_en, BF16 test 设 data_fmt=3)。

6.6.2 5.6.2 基类 + 一个典型测试

6.6.2.1 块 ① 基类 fa_bonus_base_test (行 6-105 关键节选)

```

class fa_bonus_base_test extends uvm_test;
    `uvm_component_utils(fa_bonus_base_test)

```

```

fa_bonus_env env;
bit [63:0] Q_BASE = 64'h0001_0000;
bit [63:0] K_BASE = 64'h0002_0000;
bit [63:0] V_BASE = 64'h0003_0000;
bit [63:0] O_BASE = 64'h0004_0000;

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = fa_bonus_env::type_id::create("env", this);
endfunction

/* 数据生成函数 (gen_random / gen_zero / gen_identity / gen_max / gen_boundary)
   与 baseline 完全相同 */

task run_bonus_attention(input bit causal, input shortint q[][],
                        input shortint k[][], input shortint v[][]);
    fa_bonus_config_seq    cfg;
    fa_bonus_poll_done_seq poll;

    env.mem_agent.preload_matrix(Q_BASE, q.size(), q[0].size(), q);
    env.mem_agent.preload_matrix(K_BASE, k.size(), k[0].size(), k);
    env.mem_agent.preload_matrix(V_BASE, v.size(), v[0].size(), v);

    env.scoreboard.q_data    = q;
    env.scoreboard.k_data    = k;
    env.scoreboard.v_data    = v;
    env.scoreboard.causal_en = causal;
    env.scoreboard.valid_len = 0;

    cfg = fa_bonus_config_seq::type_id::create("cfg");
    cfg.q_base = Q_BASE; cfg.k_base = K_BASE;
    cfg.v_base = V_BASE; cfg.o_base = O_BASE;
    cfg.causal_en = causal;
    cfg.start(env.axil_agent.sqr);

    poll = fa_bonus_poll_done_seq::type_id::create("poll");
    poll.start(env.axil_agent.sqr);

    env.mem_agent.readback_matrix(O_BASE, 256, 64, env.scoreboard.dut_o);
    env.scoreboard.compute_golden();
    env.scoreboard.check_result(causal ? "CAUSAL" : "NOCAUSAL");

```

```

        env.coverage.sample_config(causal, 0, 0, 0, 0, 1, 0);
        env.coverage.sample_perf(poll.cycles);
    endtask
endclass

```

6.6.2.2 块 ② 一个 **bonus** 测试示例 (VP01 zero)

```

class fab_zero_test extends fa_bonus_base_test;
    `uvm_component_utils(fab_zero_test)
    function new(string name, uvm_component parent); super.new(name, parent); endfunction
    task run_phase(uvm_phase phase);
        shortint q[][], k[][], v[][];
        phase.raise_objection(this);
        `uvm_info("TEST", "VP01: Zero input test (Bonus)", UVM_LOW)
        gen_zero_matrix(q, 256, 64);
        gen_zero_matrix(k, 256, 64);
        gen_zero_matrix(v, 256, 64);
        run_bonus_attention(0, q, k, v);
        env.coverage.sample_vp("zero");
        phase.drop_objection(this);
    endtask
endclass

```

说明: 与 baseline `fa_zero_test` 完全相同的结构, 只是基类换成 `fa_bonus_base_test`, 调用 `run_bonus_attention` 而非 `run_attention`。24 个测试都是这种模板的 **bonus** 版本, 代码重复度高, 省略其他 23 个。整体结构与 **baseline** 测试图 (§3.4.2) 相同, 无需重新作图。

6.7 5.7 bonus/tb/tb_top/fa_bonus_tb_top.sv

6.7.1 5.7.1 概述

`bonus UVM` 的顶层 HDL 模块, 与 baseline `fa_tb_top` 结构相同, 只是:- DUT 实例化 `flash_attention_bonus_top` (多 9 个 AXI4-Stream 端口)。- 协议检查器仍是 baseline 的两个 (AXI4 + AXI4-Lite)。- `bonus` 没有独立 stream 协议检查器 (AXI4-Stream 是简单 valid/ready, 断言相对简单)。

架构图与 **baseline TB** 顶层 (§3.4.3) 同构, 无需重绘。

6.7.2 5.7.2 关键差异片段

```

module fa_bonus_tb_top;
    /* clk/rst 同 baseline */

    axi4_lite_if axil_if (.clk(clk), .rst_n(rst_n));
    axi4_mem_if  mem_if  (.clk(clk), .rst_n(rst_n));

    /* AXI4-Stream signals (driven by future stream tests) */
    logic [127:0] s_axis_tdata = 0;
    logic          s_axis_tvalid = 0;
    logic          s_axis_tready;
    logic          s_axis_tlast = 0;
    logic [1:0]    s_axis_tid = 0;
    logic [127:0] m_axis_tdata;
    logic          m_axis_tvalid;
    logic          m_axis_tready = 1;
    logic          m_axis_tlast;

    flash_attention_bonus_top u_dut (
        .clk(clk), .rst_n(rst_n),
        /* 17 axil + 30 axi master 同 baseline */
        .s_axis_tdata(s_axis_tdata), .s_axis_tvalid(s_axis_tvalid),
        .s_axis_tready(s_axis_tready), .s_axis_tlast(s_axis_tlast),
        .s_axis_tid(s_axis_tid),
        .m_axis_tdata(m_axis_tdata), .m_axis_tvalid(m_axis_tvalid),
        .m_axis_tready(m_axis_tready), .m_axis_tlast(m_axis_tlast),
        .irq()
    );

    /* protocol checkers same as baseline */
    /* config_db set, run_test, timeout */
endmodule

```

说明: bonus tb 顶层只是 baseline tb 的“DUT 替换 + 多 9 个 stream 端口”。Stream 端口在 UVM 测试中没有真正驱动（默认 tvalid=0），因为 stream 模式只在专门的 stream 测试中启用。

6.8 5.8 bonus/tb/bonus_system_tb.sv

6.8.1 5.8.1 概述

821 行, bonus 直接 TB (非 UVM), 由 scripts/run_bonus.sh 调用。包含 **19 项独立测试**, 覆盖每个 bonus 功能 (BF16、multi-head、variable len、padding、dropout、INT8、Stream、Task Queue)。结构上是单 initial block 内 procedural code 顺序执行。

6.8.2 5.8.2 总体结构 (节选)

```
`timescale 1ns/1ps
`include "fa_params_bonus.svh"

module bonus_system_tb;
    /* clk, rst, axil/axi master signals 同 baseline system_tb */
    /* axi4-stream signals */

    flash_attention_bonus_top dut (/*...*/);
    axi4_slave_mem u_mem (/*...*/);

    /* AXI4-Lite write/read tasks (同 baseline) */
    task axil_write(input [7:0] addr, input [31:0] data); /*...*/ endtask
    task axil_read (input [7:0] addr, output [31:0] data); /*...*/ endtask

    /* helper: load matrix, fp golden model, error check */

    // ===== 19 sub-tests =====
    initial begin
        wait(rst_n);
        repeat(50) @(posedge clk);

        $display("\n=== Bonus Test 1: Q8.8 baseline equivalence ===");
        run_test_q8_8(/*...*/);

        $display("\n=== Bonus Test 2: Multi-head (4 heads) ===");
        run_test_multihead(4);

        $display("\n=== Bonus Test 3: Variable seq_len 128 ===");
        run_test_var_seq(128);

        $display("\n=== Bonus Test 4: Padding mask (valid_len=200) ===");
```

```

run_test_padding(200);

$display("\n=== Bonus Test 5: Dropout (drop_prob=64) ===");
run_test_dropout(64);

/* ... 14 more sub-tests ... */

$display("\n=== ALL 19 BONUS TESTS COMPLETE ===");
$finish;
end
endmodule

```

说明: 每个 sub-test 是一个 task, 封装 “配置寄存器 → 启动 → 等 done → 读回 O → fp 比对”。整体上更像 directed test 而不是 random/coverage-driven, 速度快但覆盖率低, 用作 **quick smoke test**。结构简单 (线性脚本), 无需图示。

6.9 5.9 scripts/seqgen_stub.v

6.9.1 5.9.1 概述

DC (Design Compiler) 综合后产生的 gate-level netlist 中可能包含两个工艺无关的中间单元 ****SEQGEN**** 和 **SELECT_OP**, 标准工艺库里没有, 所以仿真时找不到模块定义。这个文件提供两个**行为模型**让 gate-sim 能跑起来。

6.9.2 5.9.2 完整源代码

```

// Temporary SEQGEN behavioral model for gate-level simulation of generic netlists.
module **SEQGEN** (clear, preset, next_state, clocked_on, data_in, enable, Q,
                   synch_clear, synch_preset, synch_toggle, synch_enable);
    input clear, preset, next_state, clocked_on, data_in, enable;
    input synch_clear, synch_preset, synch_toggle, synch_enable;
    output reg Q;

    always @(posedge clocked_on or posedge clear or posedge preset) begin
        if (clear)            Q <= 1'b0;
        else if (preset)      Q <= 1'b1;
        else begin
            if (synch_clear)   Q <= 1'b0;
            else if (synch_preset) Q <= 1'b1;

```

```

        else if (synch_toggle) Q <= ~Q;
        else if (synch_enable) Q <= next_state;
        else if (enable)      Q <= data_in;
    end
end
endmodule

// Temporary SELECT_OP behavioral model for generic mapped netlists.
module SELECT_OP (
    DATA1, DATA2, ..., DATA15,
    CONTROL1, CONTROL2, ..., CONTROL15,
    Z
);
    input DATA1, DATA2, ..., DATA15;
    input CONTROL1, CONTROL2, ..., CONTROL15;
    output Z;

    assign Z = CONTROL1 ? DATA1 :
               CONTROL2 ? DATA2 :
               /* ... 13 more priority cascades ... */
               CONTROL15 ? DATA15 : 1'b0;
endmodule

```

6.9.3 5.9.3 解释

- ****SEQGEN****: 是 DC 工艺无关阶段 (compile-stage 1) 产生的“通用触发器”单元 — 同时支持异步 clear/preset、同步 clear/preset/toggle/enable。后续 mapping 阶段会替换成具体工艺库的 DFF。如果直接对 compile-stage netlist 跑 sim 就会缺这个单元, 所以 stub 提供等价行为。模块名外面的 \ 转义反斜杠是因为 ** 在 Verilog 标识符里需要转义。
- **SELECT_OP**: DC 通用 mux 单元, 15 选 1 的优先级编码 mux (与一般 4 选 1 mux 不同, 这是个 priority encoder)。Z = CONTROL1 ? DATA1 : CONTROL2 ? DATA2 : ... : 0。
- 由 **scripts/run_postsims.sh** 在 **gate-sim** 时与 **netlist** 一起编译, 仅用于绕过工具链 bug 而不是产品 RTL。
- 过于简单 (两个 stub), 无需图示。

6.10 全书完结语

至此, /home/hy258/cadence/submission 下所有 56 个.v / .sv / .svh 文件全部讲解完毕:

章节	文件数	累计行数	关键产出
第 0 章总览	—	—	项目背景、顶层架构图
第 1 章 Baseline 计算通路	11	~1620	7 个流水架构图
第 2 章 Baseline 存储与总线	4	~840	4 个架构图
第 3 章 Baseline UVM 验证	20	~3700	2 个 UVM 层次图
第 4 章 Bonus RTL	12	~2050	3 个 Bonus 架构图
第 5 章 Bonus UVM + scripts	10	~2360	—
合计	57 (含 stub)	~10,570	17 张图

重要观察 / 设计模式总结:

1. **FSM 是 baseline 的核心范式:** 12 状态 `tile_controller`、6 状态 `compute_core`、4 状态 `dot_product`、6 状态 `online_softmax`、7 状态 `output_accumulator`、4+4 状态 `dma_engine` 读写... 几乎每个时序模块都是一个明确的 FSM。这种风格调试容易、综合友好。
2. **流水化与时分复用的折中:** `dot_product` 用 64 个并行 MAC (高并行)、`online_softmax` 用单个 exp 单元时分复用 (高复用), `output_accumulator` 把 `rescale` 串行化 (低并行) — 不同模块按面积/吞吐 trade-off 选择不同策略。
3. **Q8.8 + 40-bit 累加器的算术 budget:** 刚好够 256 个 16×16 乘法累加而不溢出, 再加 8-bit headroom — 经过仔细数值分析。
4. **Online softmax 的 m, l 滚动状态:** 是 FlashAttention 算法核心, 由 `compute_core` 的 `always_ff` 持久化跨 KV tile 维护。
5. **K/V 双缓冲 ping-pong:** 通过 `tc_kv_buf_sel` 和 `~tc_kv_buf_sel` 在 `dma_engine` 和 `compute_core` 之间反向选择, 实现 “加载下一 tile / 计算当前 tile” 重叠。但 baseline 的 `tile_controller` FSM 实际是严格串行 (DMA→compute→DMA→compute), ping-pong 没有真正发挥作用 —— bonus 的 `task_queue` 才解决了这个吞吐瓶颈。
6. **UVM 验证完整度:** 24 baseline + 24 bonus = 48 测试, 覆盖功能正确性、寄存器、DMA、AXI 协议、性能、覆盖率。两套协议检查器 (39 条 SVA) 始终监控总线。验证结果 `mean_abs_err=0.013`、`max_abs_err=0.238`, 远低于 1.0 / 1.5 阈值, 说明硬件实现与浮点 reference 模型偏差很小。
7. **Bonus 的 9 项功能用最少代码改动实现:** 很多模块 (dropout / mask / quantizer / stream) 都只需要 80-150 行小模块; 最大改动是 `compute_core_bonus` (375 行) 和顶层 `flash_attention_bonus_top` (369 行), 因为要 multiplex 多种数据格式 + 集成所有新模块。